HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

TIỂU LUẬN MÔN HỌC
HỆ THỐNG THÔNG MINH

TRÍ TUỆ NHÂN TẠO VÀ CÁC MÔ HÌNH HỌC MÁY HIỆN ĐẠI

(Tiểu luận 03: Mở đầu – Chương 5)

Sinh viên thực hiện: Nguyễn Tự Kiên
Mã sinh viên: B23DCCN465
Nhóm lớp: CT01
Nhóm tiểu luận: 07

Hà Nội, tháng 10 năm 2026

MỤC LỤC

MỞ ĐẦU **1**
1. Lý do chọn đề tài 1
2. Mục tiêu nghiên cứu 2
3. Đối tượng và phạm vi nghiên cứu 3
4. Phương pháp nghiên cứu 3
5. Ý nghĩa khoa học và thực tiễn 4
6. Bố cục của tiểu luận 4
CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN CỦA TRÍ TUỆ NHÂN TẠO **6**
1.1. Khái niệm trí tuệ nhân tạo 6
1.2. Những nền tảng tiền đề (trước năm 1956) 7
1.3. Sự ra đời và giai đoạn lạc quan (1956 – đầu thập niên 1970) 8
1.4. Mùa đông AI thứ nhất (khoảng 1974 – 1980) 9
1.5. Hệ chuyên gia và mùa đông AI thứ hai (1980 – đầu thập niên 1990) 9
1.6. Sự trỗi dậy của học máy thống kê (thập niên 1990 – 2000) 10
1.7. Kỷ nguyên học sâu (từ 2006 – 2012 đến nay) 11
1.8. AI tạo sinh và mô hình ngôn ngữ lớn (từ 2020 đến nay) 12
1.9. Trạng thái AI đương đại (2025 – 2026) 14
1.9.1. Mô hình đa phương thức bản địa *14*
1.9.2. AI tác tử (Agentic AI) *14*
1.9.3. Mô hình suy luận và mô hình nhỏ *15*
1.10. Những thách thức đặt ra 15
1.11. Tiểu kết chương 1 16
CHƯƠNG 2. CÁC KỸ THUẬT HỌC MÁY CƠ BẢN **17**
2.1. Khái niệm học máy 17
2.2. Các dạng học máy 18
2.2.1. Học có giám sát *18*
2.2.2. Học không giám sát *18*
2.2.3. Học bán giám sát và học tự giám sát *18*
2.2.4. Học tăng cường *18*
2.3. Quy trình xây dựng một mô hình học máy 18
2.4. Tiền xử lý dữ liệu 19
2.5. Đánh giá mô hình 20
2.5.1. Chia tập dữ liệu và kiểm định chéo *20*

2.5.2. Thước đo cho bài toán hồi quy *20*
2.5.3. Thước đo cho bài toán phân loại *20*
2.6. Tối ưu hóa bằng hạ gradient 21
2.7. Hồi quy tuyến tính 22
2.8. Hồi quy logistic 22
2.9. K láng giềng gần nhất (KNN) 23
2.10. Naive Bayes 24
2.11. Cây quyết định và các phương pháp tổ hợp 24
2.11.1. Cây quyết định *24*
2.11.2. Rừng ngẫu nhiên và boosting *25*
2.12. Máy vector hỗ trợ (SVM) 25
2.12.1. SVM cho bài toán phân loại *25*
2.12.2. Hồi quy vector hỗ trợ (SVR) *26*
2.13. Gom cụm K-means 27
2.14. Giảm chiều dữ liệu với PCA 27
2.15. Quá khớp, thiếu khớp và điều chuẩn 28
2.16. So sánh các thuật toán 29
2.17. Từ học máy truyền thống đến mạng nơ-ron 29
2.17.1. Perceptron và thuật toán học perceptron (PLA) *30*
2.17.2. Perceptron nhiều lớp và lan truyền ngược *30*
2.18. Thực nghiệm: hồi quy softmax trên ba bộ dữ liệu 31
2.18.1. Dữ liệu và thiết lập *31*
2.18.2. Cài đặt *32*
2.18.3. Kết quả và nhận xét *32*
2.19. Tiểu kết chương 2 34
CHƯƠNG 3. MẠNG NƠ-RON TÍCH CHẬP (CNN) **35**
3.1. Động lực và nguồn gốc của CNN 35
3.1.1. Hạn chế của mạng kết nối đầy đủ với dữ liệu ảnh *35*
3.1.2. Nguồn cảm hứng sinh học và lịch sử *35*
3.2. Phép tích chập 36
3.2.1. Định nghĩa *36*
3.2.2. Ví dụ tính toán *36*
3.2.3. Ba ý tưởng then chốt *37*
3.3. Bước trượt (stride) và đệm (padding) 37
3.4. Hàm kích hoạt 38

3.5. Lớp gộp (pooling) 38
3.6. Kiến trúc tổng quát của một CNN 39
3.7. Huấn luyện CNN 39
3.7.1. Hàm mất mát và lan truyền ngược 39
3.7.2. Chuẩn hóa theo lô (Batch Normalization) 40
3.7.3. Dropout và các kỹ thuật điều chuẩn 40
3.8. Tăng cường dữ liệu 40
3.9. Học chuyển giao 41
3.10. Các kiến trúc CNN tiêu biểu 41
3.11. Ứng dụng của CNN 43
3.12. Hạn chế và thách thức 44
3.13. Thực nghiệm: CNN trên ba bộ dữ liệu ảnh 44
3.13.1. Dữ liệu và thiết lập 44
3.13.2. Cài đặt 46
3.13.3. Kết quả 47
3.13.4. So sánh CNN với mạng kết nối đầy đủ 48
3.14. Tiểu kết chương 3 49
CHƯƠNG 4. MẠNG NƠ-RON HỒI QUY (RNN) 50
4.1. Dữ liệu tuần tự và yêu cầu đối với mô hình 50
4.2. Kiến trúc và nguyên lý hoạt động của RNN 50
4.2.1. Phương trình cập nhật 50
4.2.2. Mở rộng theo thời gian 51
4.2.3. Số lượng tham số 51
4.2.4. Biểu diễn đầu vào văn bản 51
4.3. Các dạng bài toán với RNN 52
4.4. Huấn luyện RNN: lan truyền ngược theo thời gian 52
4.4.1. Thuật toán BPTT 52
4.4.2. Vấn đề biến mất và bùng nổ gradient 53
4.4.3. Các giải pháp 53
4.5. Bộ nhớ dài – ngắn hạn (LSTM) 54
4.5.1. Ý tưởng 54
4.5.2. Các phương trình 54
4.5.3. Vì sao LSTM khắc phục được biến mất gradient 55
4.5.4. Số lượng tham số 55
4.6. Đơn vị hồi quy có cổng (GRU) 56

4.7. Các mở rộng kiến trúc: RNN hai chiều và RNN nhiều tầng 56
4.7.1. RNN hai chiều *56*
4.7.2. RNN nhiều tầng *57*
4.8. Mô hình mã hóa – giải mã và cơ chế chú ý 57
4.8.1. Kiến trúc mã hóa – giải mã (Seq2Seq) *57*
4.8.2. Nút thắt cổ chai và cơ chế chú ý *57*
4.8.3. Từ attention đến Transformer *58*
4.9. Ứng dụng của RNN 58
4.10. Thực hành huấn luyện và đánh giá 58
4.10.1. Các siêu tham số và kỹ thuật thực hành *58*
4.10.2. Đánh giá mô hình tuần tự *59*
4.11. Hạn chế của RNN và xu hướng hiện nay 59
4.12. Thực nghiệm: LSTM trên ba bài toán dữ liệu tuần tự 60
4.12.1. Dữ liệu và thiết lập *60*
4.12.2. Cài đặt *61*
4.12.3. Kết quả *62*
4.12.4. So sánh RNN đơn giản, LSTM và GRU *64*
4.13. Tiểu kết chương 4 64
CHƯƠNG 5. ĐỒ THỊ TRI THỨC VÀ SINH VĂN BẢN TĂNG CƯỜNG TRUY XUẤT **65**
5.1. Động lực và lịch sử phát triển 65
5.1.1. Giới hạn của tri thức nằm trong tham số *65*
5.1.2. Lịch sử đồ thị tri thức *65*
5.1.3. Lịch sử RAG *67*
5.1.4. Vai trò và ý nghĩa của KG và RAG *68*
5.2. Đồ thị tri thức (Knowledge Graph) 69
5.2.1. Định nghĩa *69*
5.2.2. Bộ ba và các mô hình dữ liệu *69*
5.2.3. Ontology và truy vấn đồ thị *70*
5.2.4. Xây dựng đồ thị tri thức *70*
5.2.5. Suy luận và nhúng đồ thị tri thức *71*
5.2.6. Đối sánh thực thể *72*
5.2.7. Hệ sinh thái và ứng dụng *72*
5.2.8. So sánh với mạng ngữ nghĩa và các thách thức *73*
5.3. Sinh văn bản tăng cường truy xuất (RAG) 73

5.3.1. Nguyên lý 73
5.3.2. Kiến trúc và quy trình hoạt động 74
5.3.3. Chia đoạn và nhúng 74
5.3.4. Truy xuất: thưa, dày và lai 76
5.3.5. Các mô hình RAG 77
5.3.6. GraphRAG – kết hợp đồ thị tri thức và RAG 77
5.3.7. So sánh RAG và tinh chỉnh 78
5.3.8. Đánh giá hệ thống RAG 79
5.3.9. Ưu điểm, hạn chế và thách thức vận hành 79
5.4. Thực nghiệm: trợ lý hỏi đáp thị trường bất động sản Việt Nam 80
5.4.1. Bài toán, dữ liệu và tiền xử lý 80
5.4.2. Bốn hệ thống được so sánh 82
5.4.3. Cài đặt 83
5.4.4. Thiết lập đánh giá 84
5.4.5. Kết quả 84
5.4.6. Phân tích và thảo luận 86
5.5. Tiểu kết chương 5 88
TÀI LIỆU THAM KHẢO **89**

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
ML	Machine Learning	Học máy
DL	Deep Learning	Học sâu
ANN	Artificial Neural Network	Mạng nơ-ron nhân tạo
MLP	Multilayer Perceptron	Perceptron nhiều lớp
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
LSTM	Long Short-Term Memory	Bộ nhớ dài – ngắn hạn
GRU	Gated Recurrent Unit	Đơn vị hồi quy có cổng
BPTT	Backpropagation Through Time	Lan truyền ngược theo thời gian
SGD	Stochastic Gradient Descent	Hạ gradient ngẫu nhiên
SVM	Support Vector Machine	Máy vector hỗ trợ
KNN	K-Nearest Neighbors	K láng giềng gần nhất
PCA	Principal Component Analysis	Phân tích thành phần chính
MSE	Mean Squared Error	Sai số bình phương trung bình
ReLU	Rectified Linear Unit	Hàm kích hoạt tuyến tính chỉnh lưu
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
LLM	Large Language Model	Mô hình ngôn ngữ lớn
ILSVRC	ImageNet Large Scale Visual Recognition Challenge	Cuộc thi nhận dạng ảnh quy mô lớn ImageNet
PLA	Perceptron Learning Algorithm	Thuật toán học perceptron
SVR	Support Vector Regression	Hồi quy vector hỗ trợ
DQN	Deep Q-Network	Mạng Q sâu
SLM	Small Language Model	Mô hình ngôn ngữ nhỏ
KG	Knowledge Graph	Đồ thị tri thức
RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy xuất

RDF	Resource Description Framework	Khung mô tả tài nguyên
SPARQL	SPARQL Protocol and RDF Query Language	Ngôn ngữ truy vấn RDF
NER	Named Entity Recognition	Nhận dạng thực thể có tên
BM25	Best Matching 25	Hàm xếp hạng BM25
RRF	Reciprocal Rank Fusion	Hợp nhất thứ hạng nghịch đảo
ANN	Approximate Nearest Neighbor	Tìm láng giềng gần đúng
HNSW	Hierarchical Navigable Small World	Đồ thị thế giới nhỏ phân cấp
MRR	Mean Reciprocal Rank	Trung bình nghịch đảo thứ hạng
nDCG	Normalized Discounted Cumulative Gain	Lợi ích tích lũy chiết khấu chuẩn hóa

MỞ ĐẦU

1. Lý do chọn đề tài

Trí tuệ nhân tạo (Artificial Intelligence – AI) đang trở thành một trong những công nghệ có ảnh hưởng sâu rộng nhất tới đời sống kinh tế, xã hội và khoa học. Nếu như trước đây máy tính chủ yếu thực hiện các phép tính theo chương trình do con người viết sẵn từng bước, thì ngày nay các hệ thống AI có khả năng học từ dữ liệu, nhận dạng hình ảnh, hiểu và sinh ngôn ngữ tự nhiên, dự báo xu hướng và hỗ trợ con người ra quyết định. AI hiện diện trong công cụ tìm kiếm, mạng xã hội, thương mại điện tử, ngân hàng, y tế, giao thông, giáo dục và hầu hết các lĩnh vực khác của đời sống.

Sự bùng nổ của AI trong hơn một thập kỷ gần đây không phải là kết quả của một phát minh đơn lẻ. Nó là sự hội tụ của ba yếu tố: lượng dữ liệu số tăng trưởng theo cấp số nhân, năng lực tính toán song song của bộ xử lý đồ họa (GPU) và các thuật toán học máy ngày càng hiệu quả. Trong đó, học máy (Machine Learning – ML) cung cấp phương pháp để máy tính tự rút ra quy luật từ dữ liệu thay vì phải được lập trình chi tiết cho từng tình huống. Các kỹ thuật như hồi quy, phân loại, cây quyết định, máy vector hỗ trợ hay gom cụm vẫn là nền tảng cho rất nhiều hệ thống đang vận hành trong thực tế.

Bên cạnh học máy truyền thống, học sâu (Deep Learning – DL) đã tạo ra những bước nhảy vọt trong nhận dạng tiếng nói, thị giác máy tính và xử lý ngôn ngữ. Hai họ kiến trúc giữ vai trò nền móng của giai đoạn này là mạng nơ-ron tích chập (Convolutional Neural Network – CNN), vốn đặc biệt hiệu quả với dữ liệu có cấu trúc không gian như ảnh, và mạng nơ-ron hồi quy (Recurrent Neural Network – RNN), được thiết kế cho dữ liệu tuần tự như văn bản, âm thanh và chuỗi thời gian. Nhiều ý tưởng then chốt của các mô hình hiện đại – từ việc chia sẻ trọng số, học biểu diễn phân cấp cho tới cơ chế chú ý (attention) – đều có nguồn gốc trực tiếp từ hai họ kiến trúc này.

Tại Việt Nam, AI được xác định là một trong những công nghệ mũi nhọn. Chiến lược quốc gia về nghiên cứu, phát triển và ứng dụng trí tuệ nhân tạo đến năm 2030 (ban hành theo Quyết định số 127/QĐ-TTg ngày 26/01/2021 của Thủ tướng Chính phủ) [35] đặt mục tiêu đưa AI trở thành lĩnh vực công nghệ quan trọng, góp phần phát triển kinh tế – xã hội. Điều này đặt ra yêu cầu đối với sinh viên ngành Công nghệ thông tin là phải nắm vững nền tảng lý thuyết, chứ không chỉ biết sử dụng các thư viện có sẵn.

Xuất phát từ những lý do trên, tiểu luận lựa chọn tìm hiểu một cách hệ thống từ lịch sử hình thành AI, các kỹ thuật học máy cơ bản cho tới hai kiến trúc học sâu kinh điển là CNN và RNN. Đây cũng là nền tảng cần thiết để tiếp cận các chủ đề nâng cao sẽ được trình bày ở các phần sau của tiểu luận, gồm đồ thị tri thức (Knowledge Graph), sinh văn bản tăng cường truy xuất (Retrieval-Augmented Generation – RAG), học sâu trên đồ thị (Deep Graph) và hệ đa tác tử (Multi-Agent System).

2. Mục tiêu nghiên cứu

Tiểu luận hướng đến các mục tiêu cụ thể sau:

1. Trình bày quá trình hình thành và phát triển của trí tuệ nhân tạo qua các giai đoạn lịch sử chính, chỉ ra nguyên nhân của những giai đoạn thăng trầm và các yếu tố dẫn tới sự bùng nổ hiện nay.
2. Hệ thống hóa các khái niệm nền tảng của học máy: các dạng học, quy trình xây dựng mô hình, hàm mất mát, tối ưu hóa và đánh giá mô hình.
3. Phân tích nguyên lý, ưu điểm, hạn chế của một số thuật toán học máy phổ biến như hồi quy tuyến tính, hồi quy logistic, KNN, Naive Bayes, cây quyết định, SVM và K-means.
4. Làm rõ cấu trúc, nguyên lý hoạt động, quá trình huấn luyện và các kiến trúc tiêu biểu của mạng nơ-ron tích chập.
5. Trình bày cơ chế xử lý dữ liệu tuần tự của mạng nơ-ron hồi quy, vấn đề biến mất gradient và các giải pháp LSTM, GRU, RNN hai chiều, cơ chế attention.
6. Nhận diện những thách thức về dữ liệu, khả năng giải thích, chi phí tính toán và đạo đức khi triển khai các mô hình AI vào thực tế.
7. Trình bày đồ thị tri thức (mô hình dữ liệu, xây dựng, suy luận, nhúng) và kỹ thuật sinh văn bản tăng cường truy xuất (chia đoạn, nhúng, truy xuất lai, xếp hạng lại, GraphRAG, đánh giá).
8. Kiểm chứng lý thuyết bằng thực nghiệm: cài đặt hồi quy softmax, CNN và LSTM theo ba cách (tự viết bằng NumPy, Keras, PyTorch), đánh giá mỗi mô hình trên ba bộ dữ liệu thực và so sánh với các mô hình đối chứng; xây dựng và đánh giá trợ lý hỏi đáp thị trường bất động sản Việt Nam theo bốn cách (chỉ LLM, RAG cơ bản, đồ thị tri thức, kết hợp có định tuyến) trên 2,5 triệu tin đăng.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của tiểu luận là các khái niệm, mô hình và thuật toán nền tảng trong trí tuệ nhân tạo, học máy và học sâu, cụ thể là: tiến trình lịch sử của AI; các thuật toán học có giám sát và không giám sát cơ bản; mạng nơ-ron tích chập và mạng nơ-ron hồi quy cùng các biến thể; đồ thị tri thức và RAG.

Phạm vi nghiên cứu của phần này giới hạn từ Mở đầu đến Chương 5. Tiểu luận tập trung vào bản chất, trực giác và các công thức cốt lõi của từng phương pháp; không đi sâu vào việc chứng minh đầy đủ các kết quả toán học, cũng không xây dựng một sản phẩm phần mềm hoàn chỉnh. Các ví dụ số được chọn ở quy mô nhỏ để có thể kiểm tra bằng tính tay, nhằm làm rõ cơ chế hoạt động. Phần thực nghiệm ở cuối các chương 2, 3, 4 dùng mô hình kích thước nhỏ và các bộ dữ liệu công khai phổ biến, huấn luyện được trên một máy tính xách tay trong vài phút, với mục đích minh họa và kiểm chứng chứ không nhằm đạt kết quả tốt nhất hiện có. Thực nghiệm Chương 5 là một hệ thống hỏi đáp hoàn chỉnh (pipeline dữ liệu, chỉ mục vector, đồ thị tri thức, API và giao diện web) trên dữ liệu thật quy mô lớn, được đánh giá bằng bộ 100 câu hỏi có đáp án tính từ SQL. Các chủ đề Deep Graph và hệ đa tác tử sẽ được trình bày ở các chương tiếp theo.

4. Phương pháp nghiên cứu

Tiểu luận sử dụng kết hợp các phương pháp sau:

- **Phương pháp tổng hợp và phân tích tài liệu:** hệ thống hóa kiến thức từ giáo trình, sách chuyên khảo, bài báo khoa học gốc (như các công trình về LeNet, AlexNet, ResNet, LSTM, GRU, Transformer, RAG, GraphRAG) và tài liệu kỹ thuật.
- **Phương pháp lịch sử – logic:** sắp xếp các sự kiện, mô hình theo trình tự thời gian để làm rõ mối liên hệ nhân quả giữa các giai đoạn phát triển của AI.
- **Phương pháp so sánh:** đối chiếu các thuật toán theo dạng dữ liệu đầu vào, cách học, khả năng khái quát, mức độ dễ giải thích và chi phí tính toán.
- **Phương pháp minh họa bằng ví dụ:** sử dụng các ví dụ số nhỏ (tính tích chập, tính ma trận nhầm lẫn, đếm số tham số…) để làm sáng tỏ các công thức.
- **Phương pháp thực nghiệm:** cài đặt mô hình từ đầu và bằng thư viện, huấn luyện trên dữ liệu thực, đánh giá bằng các thước đo phù hợp và đối chiếu kết quả với phân tích lý thuyết. Mã nguồn được đính kèm để có thể chạy lại toàn bộ thí nghiệm.

5. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, tiểu luận kết nối những khái niệm thường được trình bày rời rạc thành một mạch kiến thức liên tục: từ AI biểu tượng dựa trên luật, qua học máy thống kê, tới học sâu với CNN và RNN. Cách trình bày này giúp người đọc thấy được vì sao mỗi kiến trúc ra đời – nó giải quyết hạn chế nào của phương pháp trước đó và để lại vấn đề gì cho thế hệ tiếp theo.

Về mặt thực tiễn, nội dung tiểu luận cung cấp cơ sở để lựa chọn mô hình phù hợp cho những bài toán thường gặp như phân loại ảnh, phát hiện vật thể, dự báo chuỗi thời gian, phân tích cảm xúc hay dịch máy. Đối với người học, hiểu được bản chất thuật toán quan trọng hơn việc chỉ gọi một hàm có sẵn: một mô hình có độ chính xác cao nhưng được huấn luyện trên dữ liệu thiên lệch, bị rò rỉ dữ liệu khi đánh giá hoặc không thể giải thích có thể gây ra rủi ro đáng kể khi triển khai. Vì vậy, tiểu luận cũng dành dung lượng thích đáng cho các vấn đề đánh giá mô hình, quá khớp và an toàn.

6. Bố cục của tiểu luận

Ngoài phần Mở đầu và Tài liệu tham khảo, nội dung phần này của tiểu luận gồm năm chương:

- **Chương 1. Lịch sử phát triển của trí tuệ nhân tạo:** trình bày khái niệm AI, các nền tảng tiền đề, các giai đoạn phát triển từ hội thảo Dartmouth (1956), hai "mùa đông AI", sự trỗi dậy của học máy, kỷ nguyên học sâu đến AI tạo sinh, mô hình ngôn ngữ lớn và trạng thái AI đương đại (đa phương thức bản địa, tác tử, mô hình suy luận).
- **Chương 2. Các kỹ thuật học máy cơ bản:** trình bày khái niệm học máy, các dạng học, tiền xử lý dữ liệu, đánh giá mô hình, tối ưu bằng hạ gradient và các thuật toán tiêu biểu; thực nghiệm hồi quy softmax trên Iris, Breast Cancer Wisconsin và Wine.
- **Chương 3. Mạng nơ-ron tích chập (CNN):** trình bày phép tích chập, các thành phần của CNN, quá trình huấn luyện, các kiến trúc kinh điển từ LeNet đến EfficientNet và các ứng dụng; thực nghiệm CNN trên MNIST, Fashion-MNIST và CIFAR-10, so sánh với mạng kết nối đầy đủ.
- **Chương 4. Mạng nơ-ron hồi quy (RNN):** trình bày nguyên lý RNN, lan truyền ngược theo thời gian, vấn đề biến mất gradient, các biến thể LSTM, GRU, RNN hai chiều, mô hình mã hóa – giải mã, cơ chế attention và ứng

dụng; thực nghiệm LSTM trên IMDB, SMS Spam Collection và chuỗi nhiệt độ Melbourne, so sánh RNN, LSTM và GRU.

- **Chương 5. Đồ thị tri thức và sinh văn bản tăng cường truy xuất:** trình bày lịch sử, vai trò, mô hình dữ liệu, cách xây dựng, suy luận và nhúng đồ thị tri thức; nguyên lý, kiến trúc, truy xuất, các mô hình RAG, GraphRAG và đánh giá; thực nghiệm trợ lý hỏi đáp bất động sản so sánh LLM, RAG, đồ thị tri thức và kết hợp có định tuyến.

Các chương 6 (Deep Graph) và 7 (Hệ đa tác tử) sẽ được trình bày trong phần tiếp theo của tiểu luận.

CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN CỦA TRÍ TUỆ NHÂN TẠO

1.1. Khái niệm trí tuệ nhân tạo

Trí tuệ nhân tạo là một ngành của khoa học máy tính nghiên cứu cách xây dựng các hệ thống có khả năng thực hiện những nhiệm vụ vốn đòi hỏi trí thông minh của con người, như nhận thức, suy luận, học tập, lập kế hoạch, giao tiếp bằng ngôn ngữ và ra quyết định. John McCarthy – người đặt ra thuật ngữ "Artificial Intelligence" – định nghĩa AI là "khoa học và kỹ thuật chế tạo ra những cỗ máy thông minh".

Trong giáo trình kinh điển *Artificial Intelligence: A Modern Approach*, Russell và Norvig [1] tổng hợp các định nghĩa về AI theo hai trục: trục thứ nhất phân biệt giữa *suy nghĩ* và *hành động*, trục thứ hai phân biệt giữa *giống con người* và *hợp lý*. Từ đó hình thành bốn cách tiếp cận:

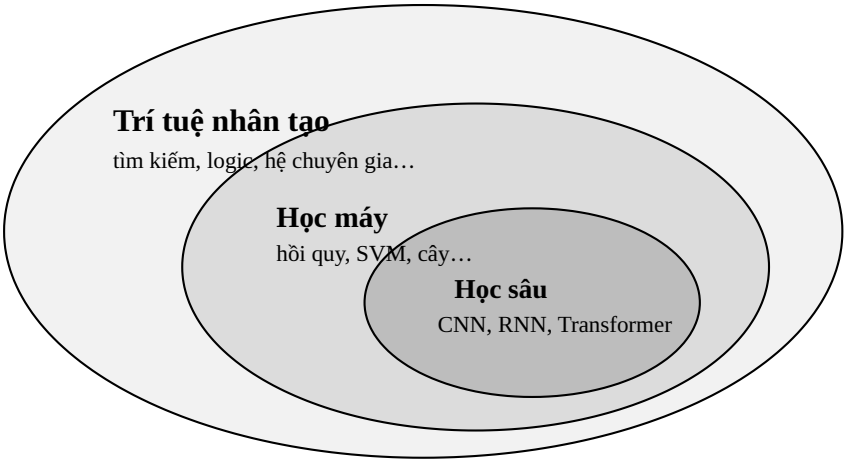
Bảng 1.1. Bốn cách tiếp cận định nghĩa trí tuệ nhân tạo

	Giống con người	**Hợp lý**
Suy nghĩ	Mô hình hóa nhận thức: xây dựng chương trình mô phỏng cách con người tư duy (khoa học nhận thức).	Suy nghĩ hợp lý: dùng logic hình thức để rút ra kết luận đúng từ tiền đề.
Hành động	Hành động giống người: tiêu biểu là phép thử Turing – máy được coi là thông minh nếu con người không phân biệt được nó với người thật qua hội thoại.	Tác tử hợp lý: hệ thống quan sát môi trường và chọn hành động tối đa hóa kết quả kỳ vọng theo một mục tiêu xác định.

Cách tiếp cận "tác tử hợp lý" (rational agent) được dùng phổ biến nhất hiện nay vì vừa đủ tổng quát, vừa đo đạc và tối ưu được bằng toán học: tác tử nhận tri giác từ môi trường, hành động tác động trở lại môi trường và được đánh giá bằng một thước đo hiệu năng.

Cũng cần phân biệt một số cặp khái niệm thường gặp:

- **AI hẹp (Narrow AI) và AI tổng quát (Artificial General Intelligence – AGI).** AI hẹp giải quyết tốt một nhiệm vụ cụ thể như chơi cờ, nhận dạng khuôn mặt hay dịch văn bản. AGI là hệ thống giả định có năng lực trí tuệ ngang con người trên phạm vi rộng các nhiệm vụ. Hầu hết hệ thống hiện nay vẫn là AI hẹp.
- **AI, học máy và học sâu.** Ba khái niệm này lồng nhau: học máy là một nhánh của AI, tập trung vào việc học từ dữ liệu; học sâu là một nhánh của học máy, sử dụng mạng nơ-ron nhiều lớp để học biểu diễn (Hình 1.1).



Hình 1.1. Quan hệ giữa trí tuệ nhân tạo, học máy và học sâu

1.2. Những nền tảng tiền đề (trước năm 1956)

Nền tảng khoa học của AI được hình thành trong nửa đầu thế kỷ XX từ ba dòng chảy chính.

Thứ nhất là logic và lý thuyết tính toán. Logic hình thức của Boole và Frege cung cấp ngôn ngữ biểu diễn mệnh đề và quy tắc suy luận. Năm 1936, Alan Turing đề xuất mô hình máy Turing, hình thức hóa khái niệm "thuật toán" và chứng minh rằng một cỗ máy đơn giản thao tác trên ký hiệu có thể thực hiện bất kỳ phép tính nào có thể mô tả được. Từ đó nảy sinh ý tưởng rằng tư duy, nếu là quá trình xử lý ký hiệu, có thể được cơ giới hóa.

Thứ hai là mô hình nơ-ron nhân tạo. Năm 1943, Warren McCulloch và Walter Pitts công bố mô hình nơ-ron nhân tạo đầu tiên: mỗi nơ-ron nhận các tín hiệu nhị phân, tính tổng có trọng số và "kích hoạt" nếu tổng vượt ngưỡng. Năm 1949, Donald Hebb đề xuất quy tắc học "các nơ-ron cùng kích hoạt thì liên kết giữa chúng được củng cố", gợi mở ý tưởng rằng mạng có thể *học* bằng cách điều chỉnh trọng số.

Thứ ba là điều khiển học và máy tính điện tử. Điều khiển học của Norbert Wiener nghiên cứu cơ chế phản hồi trong máy móc và sinh vật; máy tính điện tử như ENIAC (1945) cho phép kiểm chứng các ý tưởng đó trên phần cứng thật.

Năm 1950, Turing công bố bài báo nổi tiếng "*Computing Machinery and Intelligence*" [11], đặt câu hỏi "Máy có thể suy nghĩ không?" và đề xuất "trò chơi bắt chước" – sau này gọi là **phép thử Turing**. – một tiêu chí hành vi có thể kiểm tra thay cho tranh luận trừu tượng về bản chất của tư duy.

1.3. Sự ra đời và giai đoạn lạc quan (1956 – đầu thập niên 1970)

Mùa hè năm 1956, John McCarthy, Marvin Minsky, Nathaniel Rochester và Claude Shannon tổ chức hội thảo kéo dài khoảng hai tháng tại Đại học Dartmouth (Mỹ) [12]. Thuật ngữ *Artificial Intelligence* chính thức ra đời từ đây, và hội thảo Dartmouth thường được coi là thời điểm khai sinh của AI như một ngành khoa học.

Hai thập niên sau đó là giai đoạn tràn đầy lạc quan, với nhiều chương trình gây ấn tượng mạnh:

- **Logic Theorist** (1956) của Allen Newell, Herbert Simon và Cliff Shaw chứng minh được 38 trong 52 định lý đầu của chương 2 cuốn *Principia Mathematica*, có định lý với cách chứng minh gọn hơn bản gốc. Tiếp đó, **General Problem Solver** (GPS, 1959) giải bài toán bằng *phân tích phương tiện – mục đích* (means–ends analysis): so sánh trạng thái hiện tại với mục tiêu rồi chọn toán tử làm giảm khoảng cách giữa chúng, chia mục tiêu lớn thành các mục tiêu con.
- **Perceptron** (1958) của Frank Rosenblatt là mạng nơ-ron một lớp có khả năng học trọng số từ dữ liệu để phân loại mẫu.
- **LISP** (1958) của McCarthy trở thành ngôn ngữ chủ đạo của nghiên cứu AI trong nhiều thập kỷ.
- Chương trình chơi cờ đam của Arthur Samuel tự cải thiện qua việc chơi với chính nó; Samuel phổ biến thuật ngữ “machine learning” (1959).
- **ELIZA** (1966) của Joseph Weizenbaum mô phỏng một nhà trị liệu tâm lý chỉ bằng các luật khớp mẫu đơn giản. Dù vậy, nhiều người dùng vẫn gán cho chương trình sự thấu hiểu và cảm xúc – hiện tượng về sau được gọi là **hiệu ứng ELIZA**, vẫn còn nguyên tính thời sự với các chatbot ngày nay.
- **Robot Shakey** (SRI International, 1966 – 1972) là robot di động đầu tiên kết hợp thị giác máy tính, xử lý ngôn ngữ và lập kế hoạch hành động; bộ lập kế hoạch **STRIPS** của Shakey trở thành khuôn mẫu biểu diễn hành động (điều kiện trước – hiệu ứng) cho cả ngành lập kế hoạch tự động.

Hướng tiếp cận chủ đạo của giai đoạn này là **AI biểu tượng** (symbolic AI), về sau còn được gọi là GOFAI (*Good Old-Fashioned AI*): tri thức được biểu diễn bằng ký hiệu, luật và cấu trúc logic; trí thông minh được xem là quá trình tìm kiếm và suy diễn trên các biểu diễn đó. Newell và Simon phát biểu thành **giả thuyết hệ thống ký hiệu vật lý** (1976): một hệ thống thao tác ký hiệu vật lý có đủ điều kiện cần và đủ để hành động thông minh tổng quát. Hướng tiếp cận này để lại di sản lâu dài cho biểu diễn tri

thức – ontology, luật suy diễn và đồ thị tri thức ở Chương 5 đều là hậu duệ trực tiếp. Thành công ban đầu dẫn tới những dự đoán rất táo bạo về việc máy sẽ sớm làm được mọi việc của con người.

1.4. Mùa đông AI thứ nhất (khoảng 1974 – 1980)

Những dự đoán lạc quan nhanh chóng va phải thực tế. Các chương trình hoạt động tốt trên "thế giới đồ chơi" (bài toán nhỏ, được đơn giản hóa) nhưng thất bại khi áp dụng vào bài toán thực. Có ba nguyên nhân chính:

1. **Bùng nổ tổ hợp:** nhiều bài toán có không gian tìm kiếm tăng theo hàm mũ; phương pháp chạy được với 10 đối tượng trở nên bất khả thi với 100 đối tượng.
2. **Thiếu tri thức thường thức:** để hiểu một câu đơn giản, con người dựa vào khối lượng khổng lồ tri thức nền về thế giới, điều mà các chương trình thời đó không có. Báo cáo ALPAC (1966) kết luận dịch máy khi đó chậm, kém chính xác và đắt hơn dịch thủ công.
3. **Giới hạn của mạng nơ-ron một lớp:** năm 1969, Minsky và Papert xuất bản cuốn *Perceptrons*, chứng minh chặt chẽ rằng perceptron một lớp không thể biểu diễn những hàm đơn giản như XOR. Khi đó chưa có thuật toán hiệu quả để huấn luyện mạng nhiều lớp, nên nghiên cứu mạng nơ-ron gần như đình trệ.

Năm 1973, báo cáo của James Lighthill gửi Hội đồng Nghiên cứu Khoa học Anh đánh giá rằng AI đã không đạt được những mục tiêu "to lớn" đã hứa hẹn. Chính phủ Anh cắt giảm mạnh tài trợ; tại Mỹ, Tu chính án Mansfield (1969) yêu cầu kinh phí quốc phòng phải phục vụ nhiệm vụ quân sự cụ thể, nên cơ quan DARPA cũng thu hẹp các chương trình nghiên cứu cơ bản về AI. Giai đoạn suy giảm nguồn lực và niềm tin này được gọi là **mùa đông AI** (AI winter) thứ nhất.

1.5. Hệ chuyên gia và mùa đông AI thứ hai (1980 – đầu thập niên 1990)

AI hồi sinh vào đầu thập niên 1980 nhờ **hệ chuyên gia** (expert system) – các chương trình mã hóa tri thức của chuyên gia con người trong một lĩnh vực hẹp dưới dạng luật "NẾU… THÌ…". Hệ chuyên gia gồm *cơ sở tri thức* (sự kiện và luật) và *bộ máy suy diễn* áp dụng luật để rút ra kết luận theo hai chiều: **suy diễn tiến** (forward chaining) xuất phát từ các sự kiện đã biết, áp dụng luật để sinh sự kiện mới cho tới khi đạt mục tiêu; **suy diễn lùi** (backward chaining) xuất phát từ giả thuyết cần chứng minh, tìm các luật có kết luận khớp với giả thuyết rồi đệ quy kiểm tra điều kiện của chúng – cách MYCIN sử dụng.

Tiêu biểu là **MYCIN** (Stanford, thập niên 1970) chẩn đoán nhiễm trùng máu với khoảng 600 luật, đạt chất lượng khuyến nghị tương đương chuyên gia, và **XCON** (1980) cấu hình máy tính VAX cho công ty DEC, giúp tiết kiệm hàng chục triệu đô la mỗi năm – thành công thương mại lớn đầu tiên của AI.

Hệ chuyên gia trở thành một ngành công nghiệp trị giá hàng tỉ đô la, kéo theo thị trường **máy LISP** chuyên dụng (Symbolics, Lisp Machines Inc.). Năm 1982, Nhật Bản khởi động Dự án Máy tính Thế hệ thứ Năm (FGCS) với kinh phí khoảng 850 triệu USD, nhằm xây dựng máy tính suy luận logic song song dựa trên ngôn ngữ Prolog. Cùng thời gian, mạng nơ-ron hồi sinh: năm 1982 John Hopfield giới thiệu mạng Hopfield; năm 1986, Rumelhart, Hinton và Williams công bố bài báo phổ biến thuật toán **lan truyền ngược** (backpropagation) [13], cho phép huấn luyện hiệu quả mạng nơ-ron nhiều lớp và giải quyết được bài toán XOR mà perceptron một lớp bất lực.

Tuy nhiên, hệ chuyên gia sớm bộc lộ giới hạn: thu thập tri thức từ chuyên gia (*nút thắt thu nhận tri thức*) tốn kém; cơ sở luật lớn khó bảo trì; hệ thống "giòn", thất bại hoàn toàn khi gặp tình huống ngoài luật. Năm 1987 thị trường máy LISP sụp đổ khi máy trạm và máy tính cá nhân dùng bộ xử lý đa dụng (Sun, Apple, IBM PC) đạt tốc độ tương đương với giá rẻ hơn nhiều; Dự án Thế hệ thứ Năm không đạt mục tiêu; tài trợ lại bị cắt giảm, mở ra **mùa đông AI thứ hai** kéo dài tới khoảng đầu những năm 1990.

Bài học từ hai mùa đông AI là: tri thức của thế giới thực quá lớn, mơ hồ và biến động để có thể mã hóa thủ công hoàn toàn. AI cần những phương pháp có khả năng **tự học từ dữ liệu** và **xử lý sự không chắc chắn**.

1.6. Sự trỗi dậy của học máy thống kê (thập niên 1990 – 2000)

Từ thập niên 1990, trọng tâm của AI dịch chuyển dần từ các hệ thống dựa trên luật sang các phương pháp **thống kê và xác suất**. Một số cột mốc quan trọng:

- **Mạng Bayes** của Judea Pearl (1988) cho suy luận trong điều kiện không chắc chắn.
- **Máy vector hỗ trợ** (SVM) của Cortes và Vapnik (1995) [14] đạt kết quả hàng đầu trên nhiều bài toán phân loại nhờ nguyên lý cực đại hóa biên và thủ thuật kernel.
- Các phương pháp tổ hợp như **AdaBoost** (1995–1997) và **rừng ngẫu nhiên** (Breiman, 2001) [15] kết hợp nhiều mô hình yếu thành một mô hình mạnh.
- **LSTM** (Hochreiter và Schmidhuber, 1997) [7] giải quyết vấn đề ghi nhớ dài hạn của mạng hồi quy.

- **LeNet-5** (LeCun và cộng sự, 1998) [5] – một mạng nơ-ron tích chập – được dùng thương mại để đọc chữ số viết tay trên séc ngân hàng.
- Năm 1997, siêu máy tính **Deep Blue** của IBM đánh bại nhà vô địch cờ vua thế giới Garry Kasparov với tỉ số 3,5 – 2,5 – chủ yếu nhờ tìm kiếm quy mô lớn chứ không phải học máy: phần cứng song song với hàng trăm chip cờ chuyên dụng đánh giá khoảng 200 triệu thế cờ mỗi giây bằng tìm kiếm cắt tỉa alpha–beta, kết hợp hàm đánh giá do các đại kiện tướng tinh chỉnh.

Internet trong thập niên 2000 tạo ra nguồn dữ liệu khổng lồ, đưa học máy vào các sản phẩm đại trà như bộ lọc thư rác, hệ thống gợi ý và công cụ tìm kiếm. Đặc điểm chung của giai đoạn này là hiệu năng mô hình phụ thuộc rất nhiều vào **kỹ nghệ đặc trưng** (feature engineering): chuyên gia phải tự thiết kế các đặc trưng như SIFT, HOG cho ảnh hay túi từ (bag-of-words), TF-IDF cho văn bản trước khi đưa vào thuật toán học.

1.7. Kỷ nguyên học sâu (từ 2006 – 2012 đến nay)

Học sâu là nhánh của học máy sử dụng mạng nơ-ron nhiều lớp để **tự động học biểu diễn phân cấp** trực tiếp từ dữ liệu thô: các lớp đầu học đặc trưng đơn giản như cạnh, góc; các lớp sâu hơn tổ hợp chúng thành hình dạng, bộ phận và cuối cùng là khái niệm trừu tượng [4], [10]. Nhờ vậy, học sâu giảm đáng kể sự phụ thuộc vào kỹ nghệ đặc trưng thủ công.

Năm 2006, Geoffrey Hinton và cộng sự công bố phương pháp huấn luyện *mạng niềm tin sâu* (deep belief network) bằng tiền huấn luyện từng lớp, khơi lại sự quan tâm tới mạng sâu. Năm 2009, Fei-Fei Li và cộng sự công bố bộ dữ liệu **ImageNet** với hàng triệu ảnh được gán nhãn thuộc hàng nghìn lớp.

Bước ngoặt đến vào năm 2012: mạng **AlexNet** của Krizhevsky, Sutskever và Hinton [6] vô địch cuộc thi ILSVRC với tỉ lệ lỗi top-5 khoảng 15,3%, vượt xa vị trí thứ hai (khoảng 26,2%) vốn dùng phương pháp đặc trưng thủ công. Thành công này đến từ mạng tích chập sâu, ReLU, dropout và huấn luyện trên GPU; từ đây học sâu lan rộng sang mọi lĩnh vực.

Một số dấu mốc tiếp theo của kỷ nguyên học sâu:

- **2013 – 2015:** DeepMind công bố **Deep Q-Network** (DQN), kết hợp mạng tích chập với học tăng cường Q-learning để tự học chơi 49 trò chơi Atari 2600 trực tiếp từ điểm ảnh, đạt mức người chơi chuyên nghiệp ở hơn một nửa số trò [80] – mở đầu cho **học tăng cường sâu**.

- **2013 – 2014:** word2vec biểu diễn từ bằng vector ngữ nghĩa; mô hình chuỗi – chuỗi (seq2seq) với LSTM và GRU cho dịch máy; mạng đối sinh GAN (Goodfellow, 2014) mở ra hướng mô hình sinh ảnh.
- **2015:** ResNet với kết nối tắt cho phép huấn luyện mạng hơn 150 lớp và đạt tỉ lệ lỗi top-5 khoảng 3,6% trên ImageNet – thấp hơn mức ước lượng của con người trên cùng tác vụ.
- **2016:** AlphaGo [81] của DeepMind đánh bại kỳ thủ cờ vây Lee Sedol với tỉ số 4–1. Cờ vây có khoảng 10^{170} thế cờ hợp lệ, nên cách tìm kiếm vét cạn kiểu Deep Blue là bất khả thi; AlphaGo dùng *mạng chính sách* (policy network) đề xuất nước đi, *mạng giá trị* (value network) ước lượng khả năng thắng của một thế cờ, và tìm kiếm cây Monte Carlo (MCTS) dẫn hướng bởi hai mạng này. Nước đi thứ 37 ở ván 2, ban đầu bị bình luận viên cho là sai lầm, về sau được xem là nước cờ sáng tạo vượt khỏi kinh nghiệm của con người.
- **2017:** kiến trúc **Transformer** [9] dựa hoàn toàn trên cơ chế attention, loại bỏ tính tuần tự của RNN và cho phép huấn luyện song song quy mô lớn. Khả năng song song hóa này mở đường cho các **quy luật mở rộng** (scaling laws, Kaplan và cộng sự, 2020): sai số của mô hình ngôn ngữ giảm theo hàm lũy thừa khi tăng đồng thời số tham số, lượng dữ liệu và lượng tính toán [78].
- **2018:** Yoshua Bengio, Geoffrey Hinton và Yann LeCun nhận giải thưởng Turing vì những đóng góp nền tảng cho học sâu. Cùng năm, các mô hình tiền huấn luyện như BERT và GPT ra đời.
- **2020:** AlphaFold 2 dự đoán cấu trúc ba chiều của protein với độ chính xác gần thực nghiệm.

1.8. AI tạo sinh và mô hình ngôn ngữ lớn (từ 2020 đến nay)

AI tạo sinh (generative AI) là các mô hình có khả năng tạo ra nội dung mới như văn bản, hình ảnh, âm thanh, video hay mã nguồn. Khác với mô hình phân biệt chỉ dự đoán nhãn, mô hình sinh học phân phối của dữ liệu để lấy mẫu ra ví dụ mới.

Năm 2021, nhóm nghiên cứu Stanford đề xuất khái niệm **mô hình nền tảng** (foundation model) [79]: một mô hình rất lớn được tiền huấn luyện bằng học tự giám sát trên dữ liệu đa dạng không gán nhãn, sau đó được thích ứng (tinh chỉnh, gợi ý bằng câu lệnh) cho vô số nhiệm vụ hạ nguồn. Đại diện tiêu biểu nhất là **mô hình ngôn ngữ lớn** (Large Language Model – LLM). Một LLM là mạng Transformer với hàng tỉ tham số, được tiền huấn luyện trên kho văn bản khổng lồ với nhiệm vụ rất đơn giản: dự đoán token tiếp theo. Năm 2020, GPT-3 với 175 tỉ tham số cho thấy mô hình đủ lớn có thể thực hiện nhiều nhiệm vụ chỉ từ vài ví dụ trong câu lệnh (few-shot learning); một

số năng lực như làm toán nhiều bước hay lập trình chỉ xuất hiện rõ khi quy mô vượt một ngưỡng nhất định – hiện tượng được gọi là **năng lực bộc lộ** (emergent abilities). Tinh chỉnh theo chỉ dẫn và học tăng cường từ phản hồi của con người (RLHF) giúp mô hình hữu ích và an toàn hơn; ChatGPT (11/2022) đưa AI tạo sinh tới hàng trăm triệu người dùng. Song song, các **mô hình khuếch tán** (diffusion model) – học cách khử nhiễu dần từ nhiễu ngẫu nhiên thành dữ liệu – như Stable Diffusion, DALL·E 3 và Sora (2024) tạo ảnh và video chất lượng cao từ mô tả văn bản.

Các xu hướng nổi bật của giai đoạn hiện nay gồm:

- **Đa phương thức:** một mô hình xử lý đồng thời văn bản, hình ảnh, âm thanh và video.
- **Mô hình suy luận và tác tử:** mô hình được huấn luyện để "suy nghĩ" nhiều bước trước khi trả lời, biết sử dụng công cụ (tìm kiếm, chạy mã, gọi API) và phối hợp trong hệ đa tác tử.
- **Kết hợp truy xuất tri thức (RAG):** bổ sung tài liệu bên ngoài vào ngữ cảnh để giảm hiện tượng "ảo giác" (hallucination) – mô hình sinh ra thông tin sai nhưng trông rất thuyết phục.
- **AI cho khoa học:** năm 2024, giải Nobel Vật lý trao cho Hopfield và Hinton vì nền tảng của học máy bằng mạng nơ-ron; giải Nobel Hóa học trao một phần cho Hassabis và Jumper vì AlphaFold.

Bảng 1.2. Tóm tắt các giai đoạn phát triển chính của trí tuệ nhân tạo

Giai đoạn	Hướng tiếp cận chủ đạo	Thành tựu tiêu biểu	Hạn chế / nguyên nhân chuyển giai đoạn
1943 – 1955	Nền tảng lý thuyết	Nơ-ron McCulloch–Pitts, máy Turing, phép thử Turing	Chưa có phần cứng và chương trình cụ thể
1956 – 1974	AI biểu tượng, tìm kiếm	Dartmouth, Logic Theorist, Perceptron, LISP, ELIZA	Bùng nổ tổ hợp, thiếu tri thức thường thức → mùa đông thứ nhất
1980 – 1987	Hệ chuyên gia dựa trên luật	MYCIN, XCON; lan truyền ngược (1986)	Khó thu nhận và bảo trì tri thức → mùa đông thứ hai
1990 – 2011	Học máy thống kê	SVM, rừng ngẫu nhiên, LSTM, LeNet-5, Deep Blue	Phụ thuộc kỹ nghệ đặc trưng thủ công
2012 – 2019	Học sâu	AlexNet, ResNet, AlphaGo, Transformer, BERT	Cần nhiều dữ liệu gán nhãn, khó giải thích

2020 – 2024	Mô hình nền tảng, AI tạo sinh	GPT-3, ChatGPT, GPT-4, AlphaFold, mô hình khuếch tán, RAG	Ảo giác, tri thức lỗi thời, chi phí năng lượng
2025 – nay	Đa phương thức bản địa, tác tử, mô hình suy luận	GPT-4o, Gemini, o1/o3, DeepSeek-R1, SLM, GraphRAG	An toàn, căn chỉnh giá trị và quản trị tác tử tự chủ

1.9. Trạng thái AI đương đại (2025 – 2026)

Giai đoạn 2025 – 2026 không gắn với một phát minh đơn lẻ mà với ba xu hướng hội tụ, biến LLM từ một công cụ hỏi đáp thành nền tảng của các hệ thống thông minh hoàn chỉnh.

1.9.1. Mô hình đa phương thức bản địa

Các hệ thống đa phương thức thế hệ đầu *ghép nối* nhiều mô hình riêng: một mô hình nhận dạng tiếng nói chuyển âm thanh thành văn bản, LLM xử lý văn bản, rồi một mô hình khác tổng hợp giọng nói hay sinh ảnh. Mỗi chặng chuyển đổi làm mất thông tin (ngữ điệu, biểu cảm, bố cục ảnh) và cộng dồn độ trễ. Các **mô hình đa phương thức bản địa** (native multimodal) như GPT-4o hay dòng Gemini mã hóa văn bản, điểm ảnh, âm thanh và video thành token trong *cùng một* không gian biểu diễn của một mạng Transformer duy nhất. Nhờ vậy mô hình có thể hội thoại bằng giọng nói với độ trễ vài trăm mili giây, hiểu hình ảnh từ camera theo thời gian thực và phân tích video dài trong cửa sổ ngữ cảnh lên tới hàng triệu token.

1.9.2. AI tác tử (Agentic AI)

AI chuyển từ vai trò *trợ lý trả lời thụ động* sang **hệ thống tác tử tự chủ** – đúng với cách tiếp cận "tác tử hợp lý" ở mục 1.1. Một tác tử LLM có bốn năng lực đặc trưng:

- **Lập kế hoạch:** phân rã một mục tiêu phức tạp (ví dụ "khảo sát thị trường căn hộ quận 7 và lập báo cáo") thành chuỗi bước nhỏ.
- **Sử dụng công cụ** (tool use / function calling): tự gọi API, truy vấn cơ sở dữ liệu, tìm kiếm web hoặc chạy mã trong môi trường cô lập. Hệ thống ở mục 5.4 là một ví dụ: bộ định tuyến chọn giữa truy xuất vector, truy vấn đồ thị và mô hình định giá.
- **Bộ nhớ:** duy trì ngữ cảnh ngắn hạn trong phiên và bộ nhớ dài hạn qua nhiều phiên.
- **Tự phản tư và sửa lỗi** (reflection): kiểm tra đầu ra của chính mình, phát hiện lỗi rồi thử lại – chẳng hạn gửi thông báo lỗi của truy vấn Cypher trở lại cho

LLM để viết lại (mục 5.4.2).

Khi nhiều tác tử chuyên biệt phối hợp với nhau, ta có **hệ đa tác tử** (Multi-Agent System) – chủ đề của Chương 7.

1.9.3. Mô hình suy luận và mô hình nhỏ

Mô hình suy luận (reasoning model) như OpenAI o1/o3 hay DeepSeek-R1 được huấn luyện bằng học tăng cường để tạo ra một chuỗi suy nghĩ (chain-of-thought) dài trước khi trả lời. Thay vì chỉ tăng quy mô huấn luyện, chúng **tăng lượng tính toán lúc suy luận** (test-time compute): mô hình "nghĩ lâu hơn" cho câu khó, nhờ đó đạt kết quả gần chuyên gia ở các bài toán Olympic, lập trình thi đấu và suy luận logic nhiều bước.

Ở chiều ngược lại, **mô hình ngôn ngữ nhỏ** (Small Language Model – SLM) từ 1 đến khoảng 8 tỉ tham số đạt chất lượng tương đương các mô hình lớn của vài năm trước nhờ ba kỹ thuật: *chưng cất tri thức* (knowledge distillation – mô hình nhỏ học theo đầu ra của mô hình lớn), *lượng tử hóa* (quantization – biểu diễn trọng số bằng 8 hoặc 4 bit thay vì 16 – 32 bit) và kiến trúc *hỗn hợp chuyên gia* (Mixture-of-Experts – mỗi token chỉ kích hoạt một phần nhỏ tham số). SLM chạy được trực tiếp trên điện thoại, máy tính cá nhân (on-device AI), giảm chi phí, độ trễ và rủi ro lộ dữ liệu.

1.10. Những thách thức đặt ra

Song song với các thành tựu, sự phát triển nhanh của AI đặt ra nhiều thách thức cần được giải quyết:

- **Dữ liệu và thiên lệch:** mô hình học từ dữ liệu lịch sử nên có thể kế thừa và khuếch đại định kiến sẵn có (tuyển dụng, cho vay, tư pháp).
- **Khả năng giải thích:** mạng nơ-ron sâu với hàng triệu đến hàng tỉ tham số là "hộp đen", khó giải trình trong y tế, tài chính.
- **Độ tin cậy và an toàn:** mô hình có thể bị đánh lừa bởi nhiễu đối kháng, sinh nội dung sai lệch, hoặc bị lạm dụng để tạo tin giả, tấn công mạng.
- **Quyền riêng tư, bản quyền và chi phí tài nguyên:** dữ liệu huấn luyện quy mô lớn đặt ra câu hỏi về quyền tác giả; huấn luyện mô hình lớn tiêu tốn nhiều điện năng.
- **Căn chỉnh giá trị (AI alignment):** khi tác tử AI có quyền tự hành động (gửi thư, chạy mã, giao dịch), cần bảo đảm mục tiêu và hành vi của chúng phù hợp với ý định, đạo đức và pháp luật của con người, kể cả trong tình huống chưa được lường trước.

- **Tác động xã hội và pháp lý:** AI làm thay đổi thị trường lao động và đòi hỏi khung pháp lý mới. Liên minh châu Âu đã thông qua Đạo luật AI (EU AI Act) năm 2024 theo cách tiếp cận dựa trên mức độ rủi ro.

Lịch sử cũng để lại một bài học quản trị: cả hai mùa đông AI đều bắt nguồn từ **khoảng cách giữa lời hứa và năng lực thực tế** – kỳ vọng do nhà nghiên cứu và truyền thông tạo ra vượt xa khả năng của phần cứng và thuật toán đương thời. Với các hệ thống quan trọng như y tế, pháp lý hay tài chính, ảo giác của LLM buộc phải được kiểm soát bằng truy xuất tri thức có nguồn gốc (RAG, đồ thị tri thức – Chương 5) và sự phê duyệt của con người (human-in-the-loop).

Một hệ thống AI đáng tin cậy vì thế cần được đánh giá không chỉ bằng độ chính xác mà còn bằng tính an toàn, công bằng, minh bạch, bảo vệ dữ liệu và khả năng kiểm soát của con người.

1.11. Tiểu kết chương 1

Lịch sử gần bảy thập kỷ của AI là một chuỗi chu kỳ giữa kỳ vọng và thực tế. Giai đoạn đầu với AI biểu tượng cho thấy sức mạnh của suy luận logic nhưng cũng bộc lộ giới hạn trước sự phức tạp của thế giới thực. Hệ chuyên gia mang lại thành công thương mại nhưng vấp phải nút thắt thu nhận tri thức. Học máy thống kê chuyển trọng tâm sang học từ dữ liệu, và học sâu – nhờ dữ liệu lớn, GPU và các cải tiến thuật toán – đã loại bỏ phần lớn nhu cầu thiết kế đặc trưng thủ công. Giai đoạn hiện nay với mô hình nền tảng, mô hình đa phương thức, mô hình suy luận và tác tử đang đưa AI từ công cụ dự đoán thành hệ thống có khả năng hành động. Những “mùa đông” không làm AI biến mất mà giúp cộng đồng nhận ra giới hạn của từng cách tiếp cận và định hướng lại nghiên cứu.

Hai kiến trúc CNN và RNN là những trụ cột của giai đoạn học sâu, còn các thuật toán học máy cơ bản vẫn là nền móng để hiểu cách một mô hình học từ dữ liệu. Đồ thị tri thức và RAG thể hiện sự trở lại của biểu diễn tri thức tường minh – di sản của AI biểu tượng – để bù đắp điểm yếu về tính xác thực của LLM. Các chương tiếp theo sẽ lần lượt trình bày chi tiết những nội dung này.

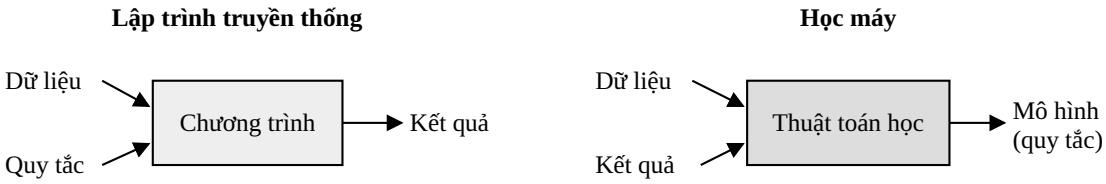
CHƯƠNG 2. CÁC KỸ THUẬT HỌC MÁY CƠ BẢN

2.1. Khái niệm học máy

Năm 1959, Arthur Samuel mô tả học máy là lĩnh vực nghiên cứu giúp máy tính có khả năng học mà không cần được lập trình một cách tường minh. Tom Mitchell [2] đưa ra định nghĩa hình thức hơn, được sử dụng rộng rãi:

> *Một chương trình máy tính được gọi là học từ kinh nghiệm E đối với lớp nhiệm vụ T và thước đo hiệu năng P, nếu hiệu năng của nó trên các nhiệm vụ thuộc T, đo bằng P, được cải thiện nhờ kinh nghiệm E.*

Ví dụ, với bộ lọc thư rác: nhiệm vụ T là phân loại thư thành "rác" hoặc "không rác"; kinh nghiệm E là tập thư đã được người dùng gán nhãn; thước đo P là tỉ lệ thư được phân loại đúng. Điểm khác biệt căn bản so với lập trình truyền thống được minh họa ở Hình 2.1: trong lập trình truyền thống, con người viết ra quy tắc và máy áp dụng quy tắc lên dữ liệu để cho ra kết quả; trong học máy, con người cung cấp dữ liệu cùng kết quả mong muốn, và máy tự suy ra "quy tắc" dưới dạng một mô hình.



Hình 2.1. So sánh lập trình truyền thống và học máy

Về mặt toán học, xét bài toán dự đoán với tập dữ liệu huấn luyện $D = \{(x_i, y_i)\}_{i=1}^{n}$, trong đó $x_i \in \mathbb{R}^d$ là vector đặc trưng của mẫu thứ i và y_i là nhãn (hoặc giá trị) tương ứng. Học máy tìm một hàm f_θ thuộc một họ hàm cho trước (gọi là *không gian giả thuyết*), được tham số hóa bởi θ, sao cho $f_\theta(x) \approx y$. Mức độ sai lệch giữa dự đoán và thực tế được đo bằng **hàm mất mát** L, và việc học trở thành bài toán tối ưu hóa *rủi ro thực nghiệm*:

$$\theta^* = \arg\min_{\theta} \; \frac{1}{n} \sum_{i=1}^{n} L\big(f_\theta(x_i), y_i\big)$$

Cần nhấn mạnh rằng mục tiêu thực sự không phải là đạt sai số nhỏ trên dữ liệu huấn luyện, mà là **khả năng khái quát hóa** (generalization) – dự đoán tốt trên dữ liệu mới chưa từng gặp. Một mô hình chỉ "học thuộc" dữ liệu huấn luyện là vô dụng trong thực tế.

2.2. Các dạng học máy

Dựa trên loại tín hiệu phản hồi mà thuật toán nhận được, học máy thường được chia thành các dạng sau [3], [33].

2.2.1. Học có giám sát

Trong **học có giám sát** (supervised learning), mỗi mẫu huấn luyện đi kèm nhãn đúng. Mô hình học ánh xạ từ đầu vào tới đầu ra bằng cách so sánh dự đoán với nhãn. Có hai nhóm bài toán chính:

- **Hồi quy** (regression): đầu ra là giá trị liên tục, ví dụ giá nhà, nhiệt độ ngày mai.
- **Phân loại** (classification): đầu ra là nhãn rời rạc – nhị phân (thư rác/không rác), đa lớp (chữ số 0–9) hoặc đa nhãn.

2.2.2. Học không giám sát

Học không giám sát (unsupervised learning) khám phá cấu trúc ẩn trong dữ liệu không nhãn: *gom cụm* khách hàng, *giảm chiều* để trực quan hóa, *phát hiện bất thường* như giao dịch gian lận, *khai phá luật kết hợp*.

2.2.3. Học bán giám sát và học tự giám sát

Khi gán nhãn đắt đỏ (ví dụ ảnh y tế) mà dữ liệu chưa nhãn dồi dào, **học bán giám sát** kết hợp ít dữ liệu có nhãn với nhiều dữ liệu không nhãn. **Học tự giám sát** tự tạo nhãn từ chính dữ liệu – che từ rồi đoán lại (BERT), dự đoán từ tiếp theo (GPT) – và là cách các mô hình nền tảng hiện đại được tiền huấn luyện.

2.2.4. Học tăng cường

Trong **học tăng cường** (reinforcement learning), một *tác tử* tương tác với *môi trường*: tại mỗi bước, tác tử quan sát trạng thái s_t, chọn hành động a_t, nhận phần thưởng r_t và chuyển sang trạng thái mới. Mục tiêu là học một *chính sách* $\pi(a \mid s)$ tối đa hóa tổng phần thưởng kỳ vọng tích lũy lâu dài. Tác tử không được chỉ ra hành động đúng mà phải tự thử – sai, cân bằng giữa *khai thác* và *khám phá*; ứng dụng trong trò chơi (AlphaGo), điều khiển robot và tinh chỉnh mô hình ngôn ngữ.

2.3. Quy trình xây dựng một mô hình học máy

Một dự án học máy thực tế hiếm khi chỉ gồm việc chọn thuật toán. Quy trình điển hình gồm các bước:

1. **Xác định bài toán:** đầu vào, đầu ra là gì; bài toán hồi quy hay phân loại; thước đo thành công về mặt nghiệp vụ là gì.
2. **Thu thập dữ liệu:** từ cơ sở dữ liệu, nhật ký hệ thống, cảm biến, khảo sát hoặc nguồn mở.
3. **Khám phá dữ liệu (EDA):** thống kê mô tả, trực quan hóa phân phối, kiểm tra tương quan, phát hiện giá trị thiếu và ngoại lai.
4. **Tiền xử lý và kỹ nghệ đặc trưng:** làm sạch, mã hóa, chuẩn hóa, tạo đặc trưng mới.
5. **Chia tập dữ liệu:** huấn luyện, xác thực, kiểm tra.
6. **Huấn luyện mô hình cơ sở (baseline)** đơn giản để có mốc so sánh, sau đó thử các mô hình phức tạp hơn.
7. **Tinh chỉnh siêu tham số** trên tập xác thực hoặc bằng kiểm định chéo.
8. **Đánh giá cuối cùng** trên tập kiểm tra – chỉ dùng một lần.
9. **Triển khai và giám sát:** theo dõi hiệu năng theo thời gian vì phân phối dữ liệu thực tế có thể thay đổi (*data drift*), từ đó huấn luyện lại khi cần.

2.4. Tiền xử lý dữ liệu

Nguyên lý “rác vào, rác ra” (garbage in, garbage out) nhấn mạnh rằng chất lượng dữ liệu quyết định trực tiếp chất lượng mô hình. Các thao tác tiền xử lý phổ biến gồm [33], [34]:

Xử lý giá trị thiếu: loại bỏ bản ghi hoặc cột có quá nhiều giá trị thiếu; điền giá trị thiếu bằng trung bình, trung vị, giá trị phổ biến nhất, hoặc dự đoán bằng mô hình khác.

Xử lý ngoại lai: phát hiện bằng quy tắc khoảng tứ phân vị (IQR) hoặc z-score, sau đó loại bỏ, cắt ngưỡng hoặc giữ lại tùy theo việc ngoại lai là lỗi đo hay là hiện tượng thực.

Mã hóa biến phân loại: *mã hóa nhãn* (label encoding) gán mỗi giá trị một số nguyên – chỉ phù hợp với biến có thứ tự (như “thấp < trung bình < cao”); *mã hóa one-hot* tạo một cột nhị phân cho mỗi giá trị – phù hợp với biến không có thứ tự (như tên thành phố); với biến có rất nhiều giá trị, có thể dùng *embedding* học được.

Chuẩn hóa thang đo: khi các đặc trưng có thang đo khác nhau (diện tích tính bằng m^2 cỡ hàng trăm, số phòng cỡ hàng đơn vị), các thuật toán dựa trên khoảng cách hoặc hạ gradient sẽ bị chi phối bởi đặc trưng có giá trị lớn. Hai cách phổ biến là chuẩn hóa min–max và chuẩn hóa z-score:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}, \qquad z = \frac{x - \mu}{\sigma}$$

trong đó μ và σ là trung bình và độ lệch chuẩn của đặc trưng.

Một nguyên tắc quan trọng là **tránh rò rỉ dữ liệu** (data leakage): mọi tham số tiền xử lý (như μ, σ, giá trị điền thiếu) phải được tính *chỉ trên tập huấn luyện*, rồi mới áp dụng cho tập xác thực và kiểm tra. Nếu tính trên toàn bộ dữ liệu, thông tin của tập kiểm tra sẽ "lọt" vào quá trình huấn luyện và làm kết quả đánh giá lạc quan một cách giả tạo.

2.5. Đánh giá mô hình

2.5.1. Chia tập dữ liệu và kiểm định chéo

Dữ liệu thường được chia thành **tập huấn luyện** (học tham số), **tập xác thực** (chọn mô hình, siêu tham số) và **tập kiểm tra** (đánh giá lần cuối), ví dụ 70% – 15% – 15%. Khi dữ liệu ít, dùng **kiểm định chéo k-fold**: chia thành k phần, lần lượt lấy mỗi phần làm tập xác thực và lấy trung bình k kết quả. Với chuỗi thời gian phải chia theo thứ tự thời gian, không xáo trộn.

2.5.2. Thước đo cho bài toán hồi quy

Với y_i là giá trị thực và $\hat{y}_i$ là giá trị dự đoán, các thước đo thường dùng gồm:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|, \qquad \mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

MSE phạt nặng các sai số lớn do bình phương, còn MAE ít nhạy cảm với ngoại lai hơn. Hệ số xác định $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$ cho biết tỉ lệ phương sai của y được mô hình giải thích.

2.5.3. Thước đo cho bài toán phân loại

Với phân loại nhị phân, kết quả dự đoán được tổng hợp trong **ma trận nhầm lẫn** gồm bốn ô: đúng dương tính (TP), sai dương tính (FP), sai âm tính (FN) và đúng âm tính (TN). Từ đó định nghĩa:

$$\mathrm{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \mathrm{Precision} = \frac{TP}{TP + FP}, \quad \mathrm{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$$

Precision là tỉ lệ đúng trong các mẫu được báo dương; *Recall* là tỉ lệ mẫu dương thực sự được phát hiện; F_1 là trung bình điều hòa của hai đại lượng.

Ví dụ. Một mô hình sàng lọc bệnh thử trên 1.000 người, trong đó 50 người mắc bệnh, cho kết quả như Bảng 2.1.

Bảng 2.1. Ma trận nhầm lẫn của ví dụ sàng lọc bệnh

	Dự đoán: có bệnh	**Dự đoán: không bệnh**
Thực tế: có bệnh	TP = 40	FN = 10
Thực tế: không bệnh	FP = 20	TN = 930

Ta có Accuracy = (40 + 930)/1000 = 97%; Precision = 40/60 ≈ 66,7%; Recall = 40/50 = 80%; F_1 ≈ 72,7%. Đáng chú ý, một mô hình "ngây thơ" luôn trả lời "không bệnh" cũng đạt Accuracy = 95% nhưng Recall = 0 – hoàn toàn vô dụng. Với dữ liệu **mất cân bằng lớp**, accuracy dễ gây hiểu lầm; cần xét precision, recall và chi phí của từng loại sai lầm – trong y tế, bỏ sót bệnh (FN) nguy hiểm hơn báo nhầm (FP) nên recall được ưu tiên.

Với mô hình trả về xác suất, thay đổi ngưỡng quyết định cho phép đánh đổi precision – recall; **đường cong ROC** và diện tích dưới nó (**AUC**, 0,5 là đoán ngẫu nhiên, 1 là hoàn hảo) đánh giá khả năng phân biệt trên mọi ngưỡng.

2.6. Tối ưu hóa bằng hạ gradient

Hầu hết mô hình học máy hiện đại, đặc biệt là mạng nơ-ron, được huấn luyện bằng **hạ gradient** (gradient descent) [4]. Ý tưởng: gradient $\nabla_\theta J(\theta)$ chỉ hướng hàm mất mát J tăng nhanh nhất, do đó di chuyển tham số theo hướng ngược lại sẽ làm giảm mất mát:

$$\theta \leftarrow \theta - \eta \, \nabla_\theta J(\theta)$$

trong đó $\eta > 0$ là **tốc độ học** (learning rate). Nếu η quá nhỏ, quá trình hội tụ rất chậm; nếu quá lớn, tham số có thể dao động quanh điểm cực tiểu hoặc phân kỳ.

Có ba biến thể chính tùy theo lượng dữ liệu dùng để tính gradient ở mỗi bước: *hạ gradient theo lô đầy đủ* (batch GD) dùng toàn bộ dữ liệu – chính xác nhưng tốn kém; *hạ gradient ngẫu nhiên* (SGD) dùng một mẫu – nhanh nhưng nhiễu; và *hạ gradient theo lô nhỏ* (mini-batch GD) dùng một nhóm nhỏ (thường 32–512 mẫu) – là lựa chọn phổ biến nhất vì cân bằng giữa tốc độ, độ ổn định và khả năng tận dụng tính toán song song trên GPU. Các thuật toán cải tiến như **Momentum** (tích lũy "quán tính" từ các bước trước), **RMSProp** và **Adam** [32] (tự điều chỉnh tốc độ học riêng cho từng tham số) giúp hội tụ nhanh và ổn định hơn.

2.7. Hồi quy tuyến tính

Hồi quy tuyến tính giả định đầu ra là tổ hợp tuyến tính của các đặc trưng:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d = \mathbf{w}^\top \mathbf{x} + w_0$$

Tham số được xác định bằng cách cực tiểu hóa sai số bình phương trung bình $J(\mathbf{w}) = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$. Bài toán này có nghiệm dạng đóng (phương trình chuẩn) $\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y}$, hoặc có thể giải bằng hạ gradient khi số đặc trưng lớn.

Ví dụ. Với bốn điểm dữ liệu (x, y) = (1; 2), (2; 4), (3; 5), (4; 8), phương pháp bình phương tối thiểu cho hệ số góc $w_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{9{,}5}{5} = 1{,}9$ và hệ số chặn $w_0 = \bar{y} - w_1 \bar{x} = 4{,}75 - 1{,}9 \times 2{,}5 = 0$. Mô hình thu được là $\hat{y} = 1{,}9x$ với MSE = 0,175.

Ưu điểm của hồi quy tuyến tính là đơn giản, huấn luyện nhanh và dễ giải thích: trọng số w_j cho biết $\hat{y}$ thay đổi bao nhiêu khi x_j tăng một đơn vị (các đặc trưng khác giữ nguyên). Hạn chế là chỉ biểu diễn được quan hệ tuyến tính; để mô hình hóa quan hệ phi tuyến cần bổ sung đặc trưng đa thức hoặc biến đổi. Mô hình cũng nhạy cảm với ngoại lai và hiện tượng đa cộng tuyến giữa các đặc trưng.

2.8. Hồi quy logistic

Dù có tên là “hồi quy”, **hồi quy logistic** là mô hình phân loại nhị phân. Mô hình tính tổ hợp tuyến tính $z = \mathbf{w}^\top \mathbf{x} + b$ rồi đưa qua hàm **sigmoid** để thu được xác suất thuộc lớp dương:

$$P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Hàm sigmoid ánh xạ mọi số thực vào khoảng (0; 1); mẫu được gán lớp dương nếu xác suất lớn hơn ngưỡng, thường là 0,5 – tương ứng với $z > 0$. Như vậy, ranh giới quyết định $\mathbf{w}^\top \mathbf{x} + b = 0$ là một siêu phẳng. Hàm mất mát phù hợp là **entropy chéo nhị phân** (binary cross-entropy):

$$J(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n} \Big[y_i \log \hat{p}_i + (1 - y_i) \log (1 - \hat{p}_i) \Big]$$

Hàm này phạt rất nặng khi mô hình tự tin dự đoán sai (ví dụ $\hat{p} \to 0$ trong khi $y = 1$). Với bài toán nhiều lớp ($C > 2$ lớp), sigmoid được thay bằng hàm **softmax**, tạo thành **hồi quy softmax** (multinomial logistic regression). Mỗi lớp c có một vector trọng số riêng, $z_c = \mathbf{w}_c^\top \mathbf{x} + b_c$, và

$$\hat{p}_c = \frac{e^{z_c}}{\sum_{k=1}^{C} e^{z_k}}, \qquad J(W) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} y_{ic}\log \hat{p}_{ic}, \qquad \nabla_{\mathbf{w}_c} J = \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_{ic} - y_{ic})\,\mathbf{x}_i$$

với y_{ic} là nhãn mã hóa one-hot. Khác với chiến lược "một lớp chống phần còn lại" (one-vs-rest) dùng C bộ phân loại nhị phân độc lập, softmax bảo đảm các xác suất dự đoán không âm và có tổng bằng 1. Gradient có dạng rất gọn "dự đoán trừ nhãn" giống hồi quy logistic, và cũng chính là gradient của lớp đầu ra trong mọi mạng nơ-ron phân loại (CNN ở Chương 3, LSTM ở Chương 4). Hồi quy logistic huấn luyện nhanh, cho đầu ra xác suất và dễ diễn giải, nên thường là mô hình cơ sở đầu tiên cho bài toán phân loại. Đáng chú ý, một nơ-ron đơn lẻ với hàm kích hoạt sigmoid chính là hồi quy logistic – đây là cầu nối tự nhiên sang mạng nơ-ron.

2.9. K láng giềng gần nhất (KNN)

KNN là thuật toán dựa trên trực giác "gần mực thì đen, gần đèn thì sáng": để dự đoán cho một điểm mới, thuật toán tìm k điểm gần nhất trong tập huấn luyện rồi lấy nhãn chiếm đa số (phân loại) hoặc giá trị trung bình (hồi quy). Khoảng cách thường dùng là khoảng cách Euclid $d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_j (x_j - x'_j)^2}$ hoặc khoảng cách Manhattan.

KNN là thuật toán *học lười* (lazy learning): không có giai đoạn huấn luyện thực sự, chỉ ghi nhớ dữ liệu. Điều này khiến nó đơn giản và linh hoạt với ranh giới quyết định phức tạp, nhưng việc dự đoán lại chậm khi dữ liệu lớn vì phải tính khoảng cách tới mọi điểm. Lựa chọn k ảnh hưởng mạnh tới kết quả: k nhỏ khiến mô hình nhạy với nhiễu, k lớn khiến ranh giới quá trơn. KNN cũng đòi hỏi chuẩn hóa dữ liệu và hoạt động kém trong không gian nhiều chiều do hiện tượng **lời nguyền số chiều** – khi số chiều tăng, khoảng cách giữa các điểm trở nên gần như nhau và mất ý nghĩa phân biệt.

Lưu ý khi dùng KNN trong thực tế. (1) k nên được chọn bằng kiểm định chéo trên tập huấn luyện (thường lấy k lẻ với phân loại nhị phân để tránh hòa), không bao giờ chọn trên tập kiểm tra. (2) Có thể gán trọng số cho láng giềng tỉ lệ nghịch với khoảng cách để điểm gần ảnh hưởng nhiều hơn. (3) Tìm kiếm vét cạn có độ phức tạp $O(nd)$ cho mỗi truy vấn và phải giữ toàn bộ tập huấn luyện trong bộ nhớ; với dữ liệu lớn, dùng cấu trúc chỉ mục như KD-tree, Ball-tree (hiệu quả khi số chiều thấp) hoặc **tìm láng giềng gần đúng** (Approximate Nearest Neighbor – ANN) như đồ thị HNSW, chấp nhận sai số nhỏ để truy vấn nhanh hơn nhiều bậc. Chính bài toán láng giềng gần nhất trên vector nhúng là lõi của bước truy xuất trong hệ thống RAG ở Chương 5. (4) Với dữ liệu nhiều chiều, nên giảm chiều (PCA – mục 2.14) hoặc chọn đặc trưng trước khi tính khoảng cách.

2.10. Naive Bayes

Naive Bayes là bộ phân loại xác suất dựa trên định lý Bayes:

$$P(c \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid c)\,P(c)}{P(\mathbf{x})} \;\propto\; P(c)\prod_{j=1}^{d} P(x_j \mid c)$$

Giả định "ngây thơ" ở đây là các đặc trưng độc lập có điều kiện với nhau khi biết lớp c. Giả định này hiếm khi đúng hoàn toàn trong thực tế, nhưng mô hình vẫn hoạt động tốt một cách đáng ngạc nhiên, đặc biệt trong phân loại văn bản (lọc thư rác, phân loại chủ đề) – nơi mỗi đặc trưng là sự xuất hiện của một từ. Naive Bayes huấn luyện cực nhanh (chỉ cần đếm tần suất), cần ít dữ liệu và dễ mở rộng; hạn chế là xác suất dự đoán thường không được hiệu chỉnh tốt và mô hình không nắm bắt được tương tác giữa các đặc trưng.

2.11. Cây quyết định và các phương pháp tổ hợp

2.11.1. Cây quyết định

Cây quyết định phân chia dữ liệu bằng chuỗi câu hỏi trên các đặc trưng. Mỗi nút trong là một phép kiểm tra (ví dụ "thu nhập > 15 triệu?"), mỗi nhánh là một kết quả của phép kiểm tra, và mỗi nút lá chứa dự đoán. Cây được xây tham lam từ trên xuống, mỗi nút chọn phép tách làm các nút con "thuần khiết" nhất theo **entropy** hoặc **chỉ số Gini**:

$$H(S) = -\sum_{c} p_c \log_2 p_c, \qquad \mathrm{Gini}(S) = 1 - \sum_{c} p_c^2$$

với p_c là tỉ lệ mẫu thuộc lớp c trong tập S. Phép tách tốt nhất là phép tách có **độ lợi thông tin** (information gain) lớn nhất – tức mức giảm entropy sau khi tách.

Ví dụ. Một nút có 10 mẫu gồm 6 mẫu lớp "Có" và 4 mẫu lớp "Không": $H = -0{,}6\log_2 0{,}6 - 0{,}4\log_2 0{,}4 \approx 0{,}971$; Gini $= 1 - 0{,}6^2 - 0{,}4^2 = 0{,}48$. Giả sử một phép tách tạo ra nút trái gồm 4 mẫu đều là "Có" (entropy 0) và nút phải gồm 6 mẫu với 2 "Có", 4 "Không" (entropy ≈ 0,918). Độ lợi thông tin là $0{,}971 - (0{,}4 \times 0 + 0{,}6 \times 0{,}918) \approx 0{,}420$.

Cây quyết định dễ hiểu, không cần chuẩn hóa dữ liệu, nhưng cây sâu dễ **quá khớp** và thay đổi nhỏ trong dữ liệu có thể tạo cây hoàn toàn khác (phương sai cao); khắc phục bằng giới hạn độ sâu hoặc cắt tỉa.

2.11.2. Rừng ngẫu nhiên và boosting

Các **phương pháp tổ hợp** (ensemble) kết hợp nhiều mô hình thành một mô hình mạnh hơn.

- **Rừng ngẫu nhiên** của Breiman [15] huấn luyện nhiều cây độc lập, mỗi cây trên một mẫu bootstrap và chỉ xét tập con ngẫu nhiên các đặc trưng ở mỗi nút, rồi bỏ phiếu đa số – giảm mạnh phương sai so với một cây.
- **Boosting** huấn luyện các mô hình yếu *tuần tự*, mỗi mô hình sửa lỗi của các mô hình trước (AdaBoost, Gradient Boosting); XGBoost, LightGBM vẫn là lựa chọn hàng đầu cho dữ liệu dạng bảng.

2.12. Máy vector hỗ trợ (SVM)

2.12.1. SVM cho bài toán phân loại

SVM [14] tìm siêu phẳng $\mathbf{w}^\top \mathbf{x} + b = 0$ phân tách hai lớp sao cho **biên** (margin) – khoảng cách từ siêu phẳng tới các điểm gần nhất của mỗi lớp – là lớn nhất. Các điểm nằm trên biên được gọi là *vector hỗ trợ*; chỉ chúng quyết định vị trí siêu phẳng. Độ rộng biên bằng $2/\|\mathbf{w}\|$, nên bài toán SVM biên cứng là:

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{với ràng buộc} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \ \forall i$$

với nhãn $y_i \in \{-1, +1\}$. Đây là bài toán tối ưu lồi bậc hai có ràng buộc; giải qua **bài toán đối ngẫu Lagrange**, điều kiện KKT cho nghiệm $\mathbf{w}^* = \sum_i \alpha_i y_i \mathbf{x}_i$, trong đó chỉ các điểm nằm đúng trên biên mới có $\alpha_i > 0$ – đó chính là các vector hỗ trợ.

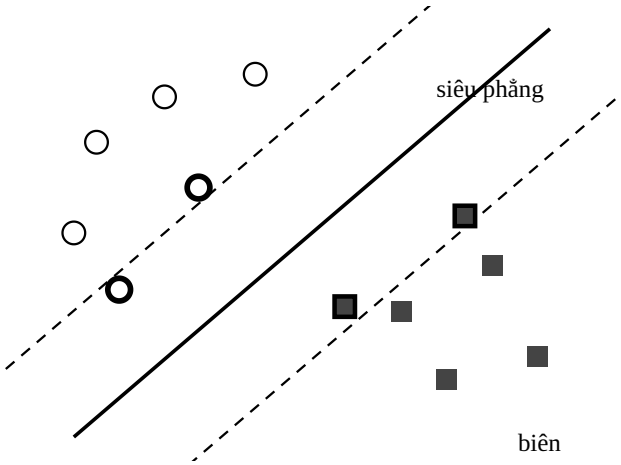
Khi dữ liệu có nhiễu hoặc chồng lấn nhẹ, **SVM biên mềm** thêm các biến bù $\xi_i \geq 0$ cho phép một số điểm vi phạm biên:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{với} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i$$

Bài toán này tương đương với cực tiểu hóa **hàm mất mát bản lề** (hinge loss) có điều chuẩn L2: $J = \sum_i \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big) + \frac{\lambda}{2}\|\mathbf{w}\|^2$, nên cũng giải được bằng hạ gradient. Siêu tham số C điều khiển sự đánh đổi giữa độ rộng biên và sai số huấn luyện: C lớn phạt nặng vi phạm (gần biên cứng, dễ quá khớp), C nhỏ cho biên rộng hơn. Khi dữ liệu không phân tách tuyến tính, **thủ thuật kernel** ánh xạ ngầm dữ liệu sang không gian nhiều chiều hơn – nơi các lớp có thể tách được – chỉ thông qua hàm tích vô hướng $K(\mathbf{x}, \mathbf{x}')$, chẳng hạn kernel tuyến tính $\mathbf{x}^\top \mathbf{x}'$, kernel đa thức $(r + \gamma\, \mathbf{x}^\top \mathbf{x}')^d$, kernel sigmoid $\tanh(\gamma\, \mathbf{x}^\top \mathbf{x}' + r)$ hoặc kernel RBF $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$. Vì bài toán đối ngẫu chỉ chứa các tích vô hướng $\mathbf{x}_i^\top \mathbf{x}_j$, thay

chúng bằng $K(\mathbf{x}_i, \mathbf{x}_j)$ là đủ – không cần tính tường minh ánh xạ sang không gian nhiều chiều.

SVM có nền tảng lý thuyết vững chắc, hiệu quả với dữ liệu nhiều chiều và kích thước vừa phải (như phân loại văn bản). Hạn chế là chi phí huấn luyện tăng nhanh (xấp xỉ bậc hai đến bậc ba) theo số mẫu, việc chọn kernel và siêu tham số C, γ cần thử nghiệm, và mô hình không trực tiếp cho ra xác suất.



Hình 2.2. SVM tìm siêu phẳng có biên lớn nhất (các điểm viền đậm là vector hỗ trợ)

2.12.2. Hồi quy vector hỗ trợ (SVR)

SVR [84] là phiên bản hồi quy của SVM. Thay vì tìm siêu phẳng phân tách, SVR tìm hàm $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ sao cho phần lớn dự đoán nằm trong một **ống sai số** độ rộng ε quanh giá trị thực, đồng thời giữ $\|\mathbf{w}\|$ nhỏ để hàm "phẳng" và tổng quát tốt. Sai số nhỏ hơn ε không bị phạt; chỉ các điểm nằm ngoài ống mới bị phạt qua biến bù ξ_i, ξ_i^* :

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i (\xi_i + \xi_i^*) \quad \text{với} \quad \begin{cases} y_i - f(\mathbf{x}_i) \le \varepsilon + \xi_i \\ f(\mathbf{x}_i) - y_i \le \varepsilon + \xi_i^* \end{cases}$$

Hàm mất mát tương ứng là **mất mát không nhạy** ε (ε-insensitive loss) $\max(0, |y - f(\mathbf{x})| - \varepsilon)$ – tăng tuyến tính chứ không bình phương như MSE, nên SVR ít bị ngoại lai chi phối hơn hồi quy tuyến tính. C điều khiển mức phạt sai số lớn, ε quyết định độ rộng vùng không phạt; các điểm nằm trên hoặc ngoài ống là vector hỗ trợ. Giống SVM, SVR nhạy với thang đo nên cần chuẩn hóa đặc trưng; với quan hệ phi tuyến có thể dùng kernel RBF hoặc đa thức nhưng chi phí tăng nhanh theo số mẫu, nên với dữ liệu lớn thường dùng SVR tuyến tính (LinearSVR).

2.13. Gom cụm K-means

K-means là thuật toán gom cụm phổ biến nhất, chia n điểm dữ liệu thành k cụm sao cho tổng bình phương khoảng cách từ mỗi điểm tới tâm cụm của nó là nhỏ nhất:

$$J = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

Thuật toán (Lloyd) gồm các bước: (1) khởi tạo ngẫu nhiên k tâm cụm; (2) *bước gán*: gán mỗi điểm vào cụm có tâm gần nhất; (3) *bước cập nhật*: tính lại mỗi tâm bằng trung bình các điểm trong cụm; (4) lặp lại bước 2–3 đến khi các tâm không thay đổi đáng kể. Mỗi vòng lặp không làm tăng J, nên thuật toán luôn hội tụ – tuy nhiên chỉ tới cực tiểu địa phương.

K-means nhanh, dễ cài đặt và mở rộng tốt cho dữ liệu lớn. Hạn chế: phải chọn trước k, kết quả phụ thuộc khởi tạo, nhạy với ngoại lai, và giả định các cụm có dạng cầu, kích thước tương đương. Với cụm hình dạng bất kỳ có thể dùng DBSCAN; với cấu trúc phân cấp có thể dùng gom cụm phân cấp.

Để giảm phụ thuộc vào khởi tạo, **K-means++** [85] chọn tâm đầu tiên ngẫu nhiên, mỗi tâm tiếp theo được chọn với xác suất tỉ lệ với $D(\mathbf{x})^2$ – bình phương khoảng cách từ điểm tới tâm gần nhất đã chọn – nhờ đó các tâm ban đầu trải đều; thuật toán còn được chạy nhiều lần với khởi tạo khác nhau rồi giữ kết quả có J nhỏ nhất. Để chọn k, **phương pháp khuỷu tay** (elbow) vẽ J theo k và chọn điểm mà J ngừng giảm mạnh; **chỉ số silhouette** của một điểm là $s = (b - a)/\max(a, b)$, với a là khoảng cách trung bình tới các điểm cùng cụm và b là khoảng cách trung bình tới cụm gần nhất khác. s gần 1 nghĩa là điểm nằm đúng cụm, gần 0 là nằm ở biên giữa hai cụm, âm là có thể bị gán nhầm; k tốt cho silhouette trung bình lớn nhất.

2.14. Giảm chiều dữ liệu với PCA

Phân tích thành phần chính (PCA) tìm một hệ trục tọa độ mới sao cho trục thứ nhất là hướng dữ liệu có phương sai lớn nhất, trục thứ hai vuông góc với trục thứ nhất và có phương sai lớn thứ hai, v.v. Về mặt toán học, các trục này là các vector riêng của ma trận hiệp phương sai của dữ liệu (đã trừ trung bình), sắp xếp theo trị riêng giảm dần. Giữ lại m thành phần đầu ($m \ll d$) cho phép biểu diễn dữ liệu bằng ít chiều hơn mà vẫn giữ phần lớn thông tin (phương sai).

PCA được dùng để trực quan hóa dữ liệu nhiều chiều trên mặt phẳng, giảm nhiễu, nén dữ liệu và tăng tốc các thuật toán khác. Hạn chế là PCA chỉ nắm bắt quan

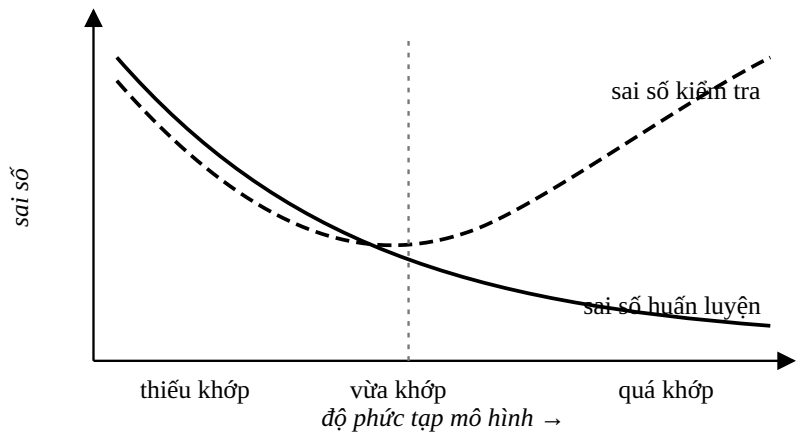
hệ tuyến tính, và các thành phần chính là tổ hợp của nhiều đặc trưng gốc nên khó diễn giải ý nghĩa. Với cấu trúc phi tuyến, có thể dùng t-SNE, UMAP hoặc autoencoder.

2.15. Quá khớp, thiếu khớp và điều chuẩn

Thiếu khớp (underfitting) xảy ra khi mô hình quá đơn giản để nắm bắt cấu trúc dữ liệu: sai số cao trên cả tập huấn luyện lẫn tập kiểm tra. **Quá khớp** (overfitting) xảy ra khi mô hình quá phức tạp, học cả nhiễu của dữ liệu huấn luyện: sai số huấn luyện rất thấp nhưng sai số kiểm tra cao. Hiện tượng này gắn với **sự đánh đổi độ chệch – phương sai** (bias–variance trade-off): sai số kỳ vọng trên dữ liệu mới có thể phân tách thành

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \mathrm{Bias}^2 + \mathrm{Variance} + \sigma^2_{\text{nhiễu}}$$

Mô hình đơn giản có độ chệch cao nhưng phương sai thấp; mô hình phức tạp có độ chệch thấp nhưng phương sai cao. Mục tiêu là tìm độ phức tạp cân bằng giữa hai thành phần (Hình 2.3).



Hình 2.3. Sai số huấn luyện và sai số kiểm tra theo độ phức tạp của mô hình

Chống quá khớp bằng thêm dữ liệu, giảm độ phức tạp, dừng sớm, tổ hợp mô hình và **điều chuẩn** – thêm số hạng phạt độ lớn trọng số vào hàm mất mát:

$$J_{\mathrm{L2}}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_j w_j^2, \qquad J_{\mathrm{L1}}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_j |w_j|$$

L2 (Ridge) làm các trọng số nhỏ đều; L1 (Lasso) đưa nhiều trọng số về 0, đồng thời lựa chọn đặc trưng; λ được chọn bằng tập xác thực.

Tổng quát hóa toàn bộ các mô hình có giám sát của chương, huấn luyện chính là **cực tiểu hóa rủi ro thực nghiệm có điều chuẩn**:

$$\hat{\theta} = \arg\min_{\theta} \; \frac{1}{n} \sum_{i=1}^{n} \ell\big(y_i, f_\theta(\mathbf{x}_i)\big) + \lambda\, \Omega(\theta)$$

Mỗi thuật toán chỉ khác nhau ở họ hàm f_θ, hàm mất mát ℓ (bình phương cho hồi quy tuyến tính, entropy chéo cho hồi quy logistic/softmax, bản lề cho SVM, không nhạy ε cho SVR) và số hạng điều chuẩn Ω.

2.16. So sánh các thuật toán

Bảng 2.2. So sánh một số thuật toán học máy cơ bản

Thuật toán	Loại bài toán	Ưu điểm chính	Hạn chế chính
Hồi quy tuyến tính	Hồi quy	Đơn giản, nhanh, dễ giải thích	Chỉ mô hình hóa quan hệ tuyến tính, nhạy ngoại lai
Hồi quy logistic	Phân loại	Cho xác suất, dễ giải thích, làm baseline tốt	Ranh giới quyết định tuyến tính
KNN	Phân loại, hồi quy	Không cần huấn luyện, ranh giới linh hoạt	Dự đoán chậm, cần chuẩn hóa, kém khi nhiều chiều
Naive Bayes	Phân loại	Rất nhanh, cần ít dữ liệu, tốt với văn bản	Giả định độc lập hiếm khi đúng
Cây quyết định	Phân loại, hồi quy	Trực quan, không cần chuẩn hóa	Dễ quá khớp, không ổn định
Rừng ngẫu nhiên / Boosting	Phân loại, hồi quy	Độ chính xác cao trên dữ liệu bảng	Khó giải thích, tốn tài nguyên hơn
SVM	Phân loại (chủ yếu)	Hiệu quả với dữ liệu nhiều chiều, có kernel	Chậm với dữ liệu lớn, khó chọn siêu tham số
K-means	Gom cụm	Nhanh, dễ cài đặt	Cần chọn k, giả định cụm dạng cầu
PCA	Giảm chiều	Giảm nhiễu, trực quan hóa	Tuyến tính, khó diễn giải thành phần

Theo định lý **"không có bữa trưa miễn phí"** (No Free Lunch), không tồn tại thuật toán nào tốt nhất cho mọi bài toán. Việc lựa chọn cần dựa trên kích thước và dạng dữ liệu, yêu cầu về khả năng giải thích, giới hạn tài nguyên và hậu quả của sai lầm. Trong thực tế, nên bắt đầu từ mô hình đơn giản làm cơ sở so sánh rồi mới tăng dần độ phức tạp.

2.17. Từ học máy truyền thống đến mạng nơ-ron

Các thuật toán kể trên hoạt động tốt khi dữ liệu đã được biểu diễn bằng những đặc trưng có ý nghĩa. Tuy nhiên, với dữ liệu thô như ảnh (hàng trăm nghìn điểm ảnh), âm thanh hay văn bản, việc thiết kế đặc trưng thủ công vừa tốn công vừa khó đạt hiệu

quả cao. **Mạng nơ-ron nhân tạo** giải quyết vấn đề này bằng cách học đồng thời cả biểu diễn và bộ phân loại.

2.17.1. Perceptron và thuật toán học perceptron (PLA)

Perceptron của Rosenblatt [82] (mục 1.3) là bộ phân loại nhị phân $\hat{y} = \operatorname{sign}(\mathbf{w}^\top \mathbf{x})$ với $y \in \{-1, +1\}$ (hệ số chặn được gộp vào $\mathbf{w}$ bằng cách thêm đặc trưng hằng $x_0 = 1$). **Thuật toán học perceptron** (Perceptron Learning Algorithm – PLA) rất đơn giản: lặp qua dữ liệu, mỗi khi gặp một điểm bị phân loại sai ($y_i \, \mathbf{w}^\top \mathbf{x}_i \le 0$) thì cập nhật

$$\mathbf{w} \leftarrow \mathbf{w} + y_i \, \mathbf{x}_i$$

Về hình học, việc cộng (hoặc trừ) $\mathbf{x}_i$ vào $\mathbf{w}$ xoay vector pháp tuyến của siêu phẳng về phía phân loại đúng điểm đó, vì $y_i \, \mathbf{w}_{\text{mới}}^\top \mathbf{x}_i = y_i \, \mathbf{w}^\top \mathbf{x}_i + \|\mathbf{x}_i\|^2$ tăng lên. **Định lý hội tụ Novikoff** [83] (1962) chứng minh: nếu dữ liệu phân tách tuyến tính được với biên γ và mọi điểm có $\|\mathbf{x}_i\| \le R$, PLA dừng sau không quá $(R/\gamma)^2$ lần cập nhật. Ngược lại, khi dữ liệu có nhiễu hoặc không phân tách tuyến tính (như XOR – mục 1.4), PLA lặp vô hạn; khi đó dùng biến thể **Pocket** (giữ lại nghiệm tốt nhất từng gặp), hoặc chuyển sang hồi quy logistic, SVM biên mềm – những mô hình có hàm mất mát trơn hoặc chấp nhận vi phạm. Cũng có thể xem PLA là hạ gradient ngẫu nhiên với tốc độ học 1 trên hàm mất mát $\max(0, -y_i \, \mathbf{w}^\top \mathbf{x}_i)$.

2.17.2. Perceptron nhiều lớp và lan truyền ngược

Một nơ-ron nhân tạo tính $a = g(\mathbf{w}^\top \mathbf{x} + b)$, trong đó g là hàm kích hoạt phi tuyến (sigmoid, tanh, ReLU). **Perceptron nhiều lớp** (MLP) xếp chồng nhiều lớp nơ-ron: đầu ra của lớp này là đầu vào của lớp sau. Định lý xấp xỉ vạn năng cho biết một mạng với chỉ một lớp ẩn đủ rộng có thể xấp xỉ bất kỳ hàm liên tục nào trên miền compact với độ chính xác tùy ý. Mạng được huấn luyện bằng **lan truyền ngược**: áp dụng quy tắc đạo hàm hàm hợp (chain rule) để tính gradient của hàm mất mát theo mọi trọng số, lan từ lớp cuối ngược về lớp đầu, rồi cập nhật bằng hạ gradient.

Tuy nhiên, MLP kết nối đầy đủ không khai thác cấu trúc đặc thù của dữ liệu: nó coi các điểm ảnh liền kề hay các từ liên tiếp như những đặc trưng độc lập, không có thứ tự. Hai kiến trúc được trình bày ở các chương sau khắc phục điều này bằng cách đưa vào những **giả định quy nạp** (inductive bias) phù hợp: CNN khai thác tính cục bộ và tính bất biến tịnh tiến của dữ liệu không gian; RNN khai thác tính tuần tự của dữ liệu theo thời gian.

2.18. Thực nghiệm: hồi quy softmax trên ba bộ dữ liệu

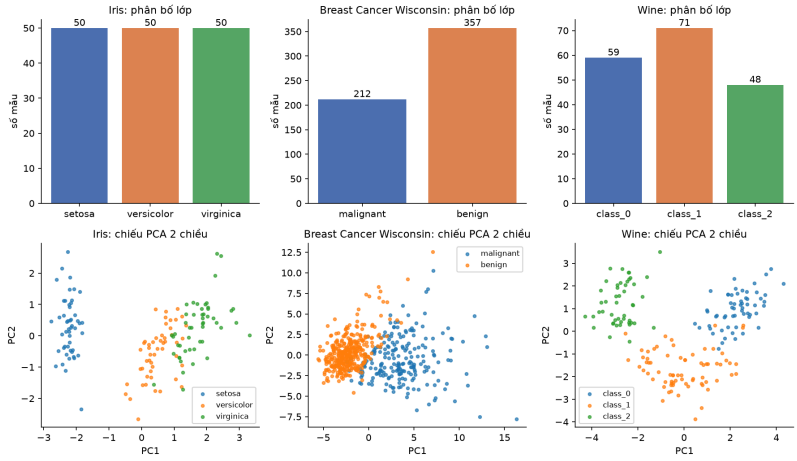
Phần này kiểm chứng các khái niệm của chương bằng **hồi quy softmax** (hồi quy logistic nhiều lớp), cài đặt theo ba cách với cùng siêu tham số và khởi tạo: tự viết bằng NumPy (tính gradient thủ công), Keras [38] và PyTorch [37]; kết quả được đối chiếu với sáu thuật toán của chương cài đặt bằng scikit-learn [36]. Mã nguồn đầy đủ nằm trong notebook `code/ch2_hoi_quy_softmax.ipynb`; các đoạn mã cốt lõi được trích ở mục 2.18.2.

2.18.1. Dữ liệu và thiết lập

Ba bộ dữ liệu dạng bảng từ kho UCI được chọn để có số lớp, số chiều và mức cân bằng khác nhau (Bảng 2.3). Không bộ nào có giá trị thiếu, nhưng thang đo đặc trưng chênh lệch rất lớn (Breast Cancer: 0 – 4.254; Wine: 0,13 – 1.680), nên dữ liệu được **chuẩn hóa z-score** bằng thống kê *chỉ của tập huấn luyện* để tránh rò rỉ dữ liệu (mục 2.4). Trên hình chiếu PCA (Hình 2.4), các lớp gần như tách rời nhau – gợi ý rằng bộ phân loại **tuyến tính** đã đủ mạnh.

Bảng 2.3. Ba bộ dữ liệu dùng trong thực nghiệm chương 2

Bộ dữ liệu	Số mẫu	Số đặc trưng	Số lớp và phân bố	Mô tả
Iris [39]	150	4	3 lớp: 50 / 50 / 50	Kích thước đài và cánh hoa của ba loài diên vĩ
Breast Cancer Wisconsin [40]	569	30	2 lớp: 212 ác tính / 357 lành tính	Đặc trưng nhân tế bào từ ảnh sinh thiết
Wine	178	13	3 lớp: 59 / 71 / 48	Thành phần hóa học của rượu từ ba giống nho



Hình 2.4. Phân bố lớp (hàng trên) và hình chiếu PCA hai chiều sau chuẩn hóa (hàng dưới)

Mô hình tối thiểu hóa entropy chéo cộng điều chuẩn L2 $\frac{\lambda}{2}\|W\|^2$ bằng hạ gradient ngẫu nhiên theo lô nhỏ; bản NumPy dùng công thức giải tích $\partial L/\partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$. Siêu tham số: tốc độ học 0,1; 100 epoch; lô 16; $\lambda = 10^{-3}$; khởi tạo $\mathcal{N}(0;\, 0{,}01^2)$. Dữ liệu chia 80/20 có phân tầng, lặp lại với **5 hạt giống** (5 cách chia); kết quả ghi trung bình ± độ lệch chuẩn. Sáu mô hình đối chứng dùng siêu tham số mặc định và kiểm định chéo phân tầng 5 phần.

2.18.2. Cài đặt

Đoạn mã 2.1 là phần cốt lõi của bản NumPy: `softmax` trừ giá trị lớn nhất trước khi lấy mũ để tránh tràn số, và `fit` cài đặt trực tiếp gradient $\partial L/\partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$ cùng quy tắc cập nhật của hạ gradient theo lô nhỏ (mục 2.6), có thêm số hạng λW của điều chuẩn L2. Với thư viện chỉ cần khai báo mô hình (Đoạn mã 2.2 với PyTorch; Keras tương tự với một lớp `Dense`); gradient do thư viện tự tính bằng vi phân tự động.

Đoạn mã 2.1. Hồi quy softmax bằng NumPy (hàm softmax và vòng lặp huấn luyện)

```
def softmax(z):
    z = z - z.max(axis=1, keepdims=True)  # ổn định số học
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

class SoftmaxRegressionNP:
    # __init__: W ~ N(0; 0,01²), b = 0, lưu các siêu tham số

    def fit(self, X, y):
        Y = np.eye(self.k, dtype=np.float32)[y]  # mã hóa one-hot
        history = []
        for _ in range(self.epochs):
            idx = self.rng.permutation(len(X))  # xáo trộn mỗi epoch
            for s in range(0, len(X), self.bs):
                b = idx[s : s + self.bs]
                P = softmax(X[b] @ self.W + self.b)
                G = (P - Y[b]) / len(b)  # dL/dz
                self.W -= self.lr * (X[b].T @ G + self.l2 * self.W)
                self.b -= self.lr * G.sum(axis=0)
            history.append(float(self.loss(X, y)))
        return history
```

Đoạn mã 2.2. Khai báo mô hình bằng PyTorch

```
class SoftmaxRegressionTorch(nn.Module):
    def __init__(self, n_features, n_classes):
        super().__init__()
        self.linear = nn.Linear(n_features, n_classes)
        nn.init.normal_(self.linear.weight, 0, 0.01)
        nn.init.zeros_(self.linear.bias)

    def forward(self, x):
        return self.linear(x)  # logits
```

2.18.3. Kết quả và nhận xét

Bảng 2.4. Độ chính xác (%) của hồi quy softmax theo ba cách cài đặt (trung bình ± độ lệch chuẩn qua 5 lần chia)

Bộ dữ liệu	Số tham số	NumPy	Keras	PyTorch
Iris	15	96,67 ± 3,65	96,00 ± 4,90	96,67 ± 3,65
Breast Cancer	62	98,25 ± 0,55	98,07 ± 0,35	98,25 ± 0,55
Wine	42	98,89 ± 1,36	98,89 ± 1,36	98,89 ± 1,36

Ba cách cài đặt cho kết quả gần như trùng khớp (F1 macro cũng vậy, xem notebook): NumPy và PyTorch giống nhau đến từng chữ số, Keras chỉ lệch một mẫu kiểm tra ở Iris do khác thứ tự xáo trộn lô. Cùng với phép so sánh với gradient số học trong notebook (sai số tương đối khoảng 10^{-10}), điều này xác nhận công thức gradient tự cài đặt là đúng. Với dữ liệu nhỏ, bản NumPy nhanh nhất (0,02 – 0,06 s mỗi lần huấn luyện) vì không có chi phí khởi tạo, PyTorch mất 0,3 – 0,8 s, còn Keras chậm nhất (3 – 4 s) do phải dựng đồ thị tính toán; lợi thế của thư viện chỉ rõ ở chương 3 và 4.

Bảng 2.5. Độ chính xác (%) của các thuật toán cơ bản, kiểm định chéo 5 phần (trung bình ± độ lệch chuẩn)

Thuật toán	Iris	Breast Cancer	Wine
Hồi quy logistic	**96,00 ± 1,33**	**97,89 ± 1,42**	**98,32 ± 1,37**
KNN ($k = 5$)	95,33 ± 3,40	96,49 ± 2,15	96,08 ± 1,34
Naive Bayes	**96,00 ± 1,33**	92,97 ± 1,24	97,19 ± 0,04
Cây quyết định	93,33 ± 4,22	92,62 ± 2,18	92,73 ± 5,13
Rừng ngẫu nhiên	94,00 ± 2,49	96,67 ± 1,87	97,76 ± 2,08
SVM (RBF)	95,33 ± 1,63	97,71 ± 0,70	98,30 ± 1,39

Bảng 2.5 minh họa nhiều nhận định của chương:

- **Mô hình tuyến tính là đủ:** hồi quy logistic dẫn đầu cả ba bộ, phù hợp với hình chiếu PCA.
- **Cây quyết định đơn lẻ kém và dao động nhất** (độ lệch chuẩn tới 5,1 điểm trên Wine) do dễ quá khớp (mục 2.11); rừng ngẫu nhiên khắc phục phần lớn nhược điểm này.
- **Naive Bayes giảm mạnh trên Breast Cancer** (93,0%) vì các đặc trưng (bán kính, chu vi, diện tích nhân tế bào) tương quan chặt, vi phạm giả định độc lập.
- **Dữ liệu nhỏ cho độ lệch chuẩn lớn:** tập kiểm tra Iris chỉ có 30 mẫu nên một mẫu sai đã làm đổi 3,3 điểm – lý do nên dùng nhiều lần chia hoặc kiểm định

chéo (mục 2.5.1).

2.19. Tiểu kết chương 2

Chương 2 đã trình bày các khái niệm nền tảng của học máy – các dạng học, quy trình xây dựng mô hình, tiền xử lý, đánh giá và tối ưu hạ gradient – cùng nguyên lý, ưu điểm và hạn chế của các thuật toán tiêu biểu, và các vấn đề xuyên suốt như quá khớp, rò rỉ dữ liệu, dữ liệu mất cân bằng. Thực nghiệm với hồi quy softmax cho thấy cài đặt thủ công bằng NumPy trùng khớp với Keras và PyTorch (96,7% – 98,9%), đồng thời minh họa các nhận định của chương về mô hình tuyến tính, cây quyết định và Naive Bayes. Đây là nền tảng cho CNN và RNN ở các chương sau – cũng là những mô hình được huấn luyện bằng cách tối thiểu hóa hàm mất mát và đánh giá theo chính các nguyên tắc trên.

CHƯƠNG 3. MẠNG NƠ-RON TÍCH CHẬP (CNN)

3.1. Động lực và nguồn gốc của CNN

3.1.1. Hạn chế của mạng kết nối đầy đủ với dữ liệu ảnh

Một ảnh màu kích thước 224 × 224 điểm ảnh với 3 kênh màu (R, G, B) tương ứng với vector đầu vào có 224 × 224 × 3 = 150.528 chiều. Nếu nối trực tiếp vào một lớp kết nối đầy đủ chỉ có 1.000 nơ-ron, lớp này đã cần khoảng **150,5 triệu trọng số**. Số tham số khổng lồ như vậy gây ra ba vấn đề: tốn bộ nhớ và tính toán; cần lượng dữ liệu rất lớn để tránh quá khớp; và quan trọng nhất, mạng **không tận dụng được cấu trúc không gian** của ảnh. Với MLP, việc hoán đổi vị trí các điểm ảnh (miễn là nhất quán cho mọi ảnh) không ảnh hưởng gì tới khả năng học – nghĩa là mạng hoàn toàn "không biết" rằng hai điểm ảnh cạnh nhau có liên quan chặt chẽ hơn hai điểm ở xa nhau.

Dữ liệu ảnh có hai tính chất quan trọng mà một kiến trúc tốt nên khai thác:

- **Tính cục bộ** (locality): các đặc trưng có ý nghĩa như cạnh, góc, kết cấu được hình thành từ những điểm ảnh lân cận nhau.
- **Tính bất biến (đồng biến) tịnh tiến** (translation equivariance): một đặc trưng – chẳng hạn mắt mèo – có ý nghĩa như nhau dù xuất hiện ở góc trái hay góc phải ảnh. Do đó, bộ phát hiện đặc trưng học được ở một vị trí nên được dùng lại ở mọi vị trí khác.

3.1.2. Nguồn cảm hứng sinh học và lịch sử

CNN lấy cảm hứng từ hệ thị giác của động vật. Trong các thí nghiệm cuối thập niên 1950 – 1960, David Hubel và Torsten Wiesel phát hiện rằng các tế bào trong vỏ não thị giác của mèo chỉ phản ứng với kích thích trong một vùng nhỏ của trường nhìn (*trường tiếp nhận* – receptive field); có tế bào "đơn giản" phản ứng với cạnh theo một hướng nhất định, và tế bào "phức tạp" phản ứng với cùng loại cạnh đó trong một vùng rộng hơn. Phát hiện này mang lại cho họ giải Nobel Sinh lý học năm 1981.

Năm 1980, Kunihiko Fukushima đề xuất mạng **Neocognitron** gồm các lớp xen kẽ mô phỏng tế bào đơn giản và phức tạp – tiền thân trực tiếp của CNN. Năm 1989, Yann LeCun và cộng sự lần đầu kết hợp kiến trúc này với thuật toán lan truyền ngược để nhận dạng mã bưu chính viết tay, và năm 1998 công bố **LeNet-5** [5]. Sau hơn một thập kỷ ít được chú ý, CNN bùng nổ trở lại với thắng lợi của AlexNet năm 2012 và từ đó trở thành kiến trúc chủ đạo của thị giác máy tính.

3.2. Phép tích chập

3.2.1. Định nghĩa

Thành phần cốt lõi của CNN là **lớp tích chập**. Một **bộ lọc** (filter, hay kernel) là một ma trận trọng số kích thước nhỏ, ví dụ 3 × 3. Bộ lọc được trượt lần lượt trên toàn bộ ảnh đầu vào; tại mỗi vị trí, các phần tử của bộ lọc được nhân với các phần tử tương ứng của vùng ảnh bên dưới rồi cộng tất cả lại (cộng thêm hệ số điều chỉnh b) để được một giá trị đầu ra. Tập hợp các giá trị đầu ra tạo thành một **bản đồ đặc trưng** (feature map). Với đầu vào hai chiều X và bộ lọc K kích thước $f \times f$:

$$Y(i,j) = \sum_{m=0}^{f-1}\sum_{n=0}^{f-1} X(i+m,\, j+n)\, K(m,n) + b$$

Về mặt toán học chặt chẽ, công thức trên là phép **tương quan chéo** (cross-correlation); phép tích chập thực sự sẽ lật bộ lọc trước khi nhân. Tuy nhiên, vì bộ lọc là tham số được học, việc lật hay không không ảnh hưởng đến khả năng biểu diễn, nên các thư viện học sâu đều cài đặt tương quan chéo và vẫn gọi là "tích chập".

Với ảnh nhiều kênh (ví dụ ảnh RGB có 3 kênh, hoặc bản đồ đặc trưng có C_{in} kênh), mỗi bộ lọc có kích thước $f \times f \times C_{in}$, tức là trải dài qua toàn bộ chiều sâu của đầu vào, và cho ra *một* bản đồ đặc trưng. Một lớp tích chập dùng C_{out} bộ lọc sẽ tạo ra C_{out} bản đồ đặc trưng xếp chồng lên nhau – đó chính là đầu ra có C_{out} kênh.

3.2.2. Ví dụ tính toán

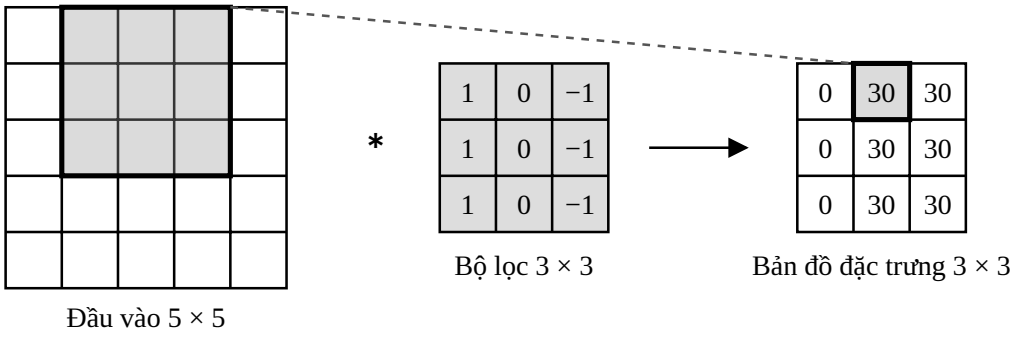
Xét ảnh xám kích thước 5 × 5 gồm nửa trái sáng (giá trị 10) và nửa phải tối (giá trị 0), và bộ lọc 3 × 3 phát hiện cạnh dọc:

$$X = \begin{bmatrix} 10 & 10 & 10 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Tại vị trí góc trên trái, vùng ảnh là ba cột giá trị 10, nên kết quả bằng $3 \times 10 \times 1 + 3 \times 10 \times 0 + 3 \times 10 \times (-1) = 0$. Dịch bộ lọc sang phải một cột, vùng ảnh gồm hai cột 10 và một cột 0, kết quả bằng $30 + 0 - 0 = 30$. Tiếp tục như vậy, ta được bản đồ đặc trưng kích thước 3 × 3:

$$Y = \begin{bmatrix} 0 & 30 & 30 \\ 0 & 30 & 30 \\ 0 & 30 & 30 \end{bmatrix}$$

Các giá trị lớn tập trung đúng tại vị trí có sự chuyển tiếp từ sáng sang tối – bộ lọc đã "phát hiện" cạnh dọc. Ở vùng đồng nhất, đầu ra bằng 0. Trong xử lý ảnh cổ điển, những bộ lọc như vậy (Sobel, Prewitt…) được thiết kế thủ công; điểm đột phá của CNN là **các giá trị của bộ lọc được học tự động** từ dữ liệu thông qua lan truyền ngược, nên mạng tự tìm ra các bộ lọc hữu ích nhất cho nhiệm vụ.



Hình 3.1. Phép tích chập: vùng 3 × 3 tô đậm của đầu vào cho ra một giá trị ở bản đồ đặc trưng

3.2.3. Ba ý tưởng then chốt

Phép tích chập mang lại ba lợi ích quan trọng so với lớp kết nối đầy đủ:

1. **Kết nối thưa** (sparse connectivity): mỗi nơ-ron đầu ra chỉ kết nối với một vùng nhỏ của đầu vào (trường tiếp nhận cục bộ), thay vì toàn bộ.
2. **Chia sẻ tham số** (parameter sharing): cùng một bộ lọc được dùng ở mọi vị trí, nên số tham số không phụ thuộc vào kích thước ảnh. Một lớp tích chập 3 × 3 với 64 kênh vào và 128 kênh ra chỉ có $3 \times 3 \times 64 \times 128 + 128 = 73.856$ tham số, bất kể ảnh lớn hay nhỏ.
3. **Đồng biến tịnh tiến** (equivariance): nếu đối tượng dịch chuyển trong ảnh, đặc trưng tương ứng trên bản đồ đặc trưng cũng dịch chuyển theo – mạng nhận ra đặc trưng ở bất kỳ vị trí nào.

3.3. Bước trượt (stride) và đệm (padding)

Bước trượt s là số điểm ảnh bộ lọc dịch chuyển sau mỗi lần tính. Với $s = 1$, bộ lọc dịch từng điểm một; với $s = 2$, kích thước đầu ra giảm khoảng một nửa theo mỗi chiều. **Đệm** p là số hàng/cột (thường mang giá trị 0) được thêm vào xung quanh biên ảnh. Không có đệm, kích thước ảnh co lại sau mỗi lớp tích chập và các điểm ảnh ở biên ít được "nhìn" tới hơn các điểm ở giữa. Với đầu vào kích thước n, bộ lọc kích thước f, kích thước đầu ra theo mỗi chiều là:

$$n_{out} = \left\lfloor \frac{n + 2p - f}{s} \right\rfloor + 1$$

Ví dụ trên có $n = 5$, $f = 3$, $p = 0$, $s = 1$ nên $n_{out} = 3$. Hai chế độ phổ biến là *valid* ($p = 0$) và *same* (chọn $p = (f - 1)/2$ với $s = 1$ để đầu ra cùng kích thước với đầu vào; với $f = 3$ thì $p = 1$). Bên cạnh đó, **tích chập giãn** (dilated convolution) chèn khoảng trống giữa các phần tử bộ lọc để mở rộng trường tiếp nhận mà không tăng số tham số, thường dùng trong phân đoạn ảnh.

3.4. Hàm kích hoạt

Phép tích chập là phép toán tuyến tính; nếu chỉ xếp chồng các lớp tuyến tính thì toàn bộ mạng vẫn chỉ tương đương một phép biến đổi tuyến tính. Vì vậy, sau mỗi lớp tích chập luôn có một **hàm kích hoạt phi tuyến**. Hàm phổ biến nhất trong CNN là **ReLU**:

$$\mathrm{ReLU}(x) = \max(0, x)$$

Áp dụng ReLU lên ví dụ ở mục 3.2.2, các giá trị dương được giữ nguyên còn giá trị âm (nếu có) bị đưa về 0. ReLU có ưu điểm tính toán rất nhanh, và đạo hàm bằng 1 với mọi $x > 0$ nên giảm đáng kể hiện tượng **biến mất gradient** – vấn đề nghiêm trọng của sigmoid và tanh vốn có đạo hàm rất nhỏ khi đầu vào lớn. Nhược điểm là các nơ-ron có thể "chết" nếu đầu vào luôn âm (gradient luôn bằng 0). Các biến thể như Leaky ReLU ($\max(0{,}01x, x)$), ELU hay GELU khắc phục điều này bằng cách cho phép một gradient nhỏ ở vùng âm. Ở lớp đầu ra, **softmax** được dùng cho phân loại đa lớp để chuyển các điểm số thành phân phối xác suất.

3.5. Lớp gộp (pooling)

Lớp gộp giảm kích thước không gian của bản đồ đặc trưng bằng cách tóm tắt từng vùng nhỏ thành một giá trị. **Gộp cực đại** (max pooling) lấy giá trị lớn nhất trong vùng, còn **gộp trung bình** (average pooling) lấy giá trị trung bình. Cấu hình phổ biến là cửa sổ 2 × 2, bước trượt 2, giúp giảm kích thước đi một nửa theo mỗi chiều (giảm 75% số phần tử). Ví dụ, với vùng 2 × 2 có các giá trị {1, 3, 2, 8}, max pooling cho 8 còn average pooling cho 3,5.

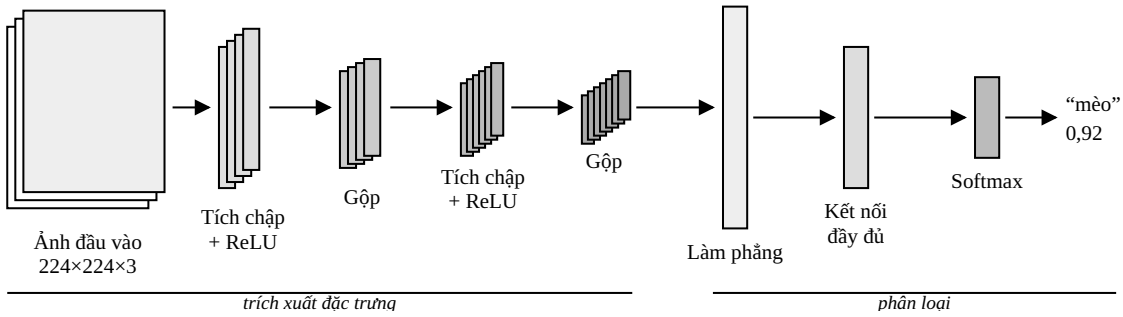
Lớp gộp không có tham số học. Tác dụng của nó là giảm chi phí tính toán cho các lớp sau, mở rộng trường tiếp nhận của các lớp sâu hơn, và tạo ra mức độ **bất biến với những dịch chuyển nhỏ** – đặc trưng vẫn được phát hiện dù lệch đi một vài điểm ảnh. Đổi lại, thông tin vị trí chi tiết bị mất đi. Trong nhiều kiến trúc hiện đại, lớp gộp được thay bằng tích chập với bước trượt 2. Ở cuối mạng, **gộp trung bình toàn cục**

(global average pooling) lấy trung bình mỗi bản đồ đặc trưng thành một số duy nhất, thay cho các lớp kết nối đầy đủ cồng kềnh.

3.6. Kiến trúc tổng quát của một CNN

Một CNN điển hình cho bài toán phân loại ảnh gồm hai phần (Hình 3.2):

- **Phần trích xuất đặc trưng:** các khối [Tích chập → (Chuẩn hóa) → ReLU] × k → Gộp; càng sâu, kích thước không gian giảm còn số kênh tăng (64 → 128 → 256 → 512).
- **Phần phân loại:** làm phẳng hoặc gộp toàn cục thành vector, qua vài lớp kết nối đầy đủ và softmax.



Hình 3.2. Kiến trúc tổng quát của một mạng nơ-ron tích chập cho bài toán phân loại ảnh

CNN **học biểu diễn phân cấp** (Zeiler và Fergus, 2014): lớp đầu phát hiện cạnh, góc, đốm màu; lớp giữa học kết cấu, họa tiết; lớp sâu phản ứng với bộ phận và toàn bộ đối tượng. Trường tiếp nhận tăng theo độ sâu: hai lớp 3 × 3 liên tiếp có trường tiếp nhận 5 × 5, ba lớp là 7 × 7.

3.7. Huấn luyện CNN

3.7.1. Hàm mất mát và lan truyền ngược

CNN được huấn luyện theo đúng khung tối ưu hóa đã trình bày ở Chương 2. Với phân loại C lớp, đầu ra softmax $\hat{\mathbf{y}}$ được so sánh với nhãn mã hóa one-hot $\mathbf{y}$ bằng hàm **entropy chéo**:

$$L = -\sum_{c=1}^{C} y_c \log \hat{y}_c$$

Do $\mathbf{y}$ chỉ có một phần tử bằng 1 tại lớp đúng, L chính là $-\log$ của xác suất mà mô hình gán cho lớp đúng: dự đoán xác suất 0,9 cho lớp đúng cho mất mát ≈ 0,105, còn dự đoán 0,1 cho mất mát ≈ 2,303. Mỗi vòng lặp huấn luyện gồm bốn bước: (1) *lan truyền tiến* một lô ảnh qua mạng để tính dự đoán; (2) tính mất mát; (3) *lan truyền*

ngược để tính gradient theo mọi tham số; (4) cập nhật tham số bằng SGD với momentum hoặc Adam. Đáng chú ý, gradient theo bộ lọc của lớp tích chập cũng có dạng một phép tích chập (giữa đầu vào và gradient của đầu ra), và do bộ lọc được chia sẻ, gradient của nó là tổng đóng góp từ mọi vị trí.

3.7.2. Chuẩn hóa theo lô (Batch Normalization)

Batch Normalization (Ioffe và Szegedy, 2015) [20] chuẩn hóa đầu ra của mỗi kênh trong một lô về trung bình 0, phương sai 1, sau đó co giãn và dịch chuyển bằng hai tham số học được γ, β:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta$$

Kỹ thuật này giúp quá trình huấn luyện ổn định hơn, cho phép dùng tốc độ học lớn hơn, giảm sự nhạy cảm với cách khởi tạo trọng số và có tác dụng điều chuẩn nhẹ. Khi suy luận, μ_B và σ_B được thay bằng trung bình trượt tích lũy trong quá trình huấn luyện.

3.7.3. Dropout và các kỹ thuật điều chuẩn

Dropout (Srivastava và cộng sự, 2014) [21] trong quá trình huấn luyện ngẫu nhiên "tắt" mỗi nơ-ron với xác suất *p* (thường 0,2 – 0,5). Điều này buộc mạng không phụ thuộc quá nhiều vào một số nơ-ron cụ thể, và có thể xem như huấn luyện đồng thời một tổ hợp rất lớn các mạng con. Khi suy luận, mọi nơ-ron đều hoạt động. Dropout thường được đặt ở các lớp kết nối đầy đủ. Ngoài ra, CNN còn sử dụng điều chuẩn L2 (weight decay), dừng sớm và đặc biệt là tăng cường dữ liệu.

3.8. Tăng cường dữ liệu

Tăng cường dữ liệu (data augmentation) tạo ra các biến thể hợp lý của ảnh huấn luyện để mô hình học được đặc trưng bền vững hơn với những thay đổi không làm đổi nhãn. Các phép biến đổi phổ biến gồm: lật ngang, xoay nhẹ, cắt ngẫu nhiên và thay đổi tỉ lệ, dịch chuyển, thay đổi độ sáng – độ tương phản – màu sắc, thêm nhiễu, làm mờ. Các kỹ thuật nâng cao hơn gồm Cutout (che ngẫu nhiên một vùng ảnh), Mixup (trộn tuyến tính hai ảnh và nhãn của chúng) và CutMix (dán một mảnh ảnh này lên ảnh khác).

Phép biến đổi cần phù hợp với ngữ nghĩa của bài toán. Lật ngang hợp lý với ảnh động vật nhưng không phù hợp với nhận dạng chữ viết (chữ "b" lật thành "d") hay ảnh y tế nơi vị trí trái – phải mang ý nghĩa giải phẫu; xoay 180° không phù hợp với nhận

dạng chữ số (6 thành 9). Tăng cường dữ liệu chỉ áp dụng cho tập huấn luyện, không áp dụng cho tập kiểm tra.

3.9. Học chuyển giao

Huấn luyện một CNN sâu từ đầu đòi hỏi hàng triệu ảnh có nhãn và nhiều ngày tính toán trên GPU. **Học chuyển giao** (transfer learning) giải quyết vấn đề này bằng cách tận dụng một mạng đã được *tiền huấn luyện* trên tập dữ liệu lớn (thường là ImageNet). Cơ sở của phương pháp là các đặc trưng ở lớp đầu (cạnh, kết cấu) mang tính phổ quát, hữu ích cho hầu hết bài toán thị giác. Có hai chiến lược chính:

- **Trích xuất đặc trưng:** "đóng băng" toàn bộ phần trích xuất đặc trưng, chỉ thay và huấn luyện lớp phân loại cuối cho bài toán mới. Phù hợp khi dữ liệu mới ít.
- **Tinh chỉnh** (fine-tuning): sau bước trên, "mở băng" một số lớp sâu và tiếp tục huấn luyện toàn mạng với tốc độ học nhỏ để điều chỉnh đặc trưng cho phù hợp với miền dữ liệu mới. Phù hợp khi dữ liệu mới nhiều hơn hoặc khác biệt đáng kể với dữ liệu nguồn.

Học chuyển giao giúp đạt độ chính xác cao chỉ với vài trăm đến vài nghìn ảnh, rút ngắn đáng kể thời gian huấn luyện, và hiện là cách tiếp cận mặc định trong hầu hết ứng dụng thực tế. Hiệu quả của nó phụ thuộc vào mức độ tương đồng giữa miền nguồn và miền đích: chuyển từ ảnh tự nhiên sang ảnh X-quang vẫn có ích, nhưng ít hơn so với chuyển sang một tập ảnh tự nhiên khác.

3.10. Các kiến trúc CNN tiêu biểu

LeNet-5 (1998). Kiến trúc của LeCun và cộng sự gồm 2 lớp tích chập 5 × 5 xen kẽ 2 lớp gộp, tiếp theo là 3 lớp kết nối đầy đủ, với khoảng 60 nghìn tham số., được dùng để đọc séc ngân hàng và định hình khuôn mẫu "tích chập – gộp – kết nối đầy đủ" cho mọi CNN về sau.

AlexNet (2012). Mạng của Krizhevsky và cộng sự [6] gồm 5 lớp tích chập và 3 lớp kết nối đầy đủ với khoảng 60 triệu tham số.; đóng góp chính là ReLU, dropout, tăng cường dữ liệu và huấn luyện trên GPU, giảm lỗi top-5 trên ImageNet từ khoảng 26% xuống 15,3%.

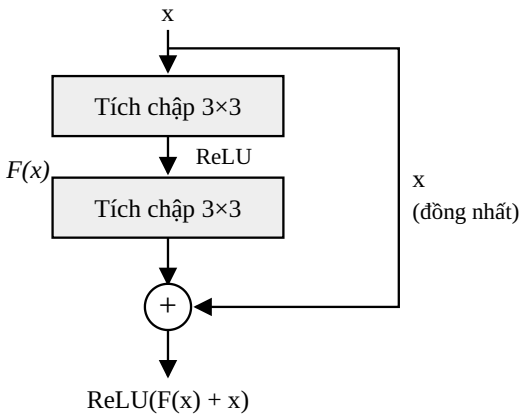
VGGNet (2014). Nhóm Visual Geometry Group (Đại học Oxford) [16] cho thấy độ sâu là yếu tố then chốt, chỉ dùng bộ lọc 3 × 3: hai lớp 3 × 3 có cùng trường tiếp nhận với một lớp 5 × 5 nhưng ít tham số hơn (18 so với 25) và thêm một lớp phi tuyến. VGG-16 có khoảng 138 triệu tham số.

GoogLeNet / Inception (2014). Mạng của Szegedy và cộng sự [18] có 22 lớp nhưng chỉ vài triệu tham số; khối **Inception** chạy song song tích chập 1 × 1, 3 × 3, 5 × 5 và gộp để nắm bắt đặc trưng ở nhiều tỉ lệ, dùng tích chập 1 × 1 để giảm số kênh.

ResNet (2015). Mạng sâu hơn lại có sai số huấn luyện *cao hơn* mạng nông (hiện tượng *suy giảm*) do khó tối ưu. He và cộng sự [19] đề xuất **khối phần dư** (residual block) với **kết nối tắt** (skip connection): thay vì học trực tiếp ánh xạ $H(\mathbf{x})$, khối học phần dư $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$, và đầu ra là

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$$

Nếu ánh xạ tối ưu gần với ánh xạ đồng nhất, mạng chỉ cần đẩy F về 0 – việc dễ hơn nhiều so với học ánh xạ đồng nhất từ đầu. Kết nối tắt còn tạo "đường cao tốc" để gradient truyền trực tiếp về các lớp đầu, giảm biến mất gradient. Nhờ đó ResNet huấn luyện được mạng 152 lớp với lỗi top-5 khoảng 3,57%; kết nối phần dư trở thành thành phần tiêu chuẩn của hầu hết mạng sâu, kể cả Transformer.



Hình 3.3. Khối phần dư trong ResNet với kết nối tắt

Các kiến trúc hiệu quả và hiện đại. *MobileNet* (2017) [23] dùng **tích chập tách theo chiều sâu** (tích chập riêng từng kênh + tích chập 1 × 1), giảm chi phí khoảng 8–9 lần cho thiết bị di động; *EfficientNet* (2019) [22] co giãn đồng thời độ sâu, độ rộng và độ phân giải. Từ 2020, *Vision Transformer* cạnh tranh với CNN khi có dữ liệu rất lớn, còn *ConvNeXt* (2022) cho thấy CNN thuần hiện đại hóa vẫn đạt hiệu năng tương đương.

Bảng 3.1. So sánh một số kiến trúc CNN tiêu biểu

Kiến trúc	Năm	Số lớp	Số tham số (xấp xỉ)	Đóng góp chính
LeNet-5	1998	7	60 nghìn	Khuôn mẫu tích chập – gộp – kết nối đầy đủ

AlexNet	2012	8	60 triệu	ReLU, dropout, huấn luyện trên GPU
VGG-16	2014	16	138 triệu	Mạng sâu đồng nhất với bộ lọc 3 × 3
GoogLeNet	2014	22	6,8 triệu	Khối Inception đa tỉ lệ, tích chập 1 × 1
ResNet-50 / 152	2015	50 / 152	25,6 / 60 triệu	Kết nối tắt, học phần dư
MobileNet	2017	28	4,2 triệu	Tích chập tách theo chiều sâu
EfficientNet-B0	2019	—	5,3 triệu	Co giãn kết hợp sâu – rộng – phân giải

3.11. Ứng dụng của CNN

Phân loại ảnh: gán một nhãn cho toàn bộ ảnh, ví dụ phân loại sản phẩm, nhận diện loài cây, phân loại ảnh da liễu.

Phát hiện đối tượng (object detection): vừa xác định loại vừa định vị đối tượng bằng hộp giới hạn. Các phương pháp *hai giai đoạn* như Faster R-CNN (2015) trước hết đề xuất vùng có khả năng chứa đối tượng rồi phân loại từng vùng, cho độ chính xác cao. Các phương pháp *một giai đoạn* như YOLO (2016) và SSD dự đoán trực tiếp hộp và nhãn trong một lần lan truyền, đạt tốc độ thời gian thực – phù hợp với camera giám sát, xe tự hành.

Phân đoạn ảnh (segmentation): gán nhãn cho từng điểm ảnh. Kiến trúc **U-Net** (2015) với cấu trúc mã hóa – giải mã đối xứng và các kết nối tắt giữa hai nhánh đặc biệt thành công trong phân đoạn ảnh y sinh (khối u, mạch máu, tế bào). Mask R-CNN (2017) kết hợp phát hiện và phân đoạn từng đối tượng riêng biệt.

Nhận dạng khuôn mặt: CNN ánh xạ ảnh khuôn mặt thành một vector đặc trưng sao cho khuôn mặt cùng người gần nhau và khác người xa nhau (như FaceNet với hàm mất mát bộ ba). Ứng dụng trong mở khóa điện thoại, chấm công, kiểm soát ra vào.

Y tế: hỗ trợ đọc ảnh X-quang, CT, MRI, phát hiện bệnh võng mạc tiểu đường, ung thư da, sàng lọc lao phổi. Nhiều hệ thống đã đạt độ chính xác tương đương bác sĩ chuyên khoa trên các tác vụ hẹp.

Công nghiệp và đời sống: kiểm tra lỗi sản phẩm trên dây chuyền, nhận dạng biển số xe, nhận dạng ký tự quang học (OCR) cho giấy tờ, nông nghiệp chính xác (phát hiện sâu bệnh qua ảnh lá), phân tích ảnh vệ tinh.

Dữ liệu ngoài ảnh: tích chập một chiều được dùng cho tín hiệu âm thanh, điện tâm đồ, dữ liệu cảm biến và phân loại văn bản; ảnh phổ (spectrogram) của âm thanh có

thể được xử lý như ảnh bằng CNN hai chiều. Ngoài ra, CNN còn là thành phần trong các mô hình sinh ảnh như GAN và mô hình khuếch tán (thông qua U-Net).

3.12. Hạn chế và thách thức

Mặc dù rất thành công, CNN vẫn có những hạn chế đáng kể:

- **Phụ thuộc vào dữ liệu gán nhãn và tài nguyên tính toán:** mạng sâu cần nhiều dữ liệu và GPU; triển khai trên thiết bị biên đòi hỏi nén mô hình (lượng tử hóa, cắt tỉa, chưng cất tri thức).
- **Khả năng khái quát khi phân phối thay đổi:** mô hình có thể giảm hiệu năng mạnh khi gặp điều kiện ánh sáng, góc nhìn, thiết bị chụp khác với dữ liệu huấn luyện. Nghiên cứu cho thấy CNN huấn luyện trên ImageNet thiên về *kết cấu* hơn *hình dạng*, khác với cách con người nhận thức.
- **Học "lối tắt"** (shortcut learning): mô hình có thể dựa vào tín hiệu giả tương quan, ví dụ nhận dạng bò dựa vào nền cỏ xanh, hay chẩn đoán dựa vào ký hiệu của bệnh viện in trên phim X-quang.
- **Ví dụ đối kháng:** Szegedy và cộng sự (2014) phát hiện rằng thêm một nhiễu rất nhỏ, mắt người không nhận ra, có thể khiến CNN phân loại sai với độ tự tin cao. Điều này đặt ra lo ngại an ninh cho các hệ thống như xe tự hành hay nhận dạng khuôn mặt.
- **Khả năng giải thích:** các kỹ thuật như bản đồ độ nổi bật (saliency map) và Grad-CAM giúp trực quan hóa vùng ảnh ảnh hưởng tới quyết định, nhưng chỉ nên xem là công cụ hỗ trợ phân tích, không phải lời giải thích đầy đủ và tuyệt đối đáng tin cậy.
- **Vấn đề đạo đức:** nhận dạng khuôn mặt có thể bị lạm dụng cho giám sát hàng loạt và đã được chứng minh có tỉ lệ lỗi khác nhau giữa các nhóm nhân khẩu học nếu dữ liệu huấn luyện không cân bằng.

3.13. Thực nghiệm: CNN trên ba bộ dữ liệu ảnh

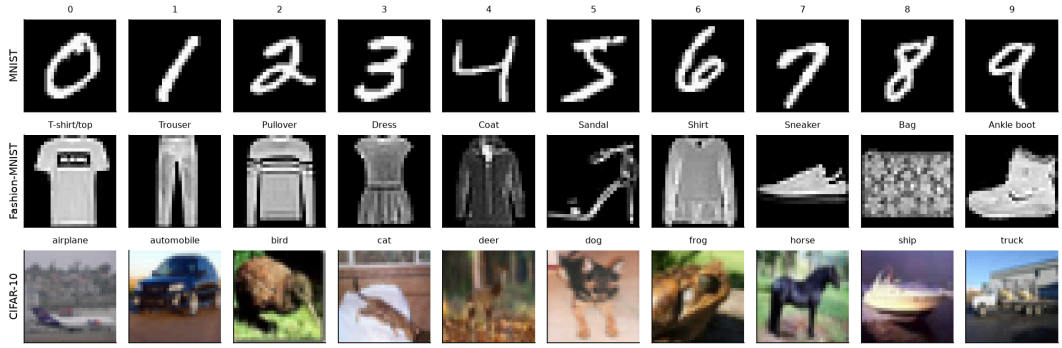
Phần này cài đặt một CNN nhỏ từ các thành phần đã trình bày theo ba cách – tự viết bằng NumPy (cả lan truyền xuôi lẫn ngược), Keras [38] và PyTorch [37] – huấn luyện trên ba bộ dữ liệu có độ khó tăng dần, rồi so sánh với mạng kết nối đầy đủ (MLP) để kiểm chứng lập luận ở mục 3.1.1. Mã nguồn nằm trong notebook `code/ch3_cnn.ipynb`; các đoạn mã cốt lõi được trích ở mục 3.13.2.

3.13.1. Dữ liệu và thiết lập

Bảng 3.2. Ba bộ dữ liệu ảnh dùng trong thực nghiệm chương 3

Bộ dữ liệu	Huấn luyện / kiểm tra	Kích thước ảnh	Nội dung 10 lớp	Trung bình / độ lệch chuẩn điểm ảnh
MNIST [5]	60.000 / 10.000	28×28, xám	Chữ số viết tay 0–9	0,131 / 0,308
Fashion-MNIST [41]	60.000 / 10.000	28×28, xám	Áo phông, quần, áo len, váy, áo khoác, xăng-đan, sơ mi, giày thể thao, túi, bốt	0,286 / 0,353
CIFAR-10 [42]	50.000 / 10.000	32×32, màu RGB	Máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu thủy, xe tải	0,473 / 0,252

Cả ba bộ gần như cân bằng (MNIST: 5.421 – 6.742 ảnh mỗi lớp; hai bộ còn lại đều tuyệt đối) nên độ chính xác là thước đo phù hợp, kèm F1 macro. Điểm ảnh được chia cho 255 về đoạn [0, 1]. Qua ảnh mẫu (Hình 3.4), MNIST dễ nhất (nền đen, chữ số ở giữa); Fashion-MNIST có nhiều lớp giống nhau về hình dáng (các loại áo); CIFAR-10 khó nhất: ảnh màu thực tế, nền phức tạp, vật thể nhiều tư thế, độ phân giải chỉ 32×32.



Hình 3.4. Một ảnh mẫu của mỗi lớp trong MNIST, Fashion-MNIST và CIFAR-10

Kiến trúc theo khuôn mẫu ở mục 3.6:

Conv 3×3, 16 bộ lọc → ReLU → MaxPool 2×2 → Conv 3×3, 32 bộ lọc → ReLU → MaxPool 2×2 → Flatten → Dense 10

Tích chập dùng đệm "same" nên chỉ lớp gộp giảm kích thước (28 → 14 → 7; với CIFAR-10: 32 → 16 → 8), cho 20.490 tham số với ảnh xám và 25.578 với ảnh màu. Bản NumPy cài đặt tích chập bằng kỹ thuật *im2col* (quy tích chập về một phép nhân ma trận) và tự viết Adam [32]. Cả ba bản dùng khởi tạo He, Adam với tốc độ học 10^{-3}

, lô 64, 5 epoch; NumPy và Keras chạy trên CPU (Intel Core i5-11400H), PyTorch chạy trên GPU (RTX 3050 Laptop).

3.13.2. Cài đặt

Lớp tích chập của bản NumPy (Đoạn mã 3.1) dùng kỹ thuật **im2col**: mỗi vùng $3 \times 3 \times C$ của ảnh đã đệm được trải thành một hàng của ma trận `cols`, nhờ đó tích chập với F bộ lọc chỉ còn là một phép nhân ma trận. Khi lan truyền ngược, gradient theo bộ lọc là $D^{\top} \cdot \text{cols}$, còn gradient theo đầu vào được cộng dồn về đúng vị trí điểm ảnh (col2im). Lớp gộp cực đại (Đoạn mã 3.2) ghi nhớ vị trí phần tử lớn nhất bằng một mặt nạ và chỉ chuyển gradient về đó.

Đoạn mã 3.1. Lớp tích chập bằng NumPy (im2col)

```
from numpy.lib.stride_tricks import sliding_window_view

class Conv2D:
    """Tích chập k×k, bước trượt 1, đệm 'same', khởi tạo He đều."""

    def __init__(self, c_in, c_out, k, rng):
        lim = np.sqrt(6 / (c_in * k * k))
        self.W = rng.uniform(-lim, lim, (c_out, c_in, k, k)).astype(np.float32)
        self.b = np.zeros(c_out, np.float32)
        self.k, self.p = k, k // 2

    def forward(self, x):
        N, C, H, W = x.shape
        xp = np.pad(x, ((0, 0), (0, 0), (self.p, self.p), (self.p, self.p)))
        win = sliding_window_view(
            xp, (self.k, self.k), axis=(2, 3)
        )  # (N, C, H, W, k, k)
        self.cols = win.transpose(0, 2, 3, 1, 4, 5).reshape(
            N * H * W, -1
        )  # im2col: (NHW, C·k·k)
        self.shape = x.shape
        out = self.cols @ self.W.reshape(len(self.W), -1).T + self.b  # (NHW, F)
        return out.reshape(N, H, W, -1).transpose(0, 3, 1, 2)

    def backward(self, dout):
        N, C, H, W = self.shape
        F, k, p = len(self.W), self.k, self.p
        d = dout.transpose(0, 2, 3, 1).reshape(-1, F)
        self.dW = (d.T @ self.cols).reshape(self.W.shape)
        self.db = d.sum(0)
        dcols = (d @ self.W.reshape(F, -1)).reshape(N, H, W, C, k, k)
        dxp = np.zeros((N, C, H + 2 * p, W + 2 * p), np.float32)
        for i in range(k):  # col2im: cộng dồn gradient về đúng vị trí điểm ảnh
            for j in range(k):
                dxp[:, :, i : i + H, j : j + W] += dcols[..., i, j].transpose(
                    0, 3, 1, 2
                )
        return dxp[:, :, p : p + H, p : p + W]

    def params(self):
        return [(self.W, "dW"), (self.b, "db")]
```

Đoạn mã 3.2. Lớp gộp cực đại 2×2 bằng NumPy

```
class MaxPool2:
    """Gộp cực đại 2×2, bước trượt 2."""

    def forward(self, x):
```

```
        N, C, H, W = x.shape
        xr = x.reshape(N, C, H // 2, 2, W // 2, 2)
        out = xr.max(axis=(3, 5))
        self.mask = xr == out[:, :, :, None, :, None]  # ghi nhớ vị trí phần tử lớn nhất
        self.shape = x.shape
        return out

    def backward(self, d):
        return (self.mask * d[:, :, :, None, :, None]).reshape(self.shape)

    def params(self):
        return []
```

3.13.3. Kết quả

Bảng 3.3. CNN: so sánh ba cách cài đặt trên tập kiểm tra

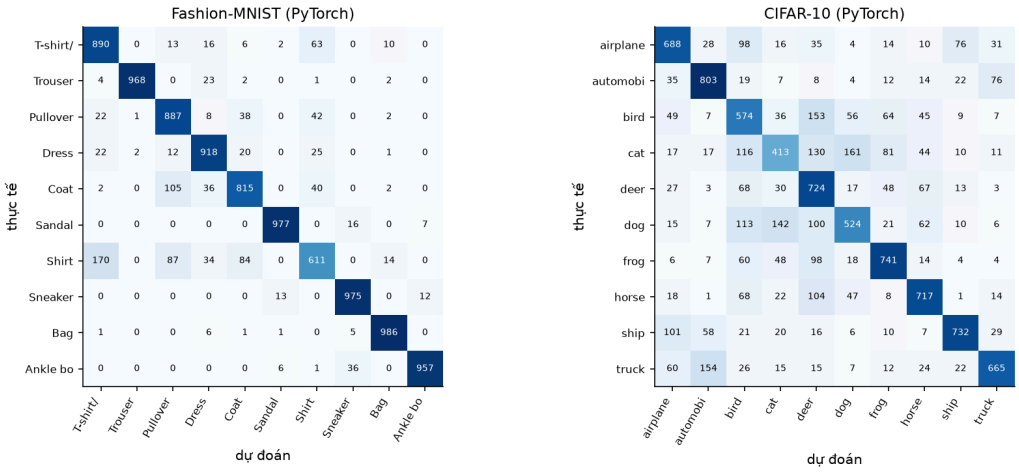
Bộ dữ liệu	Cài đặt	Số tham số	Độ chính xác (%)	F1 macro (%)	Loss cuối	Thời gian (s)
MNIST	NumPy (CPU)	20.490	98,71	98,70	0,033	211,1
	Keras (CPU)	20.490	98,27	98,26	0,031	14,0
	PyTorch (GPU)	20.490	98,70	98,69	0,031	8,0
Fashion-MNIST	NumPy (CPU)	20.490	89,44	89,15	0,242	223,4
	Keras (CPU)	20.490	90,01	89,84	0,239	16,3
	PyTorch (GPU)	20.490	89,84	89,66	0,235	6,7
CIFAR-10	NumPy (CPU)	25.578	65,39	65,48	0,955	339,2
	Keras (CPU)	25.578	65,04	64,46	0,954	24,5
	PyTorch (GPU)	25.578	65,81	65,76	0,928	18,8

Ba cách cài đặt có cùng số tham số, loss cuối gần như bằng nhau và độ chính xác chênh nhau dưới 1 điểm phần trăm – xác nhận lan truyền ngược qua tích chập và gộp cực đại tự cài đặt là đúng (kiểm tra gradient số học trong notebook cho sai số tương đối khoảng 10^{-6}). Không thể trùng khớp tuyệt đối vì ba thư viện sinh trọng số ban đầu và thứ tự lô khác nhau; ngay cả PyTorch chạy lại trên GPU cũng lệch nhẹ do cuDNN không cố định thứ tự cộng dấu phẩy động.

Về thời gian, bản NumPy chậm hơn Keras khoảng 14–15 lần dù cùng chạy trên CPU, vì phải sao chép dữ liệu khi trải im2col và nhiều bước chỉ chạy trên một lõi, trong khi Keras biên dịch đồ thị tính toán và dùng thư viện oneDNN tối ưu cho tích chập; PyTorch trên GPU còn nhanh hơn nữa.

Độ chính xác giảm theo độ khó: gần 99% (MNIST), khoảng 90% (Fashion-MNIST), 65–66% (CIFAR-10). Ma trận nhầm lẫn (Hình 3.5) cho biết mô hình sai ở đâu:

- **Fashion-MNIST:** lớp *áo sơ mi* khó nhất (chỉ 611/1.000 ảnh đúng), bị nhầm sang áo phông (170 ảnh), áo len (87) và áo khoác (84) – các loại áo có hình dáng tổng thể giống nhau ở 28×28; các lớp có hình dạng đặc trưng như quần, túi, giày đạt trên 96%.
- **CIFAR-10:** các lớp phương tiện (ô tô 80,3%, tàu thủy 73,2%) dễ hơn các lớp động vật. Lớp *mèo* tệ nhất (41,3%), bị nhầm nhiều nhất sang chó (161 ảnh); chó cũng bị nhầm sang mèo 142 lần. Các cặp khác cũng có lý: xe tải → ô tô (154), tàu thủy → máy bay (101, cùng nền xanh), chim → hươu (153, cùng nền cây cỏ) – mạng nông mới học được màu sắc, kết cấu và hình dạng thô, chưa đủ sâu cho các đặc trưng tinh tế (mục 3.6).



Hình 3.5. Ma trận nhầm lẫn của bản PyTorch trên Fashion-MNIST và CIFAR-10 (hàng: nhãn thật, cột: dự đoán)

3.13.4. So sánh CNN với mạng kết nối đầy đủ

CNN được so sánh với một MLP hai lớp ẩn 256 nơ-ron, huấn luyện bằng PyTorch với cùng thiết lập (Bảng 3.4).

Bảng 3.4. CNN so với MLP (PyTorch, 5 epoch)

Bộ dữ liệu	Mô hình	Số tham số	Độ chính xác (%)	F1 macro (%)

MNIST	MLP	269.322	97,79	97,77
	CNN	20.490	**98,67**	**98,66**
Fashion-MNIST	MLP	269.322	87,84	87,75
	CNN	20.490	**89,97**	**89,79**
CIFAR-10	MLP	855.050	47,57	47,50
	CNN	25.578	**66,08**	**66,03**

CNN tốt hơn trên cả ba bộ dữ liệu dù có **ít tham số hơn 13 lần** (ảnh xám) đến **33 lần** (CIFAR-10). Khoảng cách tăng theo độ khó của dữ liệu: 0,9 điểm trên MNIST, 2,1 điểm trên Fashion-MNIST và 18,5 điểm trên CIFAR-10. Với ảnh màu 32×32, MLP cần 786.432 trọng số chỉ riêng ở lớp đầu và coi mỗi điểm ảnh là một đặc trưng độc lập; CNN nhờ **kết nối cục bộ** và **chia sẻ trọng số** (mục 3.2.3) dùng lại cùng một bộ lọc ở mọi vị trí, nên vừa ít tham số vừa tổng quát hóa tốt hơn – đúng giả định quy nạp đã nêu ở mục 2.17.

3.14. Tiểu kết chương 3

Chương 3 đã trình bày mạng nơ-ron tích chập – kiến trúc khai thác tính cục bộ và tính đồng biến tịnh tiến của dữ liệu không gian thông qua ba ý tưởng: kết nối thưa, chia sẻ tham số và học bộ lọc tự động. Các thành phần cơ bản gồm lớp tích chập (với bước trượt, đệm), hàm kích hoạt ReLU, lớp gộp và lớp kết nối đầy đủ, kết hợp với các kỹ thuật huấn luyện như chuẩn hóa theo lô, dropout, tăng cường dữ liệu và học chuyển giao. Sự tiến hóa từ LeNet, AlexNet, VGG, Inception tới ResNet và EfficientNet cho thấy các ý tưởng như mạng sâu hơn, bộ lọc nhỏ, kết nối tắt và thiết kế hiệu quả đã liên tục nâng cao năng lực của mô hình. CNN đã trở thành nền tảng của thị giác máy tính hiện đại, dù vẫn còn những thách thức về độ bền vững, khả năng giải thích và đạo đức. Thực nghiệm với một CNN hai lớp tích chập cài đặt theo ba cách đạt khoảng 98,7% trên MNIST, 90% trên Fashion-MNIST và 66% trên CIFAR-10, và vượt mạng kết nối đầy đủ trên cả ba bộ dữ liệu dù có ít tham số hơn 13–33 lần – minh chứng trực tiếp cho giá trị của kết nối cục bộ và chia sẻ trọng số.

Tuy nhiên, CNN được thiết kế cho dữ liệu có cấu trúc lưới cố định và không có "bộ nhớ" giữa các đầu vào. Với dữ liệu tuần tự có độ dài thay đổi như văn bản, tiếng nói hay chuỗi thời gian – nơi ý nghĩa của mỗi phần tử phụ thuộc vào ngữ cảnh trước đó – cần một kiến trúc khác: mạng nơ-ron hồi quy, được trình bày ở Chương 4.

CHƯƠNG 4. MẠNG NƠ-RON HỒI QUY (RNN)

4.1. Dữ liệu tuần tự và yêu cầu đối với mô hình

Dữ liệu tuần tự là dữ liệu mà thứ tự của các phần tử mang ý nghĩa. Ví dụ điển hình gồm: văn bản (chuỗi các từ hoặc ký tự), tiếng nói (chuỗi các khung âm thanh), chuỗi thời gian tài chính (giá cổ phiếu theo ngày), dữ liệu cảm biến IoT, nhật ký hoạt động của người dùng, chuỗi DNA, hay video (chuỗi các khung hình). Trong những dữ liệu này, giá trị hoặc ý nghĩa tại thời điểm hiện tại phụ thuộc vào những gì đã xuất hiện trước đó. Chẳng hạn, trong câu "Tôi lớn lên ở Pháp nên tôi nói thành thạo tiếng …", để đoán từ cuối là "Pháp", mô hình phải nhớ thông tin xuất hiện từ nhiều từ trước.

Một mô hình xử lý tốt dữ liệu tuần tự cần đáp ứng các yêu cầu sau:

1. Xử lý được chuỗi có **độ dài thay đổi** – câu có thể dài 5 từ hoặc 50 từ.
2. **Ghi nhớ ngữ cảnh**, kể cả các phụ thuộc ở khoảng cách xa.
3. **Chia sẻ tham số** giữa các vị trí, để quy luật học được ở vị trí này có thể áp dụng ở vị trí khác (tương tự ý tưởng chia sẻ bộ lọc của CNN, nhưng theo trục thời gian).

Mạng kết nối đầy đủ nhận đầu vào kích thước cố định và xử lý mỗi mẫu độc lập, không có bộ nhớ. Có thể ghép một "cửa sổ" vài phần tử liên tiếp làm đầu vào, nhưng cách này chỉ nắm bắt ngữ cảnh ngắn, cố định và không chia sẻ tham số giữa các vị trí. **Mạng nơ-ron hồi quy** (RNN) ra đời để đáp ứng cả ba yêu cầu trên bằng cách đưa vào một **trạng thái ẩn** đóng vai trò bộ nhớ, được cập nhật liên tục qua từng bước thời gian.

Về lịch sử, ý tưởng mạng có kết nối hồi quy xuất hiện từ mạng Hopfield (1982) và mạng Jordan (1986). Mạng **Elman** (1990) [24] – còn gọi là "RNN đơn giản" – có dạng gần với RNN hiện đại nhất, trong đó trạng thái ẩn ở bước trước được đưa trở lại làm đầu vào cho bước sau.

4.2. Kiến trúc và nguyên lý hoạt động của RNN

4.2.1. Phương trình cập nhật

Tại mỗi bước thời gian $t = 1, 2, \ldots, T$, RNN nhận đầu vào $\mathbf{x}_t$ và trạng thái ẩn ở bước trước $\mathbf{h}_{t-1}$, rồi tính trạng thái ẩn mới $\mathbf{h}_t$ và đầu ra $\mathbf{y}_t$:

$$\mathbf{h}_t = \tanh\left(W_{xh}\,\mathbf{x}_t + W_{hh}\,\mathbf{h}_{t-1} + \mathbf{b}_h\right)$$

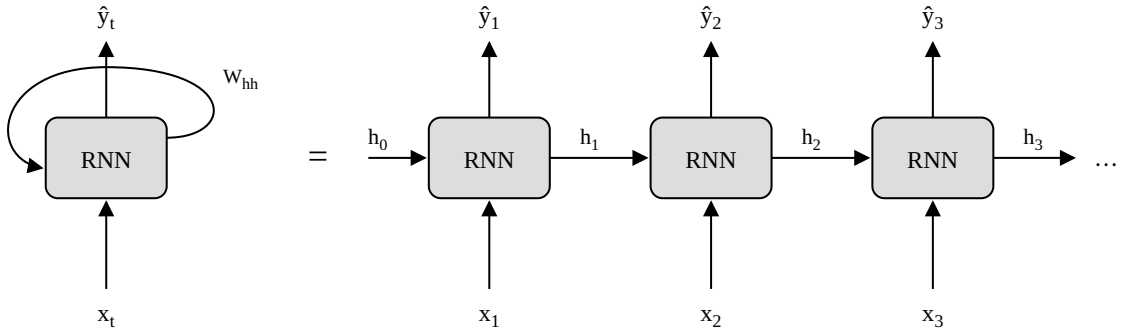
$$\hat{\mathbf{y}}_t = \mathrm{softmax}\big(W_{hy}\,\mathbf{h}_t + \mathbf{b}_y\big)$$

trong đó W_{xh}, W_{hh}, W_{hy} lần lượt là ma trận trọng số đầu vào, hồi quy và đầu ra, $\mathbf{b}_h$, $\mathbf{b}_y$ là độ lệch, $\mathbf{h}_0 = \mathbf{0}$; hàm tanh giữ trạng thái trong khoảng $(-1; 1)$.

Điểm mấu chốt là **các ma trận trọng số được dùng chung cho mọi bước thời gian**, nên RNN xử lý chuỗi dài tùy ý với số tham số cố định và áp dụng một quy luật học được ở mọi vị trí; $\mathbf{h}_t$ là bản tóm tắt (có mất mát) của $\mathbf{x}_1, \ldots, \mathbf{x}_t$.

4.2.2. Mở rộng theo thời gian

Để dễ phân tích, RNN thường được biểu diễn dưới dạng **mở rộng theo thời gian** (unrolled/unfolded): vòng lặp hồi quy được "trải phẳng" thành một chuỗi các bản sao của cùng một ô RNN, mỗi bản sao ứng với một bước thời gian và truyền trạng thái ẩn cho bản sao kế tiếp (Hình 4.1). Khi mở rộng, RNN trở thành một mạng truyền thẳng rất sâu – có độ sâu bằng độ dài chuỗi – với điều kiện ràng buộc là mọi lớp dùng chung trọng số.



Hình 4.1. RNN dạng gọn (trái) và dạng mở rộng theo thời gian (phải); mọi ô dùng chung trọng số

4.2.3. Số lượng tham số

Với đầu vào có d_x chiều và trạng thái ẩn có d_h chiều, phần hồi quy của RNN đơn giản có $d_h(d_x + d_h + 1)$ tham số (W_{xh}: $d_h \times d_x$; W_{hh}: $d_h \times d_h$; $\mathbf{b}_h$: d_h). Ví dụ với $d_x = 100$ (vector nhúng từ 100 chiều) và $d_h = 128$, số tham số là $128 \times (100 + 128 + 1) = 29.312$, không phụ thuộc vào độ dài câu.

4.2.4. Biểu diễn đầu vào văn bản

Khi xử lý ngôn ngữ, mỗi từ (hoặc token) trước hết được ánh xạ thành một vector. Cách đơn giản là mã hóa one-hot với số chiều bằng kích thước từ điển, nhưng vector này rất thưa và không thể hiện quan hệ ngữ nghĩa. Cách phổ biến hơn là dùng **vector nhúng từ** (word embedding) dày, có vài trăm chiều, học được từ dữ liệu (như word2vec, GloVe hoặc lớp embedding huấn luyện cùng mô hình), sao cho các từ có

nghĩa gần nhau có vector gần nhau. Chuỗi vector nhúng này chính là chuỗi $\mathbf{x}_1, \ldots, \mathbf{x}_T$ đưa vào RNN.

4.3. Các dạng bài toán với RNN

Tùy theo quan hệ giữa chuỗi đầu vào và đầu ra, RNN có thể được tổ chức theo nhiều cấu hình:

Bảng 4.1. Các dạng cấu hình bài toán của RNN

Cấu hình	Mô tả	Ví dụ ứng dụng
Một – một	Một đầu vào, một đầu ra (thực chất là mạng truyền thẳng)	Phân loại ảnh
Một – nhiều	Một đầu vào sinh ra chuỗi đầu ra	Sinh chú thích cho ảnh, sinh nhạc từ một chủ đề
Nhiều – một	Chuỗi đầu vào, một đầu ra tại bước cuối	Phân tích cảm xúc bình luận, phân loại văn bản, dự báo giá ngày mai
Nhiều – nhiều (đồng bộ)	Mỗi bước đầu vào có một đầu ra tương ứng	Gán nhãn từ loại, nhận dạng thực thể có tên, phân loại từng khung video
Nhiều – nhiều (không đồng bộ)	Đọc hết chuỗi vào rồi sinh chuỗi ra có độ dài khác	Dịch máy, tóm tắt văn bản, nhận dạng tiếng nói

Cấu hình cuối cùng đòi hỏi kiến trúc **mã hóa – giải mã** (encoder–decoder), sẽ được trình bày ở mục 4.8.

4.4. Huấn luyện RNN: lan truyền ngược theo thời gian

4.4.1. Thuật toán BPTT

Hàm mất mát tổng của một chuỗi là tổng mất mát tại các bước: $L = \sum_{t=1}^{T} L_t(\hat{\mathbf{y}}_t, \mathbf{y}_t)$. RNN được huấn luyện bằng **lan truyền ngược theo thời gian** (Backpropagation Through Time – BPTT) [25]: sau khi mở rộng mạng theo thời gian, áp dụng thuật toán lan truyền ngược thông thường trên mạng truyền thẳng thu được. Vì trọng số được chia sẻ, gradient theo mỗi ma trận trọng số bằng tổng các gradient đóng góp từ mọi bước thời gian. Chẳng hạn:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial L_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^{+} \mathbf{h}_k}{\partial W_{hh}}$$

trong đó ∂^{+} ký hiệu đạo hàm "trực tiếp" tại bước k. Thành phần quan trọng nhất là $\partial \mathbf{h}_t / \partial \mathbf{h}_k$ – mức độ trạng thái ở bước k ảnh hưởng tới trạng thái ở bước t sau đó.

4.4.2. Vấn đề biến mất và bùng nổ gradient

Theo quy tắc đạo hàm hàm hợp, $\partial \mathbf{h}_t / \partial \mathbf{h}_k$ là tích của $t-k$ ma trận Jacobi:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \prod_{j=k+1}^{t} \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) W_{hh}$$

Tích của nhiều ma trận này hành xử gần giống lũy thừa W_{hh}^{t-k} nhân với các hệ số đạo hàm tanh (luôn ≤ 1). Bengio, Simard và Frasconi (1994) [17] đã phân tích rằng:

- Nếu giá trị riêng lớn nhất (theo độ lớn) của W_{hh} nhỏ hơn 1, tích này **giảm theo hàm mũ** về 0 khi khoảng cách $t-k$ tăng – hiện tượng **biến mất gradient** (vanishing gradient). Khi đó, tín hiệu lỗi từ bước t gần như không truyền được về các bước xa trong quá khứ, nên mạng không học được các phụ thuộc dài hạn. Ví dụ, nếu mỗi bước nhân gradient với hệ số 0,9 thì sau 50 bước chỉ còn $0{,}9^{50} \approx 0{,}005$.
- Ngược lại, nếu giá trị riêng lớn hơn 1, gradient có thể **tăng theo hàm mũ** – hiện tượng **bùng nổ gradient** (exploding gradient), khiến tham số cập nhật quá mạnh, hàm mất mát dao động hoặc xuất hiện giá trị NaN. Với hệ số 1,1, sau 50 bước gradient tăng khoảng $1{,}1^{50} \approx 117$ lần.

Trong thực tế, RNN đơn giản thường chỉ học hiệu quả các phụ thuộc trong phạm vi khoảng 10 bước.

4.4.3. Các giải pháp

Cắt gradient (gradient clipping) [26] là giải pháp đơn giản và hiệu quả cho bùng nổ gradient: nếu chuẩn của vector gradient vượt ngưỡng τ, nó được co lại về đúng ngưỡng:

$$\mathbf{g} \leftarrow \frac{\tau}{\|\mathbf{g}\|}\,\mathbf{g} \quad \text{nếu} \quad \|\mathbf{g}\| > \tau$$

Cắt gradient giữ nguyên hướng cập nhật nhưng giới hạn độ lớn bước nhảy. Tuy nhiên, nó không giải quyết được biến mất gradient. Đối với vấn đề này, các biện pháp như khởi tạo W_{hh} trực giao hoặc bằng ma trận đơn vị, dùng ReLU có hỗ trợ một phần, nhưng giải pháp căn bản và thành công nhất là thay đổi kiến trúc của ô hồi quy bằng các **cơ chế cổng** – tiêu biểu là LSTM và GRU.

BPTT cắt cụt (truncated BPTT): với chuỗi rất dài (ví dụ cả một cuốn sách), việc lan truyền ngược qua toàn bộ chuỗi tốn bộ nhớ không khả thi. Chuỗi được chia thành các đoạn ngắn (ví dụ 100 bước); trạng thái ẩn vẫn được truyền tiếp giữa các đoạn khi

lan truyền tiến, nhưng gradient chỉ lan truyền ngược trong phạm vi mỗi đoạn. Đổi lại, mạng khó học được phụ thuộc dài hơn độ dài đoạn.

4.5. Bộ nhớ dài – ngắn hạn (LSTM)

4.5.1. Ý tưởng

LSTM (Long Short-Term Memory) được Hochreiter và Schmidhuber đề xuất năm 1997 [7] nhằm giải quyết trực tiếp vấn đề biến mất gradient. Ý tưởng cốt lõi là bổ sung một **trạng thái ô nhớ** (cell state) $\mathbf{c}_t$ – có thể hình dung như một "băng chuyền" chạy suốt chuỗi – mà thông tin trên đó chỉ bị thay đổi qua các phép toán tuyến tính đơn giản (nhân từng phần tử và cộng). Việc đọc, ghi và xóa thông tin trên ô nhớ được điều khiển bởi ba **cổng** (gate). Mỗi cổng là một lớp nơ-ron với hàm sigmoid cho đầu ra trong khoảng (0; 1), đóng vai trò như van điều tiết: 0 nghĩa là "chặn hoàn toàn", 1 nghĩa là "cho qua hoàn toàn". Cổng quên được bổ sung sau đó bởi Gers và cộng sự (2000) [27] và trở thành một phần của LSTM tiêu chuẩn.

4.5.2. Các phương trình

Ký hiệu $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ là phép ghép hai vector, σ là hàm sigmoid, $\odot$ là phép nhân từng phần tử. Tại mỗi bước t:

Cổng quên (forget gate) quyết định giữ lại bao nhiêu thông tin cũ trong ô nhớ:

$$\mathbf{f}_t = \sigma\big(W_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\big)$$

Cổng vào (input gate) quyết định ghi bao nhiêu thông tin mới, và **giá trị ứng viên** là nội dung mới có thể được ghi:

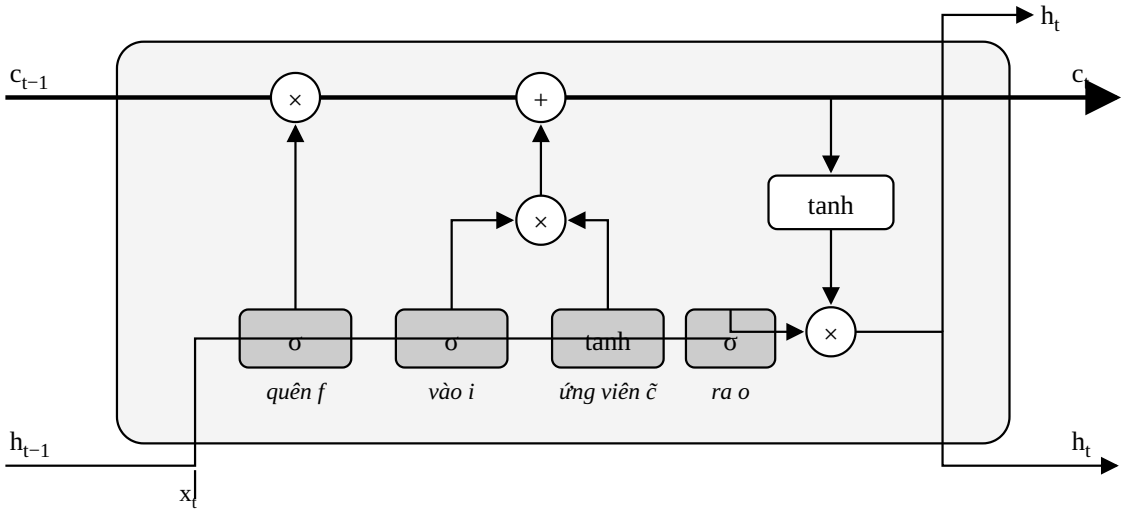
$$\mathbf{i}_t = \sigma\big(W_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\big), \qquad \tilde{\mathbf{c}}_t = \tanh\big(W_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c\big)$$

Cập nhật ô nhớ: xóa bớt thông tin cũ theo cổng quên và cộng thêm thông tin mới theo cổng vào:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

Cổng ra (output gate) quyết định phần nào của ô nhớ được đưa ra làm trạng thái ẩn:

$$\mathbf{o}_t = \sigma\big(W_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\big), \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$



Hình 4.2. Cấu trúc một ô LSTM với ba cổng quên, vào, ra và trạng thái ô nhớ c_t

4.5.3. Vì sao LSTM khắc phục được biến mất gradient

Xét đường đi của gradient qua ô nhớ: từ phương trình cập nhật, $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathrm{diag}(\mathbf{f}_t)$ (bỏ qua các đường gián tiếp qua các cổng). Khác với RNN đơn giản, ở đây **không có phép nhân lặp lại với ma trận trọng số W_{hh} và không có đạo hàm tanh làm co gradient**. Nếu cổng quên mở ($\mathbf{f}_t \approx 1$), gradient được truyền qua nhiều bước gần như nguyên vẹn – Hochreiter gọi đây là "băng chuyền lỗi không đổi" (constant error carousel). Quan trọng hơn, các cổng được *học* từ dữ liệu, nên mạng tự quyết định khi nào cần nhớ lâu (giữ $\mathbf{f}_t$ gần 1) và khi nào cần quên (đưa $\mathbf{f}_t$ về gần 0, ví dụ khi bắt đầu một câu mới). Trong thực tế, độ lệch của cổng quên thường được khởi tạo bằng giá trị dương (như 1) để mạng mặc định "nhớ" ở giai đoạn đầu huấn luyện. Ý tưởng tạo đường truyền gradient gần như đồng nhất này tương tự kết nối tắt trong ResNet ở Chương 3.

Ví dụ minh họa. Khi mô hình đọc câu "*Những học sinh* trong lớp mà cô giáo chủ nhiệm rất yêu quý *đều* …", ô nhớ có thể lưu thông tin "chủ ngữ số nhiều" khi gặp "những học sinh", cổng quên giữ thông tin này qua mệnh đề phụ chen giữa, và cổng ra sử dụng nó khi cần dự đoán từ thích hợp sau đó.

4.5.4. Số lượng tham số

LSTM có bốn khối biến đổi (ba cổng và giá trị ứng viên), mỗi khối có cùng kích thước với lớp hồi quy của RNN đơn giản, nên số tham số gấp 4 lần: $4d_h(d_x + d_h + 1)$. Với $d_x = 100$, $d_h = 128$, LSTM có $4 \times 29.312 = 117.248$ tham số.

4.6. Đơn vị hồi quy có cổng (GRU)

GRU (Gated Recurrent Unit) được Cho và cộng sự đề xuất năm 2014 [8] như một phiên bản đơn giản hóa của LSTM. GRU gộp trạng thái ô nhớ và trạng thái ẩn làm một, và chỉ dùng hai cổng:

$$\mathbf{z}_t = \sigma\big(W_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z\big) \qquad \text{(cổng cập nhật)}$$

$$\mathbf{r}_t = \sigma\big(W_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r\big) \qquad \text{(cổng đặt lại)}$$

$$\tilde{\mathbf{h}}_t = \tanh\big(W_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h\big)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

Cổng đặt lại $\mathbf{r}_t$ quyết định mức độ sử dụng trạng thái cũ khi tính trạng thái ứng viên: nếu $\mathbf{r}_t \approx 0$, mạng "quên" quá khứ và tính ứng viên chủ yếu từ đầu vào hiện tại. **Cổng cập nhật** $\mathbf{z}_t$ đóng vai trò kết hợp cả cổng quên và cổng vào của LSTM: trạng thái mới là trung bình có trọng số giữa trạng thái cũ và trạng thái ứng viên. Khi $\mathbf{z}_t \approx 0$, trạng thái được sao chép nguyên vẹn qua bước thời gian, tạo đường truyền gradient thông suốt tương tự LSTM. (Một số thư viện dùng quy ước ngược lại cho $\mathbf{z}_t$; hai cách tương đương.)

GRU có ba khối biến đổi nên số tham số khoảng $3d_h(d_x + d_h + 1)$, tức 87.936 với ví dụ trên – ít hơn LSTM khoảng 25%. Thực nghiệm (Chung và cộng sự [31]) cho thấy GRU và LSTM tương đương trên nhiều bài toán: GRU nhanh hơn và thường tốt hơn khi ít dữ liệu, LSTM đôi khi tốt hơn khi cần ghi nhớ rất dài.

Bảng 4.2. So sánh RNN đơn giản, LSTM và GRU

Tiêu chí	RNN đơn giản	LSTM	GRU
Năm đề xuất	1990 (Elman)	1997	2014
Trạng thái	$\mathbf{h}_t$	$\mathbf{h}_t$ và ô nhớ $\mathbf{c}_t$	$\mathbf{h}_t$
Số cổng	0	3 (quên, vào, ra)	2 (cập nhật, đặt lại)
Số tham số (tương đối)	1×	4×	3×
Phụ thuộc dài hạn	Kém (biến mất gradient)	Tốt	Tốt
Tốc độ huấn luyện	Nhanh nhất	Chậm nhất	Trung bình

4.7. Các mở rộng kiến trúc: RNN hai chiều và RNN nhiều tầng

4.7.1. RNN hai chiều

Trong nhiều bài toán, ý nghĩa của một phần tử phụ thuộc cả vào ngữ cảnh phía *sau*. Ví dụ, trong câu "Tôi đi *bàn* bạc với đối tác" và "Tôi ngồi vào *bàn*", để xác định

từ loại và nghĩa của "bàn" cần nhìn cả những từ phía sau. **RNN hai chiều** (Bidirectional RNN – Schuster và Paliwal, 1997) [28] dùng hai RNN độc lập: một RNN đọc chuỗi theo chiều thuận (từ trái sang phải) tạo ra $\overrightarrow{\mathbf{h}}_t$, một RNN đọc theo chiều ngược tạo ra $\overleftarrow{\mathbf{h}}_t$. Biểu diễn tại vị trí t là ghép của cả hai:

$$\mathbf{h}_t = \left[\overrightarrow{\mathbf{h}}_t \,;\, \overleftarrow{\mathbf{h}}_t\right]$$

nhờ đó chứa thông tin của toàn bộ chuỗi. BiLSTM từng là kiến trúc tiêu chuẩn cho gán nhãn từ loại, nhận dạng thực thể có tên và là thành phần của mô hình ELMo (2018). Tuy nhiên, RNN hai chiều yêu cầu có toàn bộ chuỗi trước khi xử lý, nên **không phù hợp với các ứng dụng trực tuyến** như nhận dạng tiếng nói thời gian thực hay dự báo chuỗi thời gian, nơi tương lai chưa được biết.

4.7.2. RNN nhiều tầng

Tương tự việc xếp chồng nhiều lớp trong CNN, có thể xếp chồng nhiều tầng RNN: chuỗi trạng thái ẩn của tầng dưới là chuỗi đầu vào của tầng trên. Các tầng thấp có xu hướng nắm bắt đặc trưng cục bộ (hình thái từ, cú pháp gần), các tầng cao nắm bắt ngữ nghĩa trừu tượng hơn. Trong thực tế thường dùng 2–4 tầng; nhiều tầng hơn khó huấn luyện và thường cần kết nối tắt giữa các tầng. Ví dụ, hệ thống dịch máy GNMT của Google (2016) dùng 8 tầng LSTM cho bộ mã hóa và 8 tầng cho bộ giải mã, với kết nối phần dư.

4.8. Mô hình mã hóa – giải mã và cơ chế chú ý

4.8.1. Kiến trúc mã hóa – giải mã (Seq2Seq)

Cho bài toán chuyển chuỗi thành chuỗi như dịch máy, Sutskever và cộng sự [29] cùng Cho và cộng sự (2014) đề xuất kiến trúc **mã hóa – giải mã** (Seq2Seq): **bộ mã hóa** (RNN/LSTM) nén câu nguồn thành vector ngữ cảnh $\mathbf{c}$; **bộ giải mã** khởi tạo từ $\mathbf{c}$ và sinh lần lượt từng từ của câu đích. Khi huấn luyện dùng *teacher forcing*, khi suy luận dùng *tìm kiếm chùm* (beam search).

4.8.2. Nút thắt cổ chai và cơ chế chú ý

Seq2Seq cơ bản phải nén cả câu nguồn vào **một vector kích thước cố định** – "nút thắt cổ chai" khiến chất lượng dịch giảm rõ khi câu dài hơn 20–30 từ. **Cơ chế chú ý** của Bahdanau và cộng sự [30] cho bộ giải mã, ở mỗi bước, **"nhìn lại" mọi trạng thái ẩn** $\mathbf{h}_1, \dots, \mathbf{h}_T$ của bộ mã hóa. Tại bước giải mã t với trạng thái $\mathbf{s}_{t-1}$:

$$e_{t,j} = \mathrm{score}(\mathbf{s}_{t-1}, \mathbf{h}_j), \qquad \alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^{T} \exp(e_{t,k})}, \qquad \mathbf{c}_t = \sum_{j=1}^{T} \alpha_{t,j}\, \mathbf{h}_j$$

Hàm score (một mạng nhỏ, hoặc tích vô hướng như Luong và cộng sự, 2015) đo mức liên quan; **trọng số chú ý** $\alpha_{t,j}$ là phân phối trên các vị trí nguồn, và **vector ngữ cảnh** $\mathbf{c}_t$ thay đổi theo từng bước. Ví dụ khi dịch "I love you" thành "Tôi yêu bạn", lúc sinh "yêu" trọng số tập trung vào "love"; ma trận trọng số còn giúp diễn giải sự căn chỉnh hai câu.

4.8.3. Từ attention đến Transformer

Năm 2017, **Transformer** [9] bỏ hẳn kết nối hồi quy, chỉ dùng **tự chú ý** (mỗi vị trí chú ý tới mọi vị trí trong cùng chuỗi) kết hợp mã hóa vị trí. Nhờ **song song hóa** được toàn bộ chuỗi khi huấn luyện và đường đi giữa hai vị trí bất kỳ chỉ dài 1 thay vì $O(T)$, Transformer thay thế RNN trong hầu hết bài toán ngôn ngữ và là nền tảng của BERT, GPT, LLM. Đổi lại, chi phí self-attention tăng theo bình phương độ dài chuỗi, $O(T^2)$, trong khi RNN chỉ tăng tuyến tính $O(T)$ và có bộ nhớ trạng thái cố định khi suy luận.

4.9. Ứng dụng của RNN

Xử lý ngôn ngữ tự nhiên. RNN từng là kiến trúc chủ đạo của NLP giai đoạn 2013–2018: mô hình ngôn ngữ, phân tích cảm xúc, nhận dạng thực thể (BiLSTM-CRF), dịch máy – năm 2016 Google Translate chuyển sang hệ thống dịch nơ-ron dựa trên LSTM (GNMT).

Nhận dạng tiếng nói. LSTM hai chiều nhiều tầng ánh xạ chuỗi khung đặc trưng phổ (10–25 ms) thành chuỗi ký tự; hàm mất mát **CTC** cho phép huấn luyện mà không cần căn chỉnh từng khung với từng ký tự.

Dự báo chuỗi thời gian (điện năng, giao thông, doanh số, thời tiết): dùng cửa sổ trượt w bước quá khứ để dự đoán tương lai; cần chia tập theo thời gian và so sánh với mô hình cơ sở đơn giản, vốn thường cạnh tranh được với mạng nơ-ron.

Phát hiện bất thường và ứng dụng khác. LSTM học dự đoán chuỗi bình thường (log hệ thống, cảm biến, giao dịch thẻ) và cảnh báo khi sai số vượt ngưỡng; ngoài ra còn sinh chú thích ảnh (CNN + LSTM), nhận dạng chữ viết tay, phân tích video và hệ thống gợi ý theo chuỗi hành vi.

4.10. Thực hành huấn luyện và đánh giá

4.10.1. Các siêu tham số và kỹ thuật thực hành

Khi huấn luyện mô hình hồi quy, cần chú ý các điểm sau:

- **Kích thước trạng thái ẩn và số tầng:** thường từ 64 đến 1.024 chiều và 1–4 tầng; tăng lên làm mô hình mạnh hơn nhưng dễ quá khớp và chậm hơn.
- **Đệm và mặt nạ chuỗi:** chuỗi trong cùng lô được đệm cho bằng độ dài, dùng mặt nạ để vị trí đệm không ảnh hưởng tới trạng thái và hàm mất mát.
- **Tối ưu hóa:** Adam hoặc RMSProp với tốc độ học khoảng 10^{-3}, kết hợp cắt gradient (ngưỡng khoảng 1–5).
- **Điều chuẩn:** dropout giữa các tầng; với kết nối hồi quy dùng *dropout biến phân* (cùng một mặt nạ ở mọi bước); dừng sớm theo tập xác thực.
- **Khởi tạo:** trọng số hồi quy khởi tạo trực giao; độ lệch cổng quên của LSTM khởi tạo bằng 1.

4.10.2. Đánh giá mô hình tuần tự

Thước đo đánh giá phụ thuộc vào loại bài toán. Với dự báo giá trị liên tục: MAE, RMSE, MAPE. Với phân loại chuỗi: accuracy, precision, recall, F_1 như Chương 2. Với gán nhãn chuỗi như nhận dạng thực thể: F_1 tính trên mức thực thể. Với mô hình ngôn ngữ: **độ hỗn loạn** (perplexity) $\mathrm{PPL} = \exp\big(-\frac{1}{T}\sum_t \log p(w_t \mid w_{<t})\big)$ – càng thấp càng tốt. Với dịch máy: điểm **BLEU**; với nhận dạng tiếng nói: tỉ lệ lỗi từ (WER); các thước đo tự động cho sinh văn bản cần kết hợp đánh giá của con người.

Với dữ liệu theo thời gian, cần đánh giá *walk-forward* (huấn luyện trên quá khứ, kiểm tra trên giai đoạn kế tiếp) và theo dõi hiệu năng sau triển khai.

4.11. Hạn chế của RNN và xu hướng hiện nay

Các hạn chế chính của RNN gồm:

- **Tính toán tuần tự:** vì $\mathbf{h}_t$ phụ thuộc $\mathbf{h}_{t-1}$, không thể song song hóa theo trục thời gian khi huấn luyện, khiến RNN không tận dụng tốt GPU và khó mở rộng lên dữ liệu khổng lồ như Transformer.
- **Phụ thuộc rất dài vẫn khó học:** LSTM/GRU cải thiện đáng kể, nhưng thông tin vẫn phải đi qua nhiều bước trung gian và bị nén trong một vector trạng thái kích thước cố định; với chuỗi hàng nghìn bước, hiệu quả giảm rõ.
- **Khó huấn luyện:** nhạy cảm với khởi tạo, tốc độ học; cần cắt gradient và nhiều kinh nghiệm tinh chỉnh.
- **Khả năng giải thích hạn chế:** trạng thái ẩn là vector dày, khó diễn giải trực tiếp thông tin nào đang được lưu.

Tuy vậy, RNN vẫn có những ưu điểm riêng: chi phí suy luận mỗi bước là hằng số và bộ nhớ cố định bất kể độ dài chuỗi, phù hợp với **xử lý luồng dữ liệu trực tuyến**

(streaming), thiết bị nhúng, IoT với tài nguyên hạn chế và yêu cầu độ trễ thấp. Gần đây, ý tưởng hồi quy quay trở lại qua các **mô hình không gian trạng thái** như Mamba (2023), RWKV hay xLSTM (2024): huấn luyện song song như Transformer nhưng suy luận tuyến tính như RNN.

4.12. Thực nghiệm: LSTM trên ba bài toán dữ liệu tuần tự

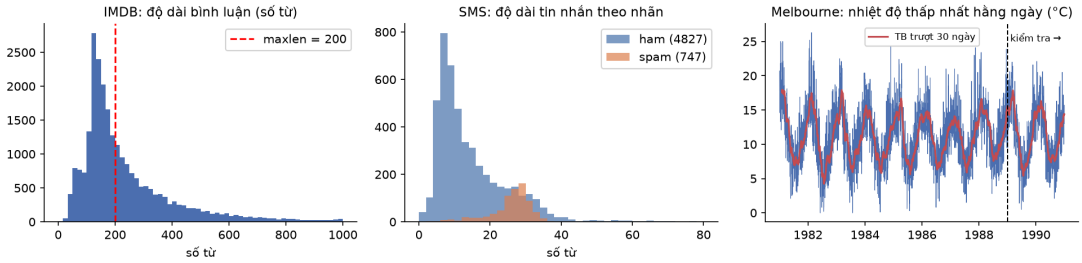
Phần này cài đặt LSTM theo đúng các phương trình ở mục 4.5.2 bằng ba cách – tự viết bằng NumPy (gồm cả lan truyền ngược theo thời gian), Keras [38] và PyTorch [37] – cho ba dạng bài toán: phân loại cảm xúc văn bản dài, phát hiện tin nhắn rác và dự báo chuỗi thời gian; cuối cùng so sánh RNN đơn giản, LSTM và GRU. Mã nguồn nằm trong notebook `code/ch4_lstm.ipynb`; các đoạn mã cốt lõi được trích ở mục 4.12.2.

4.12.1. Dữ liệu và thiết lập

Bảng 4.3. Ba bộ dữ liệu tuần tự dùng trong thực nghiệm chương 4

Bộ dữ liệu	Bài toán	Quy mô	Đặc điểm chính
IMDB Movie Reviews [43]	Phân loại nhị phân (nhiều – một): bình luận phim tích cực / tiêu cực	25.000 huấn luyện + 25.000 kiểm tra	Cân bằng tuyệt đối 50/50; dài trung bình 239 từ, trung vị 178, dài nhất 2.494
SMS Spam Collection [44]	Phân loại nhị phân: tin nhắn rác / bình thường	5.574 tin nhắn	Lệch lớp: chỉ 13,4% là tin rác (747 tin)
Daily Min Temperatures (Melbourne)	Hồi quy: dự báo nhiệt độ thấp nhất ngày kế tiếp	3.650 ngày (1981–1990)	Trung bình 11,2 °C, độ lệch chuẩn 4,1 °C, dao động 0–26,3 °C, có chu kỳ năm rõ rệt

Khảo sát dữ liệu (Hình 4.3) định hướng cho tiền xử lý. Độ dài bình luận IMDB lệch phải với đuôi rất dài; độ dài tối đa 200 từ giữ trọn 57% bình luận. Tin rác SMS dài hơn rõ rệt (trung bình 25,3 so với 14,4 từ); độ dài tối đa 50 từ bao trọn 98,8% tin nhắn. Chuỗi nhiệt độ có tính mùa vụ mạnh cộng với dao động ngẫu nhiên lớn giữa các ngày.



Hình 4.3. Độ dài bình luận IMDB, độ dài tin nhắn SMS theo nhãn, và chuỗi nhiệt độ Melbourne (đường đứt: điểm chia tập kiểm tra)

Văn bản được tách từ, giữ 10.000 (IMDB) hoặc 5.000 (SMS) từ phổ biến nhất – từ điển SMS chỉ xây trên tập huấn luyện để tránh rò rỉ dữ liệu – rồi đệm hoặc cắt ở *phía trước* để bước cuối, nơi lấy trạng thái ẩn ra phân loại, luôn là từ thật. Với chuỗi thời gian, mỗi mẫu là cửa sổ 30 ngày, mục tiêu là ngày thứ 31; dữ liệu được chia **theo thời gian** (80% đầu huấn luyện, 730 ngày cuối kiểm tra), không xáo trộn để mô hình không "nhìn thấy tương lai".

Kiến trúc gồm Embedding 32 chiều → LSTM 32 đơn vị → lớp Dense lấy $\mathbf{h}_T$ (bài toán nhiệt độ không có Embedding). Cả ba bản dùng khởi tạo theo mục 4.10.1 – Glorot cho ma trận đầu vào, **trực giao** cho ma trận hồi quy, **bias cổng quên bằng 1** [27] – và Adam với tốc độ học 10^{-3}, **cắt gradient** theo chuẩn 5; số epoch / kích thước lô là 3/64 (IMDB), 5/64 (SMS), 20/32 (nhiệt độ).

4.12.2. Cài đặt

Đoạn mã 4.1 cài đặt đúng các phương trình ở mục 4.5.2: bốn khối được tính chung bằng một phép nhân ma trận rồi cắt theo thứ tự cổng i, f, g, o (giống Keras và PyTorch). Lan truyền ngược theo thời gian (mục 4.4.1) đi từ bước cuối về bước đầu, mang theo hai gradient `dh` và `dc`; dòng `dc = dc * f` chính là "đường cao tốc" gradient qua ô nhớ – gradient chỉ bị nhân với cổng quên chứ không qua ma trận trọng số và đạo hàm tanh như ở RNN đơn giản (mục 4.5.3).

Đoạn mã 4.1. Lan truyền xuôi và lan truyền ngược theo thời gian của LSTM bằng NumPy

```
def lstm_forward(self, inp):
    p, H = self.p, self.H
    X = p["E"][inp] if self.use_emb else inp  # (B, T, D)
    B, T, _ = X.shape
    Zx = X @ p["Wx"] + p["b"]  # phần đầu vào, tính trước cho mọi bước
    h = np.zeros((B, H), np.float32)
    c = np.zeros((B, H), np.float32)
    cache = []
    for t in range(T):
        z = Zx[:, t] + h @ p["Wh"]
        i, f = sigmoid(z[:, :H]), sigmoid(z[:, H : 2 * H])
        g, o = np.tanh(z[:, 2 * H : 3 * H]), sigmoid(z[:, 3 * H :])
        c_prev, h_prev = c, h
        c = f * c_prev + i * g  # cập nhật ô nhớ
```

```
        tc = np.tanh(c)
        h = o * tc  # trạng thái ẩn
        cache.append((i, f, g, o, c_prev, h_prev, tc))
    self.cache, self.X, self.inp, self.h_last = cache, X, inp, h
    return (h @ p["Wd"] + p["bd"]).ravel()  # đầu ra từ h_T

def lstm_backward(self, dout):
    p, H = self.p, self.H
    gr = {k: np.zeros_like(v) for k, v in p.items()}
    dout = dout[:, None].astype(np.float32)
    gr["Wd"], gr["bd"] = self.h_last.T @ dout, dout.sum(0)
    dh = dout @ p["Wd"].T
    dc = np.zeros_like(dh)
    B, T, D = self.X.shape
    dZ = np.empty((B, T, 4 * H), np.float32)
    for t in reversed(range(T)):  # lan truyền ngược theo thời gian
        i, f, g, o, c_prev, h_prev, tc = self.cache[t]
        do = dh * tc
        dc = dc + dh * o * (1 - tc**2)
        di, df, dg = dc * g, dc * c_prev, dc * i
        dz = np.concatenate(
            [di * i * (1 - i), df * f * (1 - f), dg * (1 - g**2), do * o * (1 - o)], 1
        )
        dZ[:, t] = dz
        gr["Wh"] += h_prev.T @ dz
        dh = dz @ p["Wh"].T  # gradient về h_{t-1}
        dc = dc * f  # gradient về c_{t-1}
    gr["Wx"] = self.X.reshape(-1, D).T @ dZ.reshape(-1, 4 * H)
    gr["b"] = dZ.sum((0, 1))
    if self.use_emb:
        # chỉ cộng gradient vào các hàng ứng với từ xuất hiện trong lô
        np.add.at(gr["E"], self.inp, dZ @ p["Wx"].T)
    return gr
```

4.12.3. Kết quả

Bảng 4.4. LSTM: so sánh ba cách cài đặt trên hai bài toán phân loại văn bản

Bộ dữ liệu	Cài đặt	Số tham số	Độ chính xác (%)	Precision (%)	Recall (%)	F1 (%)	Thời gian (s)
IMDB	NumPy (CPU)	328.353	85,40	90,55	79,06	84,42	37,4
	Keras (CPU)	328.353	85,64	81,92	91,48	86,44	30,3
	PyTorch (GPU)	328.481	86,67	85,16	88,81	86,95	4,2
SMS Spam	NumPy (CPU)	168.353	99,01	97,92	94,63	96,25	2,5
	Keras (CPU)	168.353	99,01	98,59	93,96	96,22	3,7
	PyTorch (GPU)	168.481	98,65	94,67	95,30	94,98	0,7

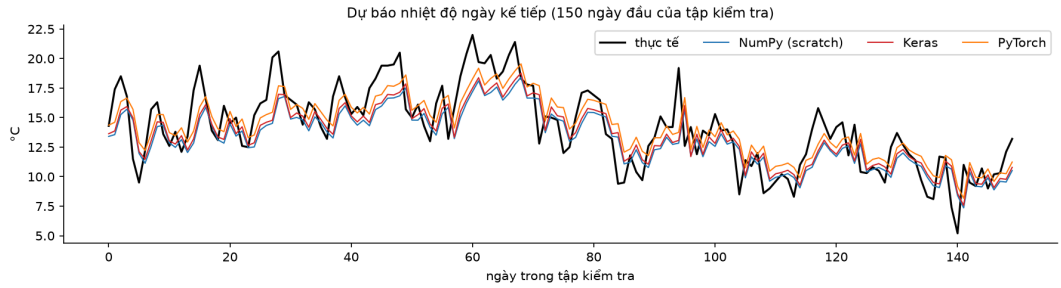
Bảng 4.5. LSTM: dự báo nhiệt độ ngày kế tiếp trên 730 ngày kiểm tra (đơn vị °C)

Mô hình	Số tham số	MAE	RMSE	Thời gian (s)
Cơ sở: lấy nhiệt độ hôm nay làm dự báo cho ngày mai	0	1,95	2,48	–
LSTM – NumPy (CPU)	4.385	1,85	2,34	3,9
LSTM – Keras (CPU)	4.385	1,79	2,26	7,5
LSTM – PyTorch (GPU)	4.513	**1,74**	**2,21**	3,2

Ba cách cài đặt cho độ chính xác chênh nhau không quá 1,3 điểm và loss gần như trùng nhau, xác nhận phần BPTT tự cài đặt là đúng (kiểm tra gradient số học cho sai số tương đối khoảng 10^{-6}). Số tham số khớp công thức ở mục 4.5.4 (lớp LSTM: $4 \cdot 32 \cdot 65 = 8.320$); PyTorch nhiều hơn 128 vì dùng hai vector bias. Trên IMDB, cặp precision – recall khác nhau giữa các bản dù độ chính xác gần bằng nhau – dấu hiệu mô hình chưa hội tụ hẳn sau 3 epoch, nên F1 là thước đo đáng tin hơn.

Với **SMS**, độ chính xác 99% cần đọc cùng tỉ lệ lớp: đoán "không phải rác" cho mọi tin đã đạt 86,6%. Thông tin quan trọng là precision (95–99%) và recall (94–95%) của lớp rác – đúng như phân tích về dữ liệu mất cân bằng ở mục 2.5.3.

Với **chuỗi thời gian**, LSTM tốt hơn mô hình cơ sở "ngày mai giống hôm nay" nhưng chỉ giảm MAE 5–11%, vì nhiệt độ hằng ngày có thành phần ngẫu nhiên lớn mà 30 ngày quá khứ không đủ dự đoán. Hình 4.4 cho thấy dự báo bám tốt xu hướng và mùa vụ nhưng **mượt hơn thực tế** – hành vi điển hình của mô hình tối thiểu hóa MSE.



Hình 4.4. Dự báo nhiệt độ thấp nhất ngày kế tiếp so với thực tế (150 ngày đầu của tập kiểm tra)

Khác với CNN, bản NumPy chỉ chậm hơn Keras một chút (37,4 s so với 30,3 s trên IMDB): RNN phải tính tuần tự theo thời gian nên thư viện trên CPU cũng không song song hóa được (mục 4.11); PyTorch trên GPU dùng nhân LSTM tối ưu của cuDNN nên nhanh hơn 7–9 lần.

4.12.4. So sánh RNN đơn giản, LSTM và GRU

Cùng kiến trúc và thiết lập (PyTorch), chỉ thay loại ô hồi quy (Bảng 4.6):

Bảng 4.6. So sánh ba loại ô hồi quy (PyTorch)

Bộ dữ liệu	Ô hồi quy	Tham số của lớp hồi quy	Độ chính xác (%)	F1 (%)	Loss huấn luyện cuối
IMDB (200 từ)	RNN đơn giản	2.112	83,44	82,76	0,251
	LSTM	8.448	**86,67**	**86,95**	0,179
	GRU	6.336	86,50	86,92	0,182
SMS (50 từ)	RNN đơn giản	2.112	**98,83**	**95,56**	0,017
	LSTM	8.448	98,65	94,98	0,013
	GRU	6.336	98,65	95,05	0,009

- **Trên chuỗi dài (IMDB, 200 bước), LSTM và GRU vượt RNN đơn giản hơn 3 điểm** và hội tụ nhanh hơn (loss sau epoch đầu 0,404 so với 0,554) – hệ quả trực tiếp của biến mất gradient (mục 4.4.2), mà đường truyền qua ô nhớ của LSTM khắc phục được (mục 4.5.3).
- **Trên chuỗi ngắn (SMS, 50 bước), ba loại ô tương đương**: khi các từ khóa như “FREE”, “WIN” đã đủ nhận diện tin rác, cơ chế cổng không mang lại lợi ích đáng kể.
- **GRU gần như bằng LSTM với ít hơn 25% tham số** ở lớp hồi quy, phù hợp với nhận định của Chung và cộng sự [31] ở mục 4.6.

4.13. Tiểu kết chương 4

Chương 4 đã trình bày mạng nơ-ron hồi quy – kiến trúc cho dữ liệu tuần tự với trạng thái ẩn cập nhật qua từng bước và trọng số chia sẻ giữa các bước. Huấn luyện bằng BPTT gặp vấn đề biến mất và bùng nổ gradient; LSTM và GRU giải quyết bằng các cổng và đường truyền gradient gần như không suy giảm. RNN hai chiều, nhiều tầng và mã hóa – giải mã mở rộng phạm vi ứng dụng; cơ chế chú ý khắc phục nút thắt cổ chai và dẫn tới Transformer. Thực nghiệm với LSTM (khoảng 86% trên IMDB, 99% trên SMS, tốt hơn mô hình cơ sở 5–11% khi dự báo nhiệt độ) và so sánh ba loại ô hồi quy đã kiểm chứng các phân tích về biến mất gradient. RNN vẫn là mắt xích lý thuyết quan trọng và hữu ích cho xử lý luồng dữ liệu với tài nguyên hạn chế.

CHƯƠNG 5. ĐỒ THỊ TRI THỨC VÀ SINH VĂN BẢN TĂNG CƯỜNG TRUY XUẤT

5.1. Động lực và lịch sử phát triển

5.1.1. Giới hạn của tri thức nằm trong tham số

Các mô hình ở Chương 2 – 4 đều lưu "tri thức" học được dưới dạng **tham số**: trọng số của hồi quy, bộ lọc của CNN, ma trận chuyển trạng thái của LSTM. Mô hình ngôn ngữ lớn (mục 1.8) đẩy cách làm này tới cực hạn – hàng trăm tỉ tham số nén lại một phần lớn văn bản trên Internet. Tuy nhiên, tri thức tham số có bốn hạn chế cố hữu:

1. **Khó truy xuất chính xác:** mô hình dự đoán token có xác suất cao nhất chứ không "tra cứu" một sự kiện, nên có thể sinh ra câu trôi chảy nhưng sai – hiện tượng **ảo giác** (hallucination).
2. **Khó cập nhật:** tri thức dừng ở thời điểm cắt dữ liệu huấn luyện (knowledge cutoff); muốn bổ sung phải huấn luyện lại hoặc tinh chỉnh, tốn kém và có nguy cơ làm quên tri thức cũ.
3. **Không có nguồn gốc:** mô hình không chỉ ra được nhận định của mình đến từ tài liệu nào, nên người dùng khó kiểm chứng.
4. **Không chứa dữ liệu riêng:** sổ tay nội bộ, hồ sơ khách hàng hay bảng giá cập nhật hằng ngày của một doanh nghiệp không có trong dữ liệu huấn luyện công khai.

Chương này trình bày hai kỹ thuật bổ khuyết cho nhau. **Đồ thị tri thức** (Knowledge Graph – KG) trả lời câu hỏi *tri thức được tổ chức ở đâu và như thế nào* – dưới dạng thực thể và quan hệ tường minh, kiểm chứng được. **Sinh văn bản tăng cường truy xuất** (Retrieval-Augmented Generation – RAG) trả lời câu hỏi *làm sao để LLM dùng tri thức đó đúng lúc* – truy xuất bằng chứng liên quan và đưa vào ngữ cảnh trước khi sinh câu trả lời. Điểm hội tụ của hai hướng là **GraphRAG**, được kiểm chứng bằng thực nghiệm ở mục 5.4.

5.1.2. Lịch sử đồ thị tri thức

Đồ thị tri thức không phải sản phẩm ra đời để đối phó với LLM mà là kết quả của hơn nửa thế kỷ nghiên cứu về biểu diễn tri thức – giúp máy tính hiểu *quan hệ* giữa các khái niệm thay vì chỉ so khớp chuỗi ký tự [53], [54]. Có thể chia thành năm giai đoạn:

- **Thập niên 1960 – 1970: mạng ngữ nghĩa.** Quillian [45] đề xuất **mạng ngữ nghĩa** (semantic network) làm mô hình trí nhớ: các khái niệm là nút, quan hệ như "là một" (is-a), "có" (has) là cạnh, và suy luận là lan truyền dọc theo

cạnh. Năm 1972, thuật ngữ “knowledge graph” lần đầu được Edgar W. Schneider dùng khi bàn về hệ thống khóa học mô-đun.

- **Thập niên 1980 – 1990: cơ sở tri thức chuyên đề.** Dự án **Cyc** [47] (Lenat, từ 1984) thủ công mã hóa hàng triệu khẳng định về tri thức thường thức bằng ngôn ngữ logic bậc cao CycL; **WordNet** [46] (từ 1985) tổ chức từ vựng tiếng Anh thành mạng các tập đồng nghĩa và quan hệ ngữ nghĩa. Cuối thập niên 1980, các đại học Groningen và Twente (Hà Lan) khởi động dự án *Knowledge Graphs*, giới hạn cạnh trong một tập quan hệ chuẩn để có thể thực hiện phép toán trên đồ thị.
- **Thập niên 1990 – 2000: Web và Web ngữ nghĩa.** Công cụ tìm kiếm thời kỳ đầu chỉ khớp từ khóa: truy vấn “apple” trả về lẫn lộn quả táo và công ty Apple. Tầm nhìn **Web ngữ nghĩa** (Semantic Web) của Berners-Lee, Hendler và Lassila [48] (2001) đề xuất gắn ý nghĩa máy đọc được cho dữ liệu Web; W3C chuẩn hóa mô hình **RDF**, ngôn ngữ ontology **OWL** và ngôn ngữ truy vấn **SPARQL**.
- **2007 – 2018: bùng nổ đồ thị tri thức quy mô lớn.** Năm 2007, **DBpedia** [49] (trích xuất từ hộp thông tin Wikipedia), **YAGO** và **Freebase** [50] ra đời; năm 2012, **Wikidata** [51] trở thành cơ sở tri thức mở do cộng đồng biên tập. Ngày 16/5/2012, Google công bố **Google Knowledge Graph** [52] với khẩu hiệu *“things, not strings”* (sự vật, không phải chuỗi ký tự): thay vì chỉ liệt kê liên kết, trang kết quả hiển thị bảng thông tin về thực thể được hỏi. Thuật ngữ “knowledge graph” từ đó trở nên phổ biến; Microsoft (Satori), Amazon (Product Graph), Facebook, LinkedIn, Airbnb, eBay… lần lượt xây dựng KG riêng.
- **2019 – nay: KG trong kỷ nguyên LLM.** Wikidata đạt hơn 100 triệu thực thể; khoảng một phần ba các trang Web dùng từ vựng schema.org để chú thích dữ liệu có cấu trúc. LLM được dùng để tự động trích xuất thực thể, quan hệ và đối sánh thực thể; ngược lại, KG được dùng làm nguồn bằng chứng có cấu trúc cho LLM – tiêu biểu là GraphRAG của Microsoft Research (2024) [70].

Xét về phương pháp luận, lịch sử trên phản ánh sự chuyển dịch từ thiết kế **từ trên xuống** (top-down), luật suy diễn phức tạp và quy mô hàng nghìn khái niệm của mạng ngữ nghĩa, sang phát triển **từ dưới lên** (bottom-up) với bộ ba đơn giản, vector nhúng và quy mô hàng tỉ sự kiện, kết hợp kỹ thuật tri thức, đóng góp cộng đồng và học máy [54].

5.1.3. Lịch sử RAG

Ý tưởng kết hợp truy xuất với sinh văn bản có gốc rễ từ các hệ thống hỏi đáp miền mở: tìm đoạn văn liên quan bằng truy xuất thông tin (TF-IDF, BM25) rồi trích câu trả lời từ đoạn đó. Các mốc chính:

- **2019 – 2020: truy xuất dày bằng mạng nơ-ron. DPR** [59] (Dense Passage Retrieval) mã hóa câu hỏi và đoạn văn bằng hai bộ mã hóa BERT, tìm đoạn liên quan bằng tích vô hướng và vượt BM25 trên nhiều bộ hỏi đáp; **REALM** [60] tích hợp bộ truy xuất vào quá trình tiền huấn luyện mô hình ngôn ngữ.
- **2020: RAG ra đời.** Lewis và cộng sự [58] (Facebook AI Research, NeurIPS 2020) đặt tên *Retrieval-Augmented Generation* cho mô hình kết hợp **bộ nhớ tham số** (mô hình seq2seq BART tiền huấn luyện) với **bộ nhớ phi tham số** (chỉ mục vector dày của Wikipedia, truy cập qua DPR). Hai biến thể RAG-Sequence và RAG-Token đạt kết quả tốt nhất thời điểm đó trên ba bộ hỏi đáp miền mở và sinh câu trả lời cụ thể, đúng sự thật hơn mô hình chỉ dùng tham số.
- **2022 – 2023: RAG cho LLM.** Khi ChatGPT phổ biến, ảo giác và nhu cầu hỏi đáp trên dữ liệu riêng khiến RAG trở thành mẫu kiến trúc mặc định của ứng dụng LLM. Mở rộng cửa sổ ngữ cảnh không thay thế được RAG: Liu và cộng sự [66] chỉ ra hiện tượng **"lạc ở giữa"** (lost in the middle) – thông tin nằm ở giữa một ngữ cảnh dài dễ bị mô hình bỏ qua. RAG tiến hóa từ dạng *đơn giản* (Naive RAG) sang *nâng cao* (Advanced RAG) với truy xuất lai, xếp hạng lại, viết lại truy vấn, và *mô-đun* (Modular RAG) [67]; các khung LangChain, LlamaIndex chuẩn hóa lớp điều phối.
- **2023 – 2024: RAG tự kiểm soát và GraphRAG. Self-RAG** [68] huấn luyện mô hình tự quyết định khi nào cần truy xuất và tự phê bình mức độ được hỗ trợ của câu trả lời; **Corrective RAG** [69] đánh giá chất lượng tài liệu truy xuất và truy xuất lại khi kém. Microsoft Research công bố **GraphRAG** [70] dùng LLM dựng đồ thị thực thể từ văn bản, gom cộng đồng và tóm tắt từng cộng đồng để trả lời cả các câu hỏi mang tính toàn cục.
- **2025 – nay: Agentic RAG.** Truy xuất được giao cho các tác tử LLM tự lập kế hoạch, chọn công cụ (tìm vector, truy vấn đồ thị, gọi API), kiểm chứng và lặp lại [72]; đánh giá RAG chuẩn hóa quanh các thước đo độ trung thực (faithfulness), độ liên quan của câu trả lời và độ chính xác/độ phủ của ngữ cảnh [73].

5.1.4. Vai trò và ý nghĩa của KG và RAG

Đối với đồ thị tri thức:

- **Tích hợp dữ liệu phân mảnh.** Dữ liệu doanh nghiệp nằm rải rác trong nhiều cơ sở dữ liệu, ứng dụng và văn bản. KG cung cấp một lược đồ chung theo khái niệm nghiệp vụ (khách hàng – *có nhà cung cấp* – nhà cung cấp; sự kiện tin tức – *liên quan tới* – khách hàng), cho phép hợp nhất dữ liệu nội bộ với dữ liệu bên ngoài. Ví dụ: tin một nhà bán lẻ phá sản có thể được lan dọc theo chuỗi quan hệ cung ứng để cảnh báo rủi ro cho các nhà cung cấp liên quan [54].
- **Suy luận ra tri thức ẩn.** Trong Wikidata, quan hệ "thành phố kết nghĩa" được khai báo là *đối xứng*; nếu trang của thành phố A ghi B là thành phố kết nghĩa, bộ suy luận tự suy ra chiều ngược lại dù trang của B quên ghi [54].
- **Tìm kiếm theo thực thể.** Google Knowledge Graph chuyển tìm kiếm từ danh sách liên kết sang câu trả lời trực tiếp về thực thể; trợ lý ảo và các công cụ tìm kiếm AI cũng cần dữ liệu có cấu trúc để trích dẫn chính xác thông tin như giờ mở cửa, giá, dịch vụ.
- **Biểu diễn cho các thuật toán AI.** Trong xử lý ngôn ngữ, câu văn được chuyển thành tập thực thể – quan hệ; trong thị giác, ảnh được biểu diễn thành *đồ thị cảnh* (scene graph: người – *mặc* – áo; người – *cho ăn* – bò) phục vụ hỏi đáp về ảnh; trong hệ gợi ý, sở thích lan truyền trên đồ thị người dùng – sản phẩm.
- **Nền tri thức cho LLM.** KG cung cấp các sự kiện có nguồn gốc và độ tin cậy để LLM dựa vào, giảm ảo giác và tăng khả năng giải thích – đây là vai trò trung tâm trong chương này.

Đối với RAG:

- **Giảm ảo giác:** neo câu trả lời vào bằng chứng được truy xuất thay vì chỉ dựa vào mẫu thống kê trong trọng số (dù không loại bỏ hoàn toàn).
- **Tri thức cập nhật và đặc thù miền mà không huấn luyện lại:** chỉ cần cập nhật kho tri thức; một mô hình nền chung có thể phục vụ nhiều miền khác nhau.
- **Chi phí thấp:** rẻ hơn nhiều so với huấn luyện lại hay tinh chỉnh; lớp truy xuất tốt cho phép dùng mô hình nhỏ hơn cho tác vụ chuyên biệt.
- **Kiểm chứng được:** câu trả lời kèm trích dẫn tới đoạn nguồn – yêu cầu bắt buộc trong y tế, pháp lý, tài chính.

- **Kiểm soát và bảo mật:** tri thức nằm ngoài mô hình nên có thể cấp, thu hồi quyền truy cập và lọc theo quyền của người hỏi ngay ở bước truy xuất.

5.2. Đồ thị tri thức (Knowledge Graph)

5.2.1. Định nghĩa

Chưa có một định nghĩa duy nhất được mọi bên chấp nhận. Ehrlinger và Wöß [87] tổng hợp các đặc trưng thường gặp: KG mô tả các thực thể của thế giới thực và quan hệ giữa chúng dưới dạng đồ thị; định nghĩa các lớp và quan hệ trong một lược đồ (schema); cho phép liên kết tùy ý giữa các thực thể; và có thể bao phủ nhiều lĩnh vực. Một định nghĩa ngắn gọn, phù hợp với tiểu luận này là:

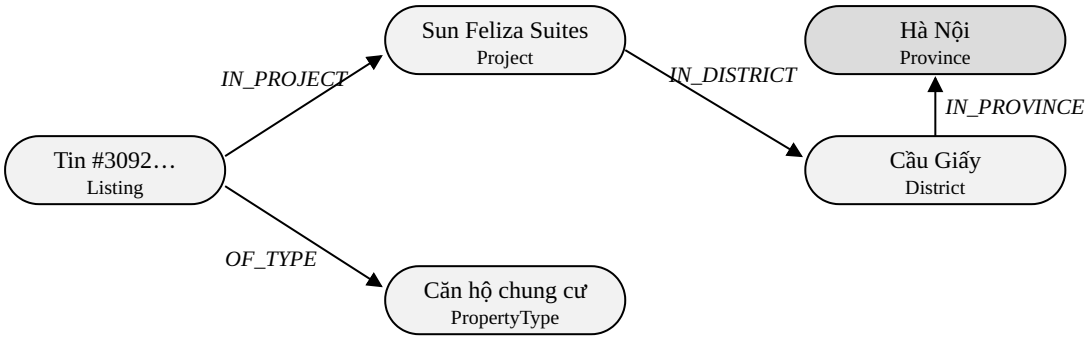
> ***Đồ thị tri thức*** *là một đồ thị có hướng, có gán nhãn, trong đó các nút biểu diễn thực thể (người, tổ chức, địa điểm, sự kiện, khái niệm), các cạnh biểu diễn quan hệ có ý nghĩa giữa chúng, và ý nghĩa của nút, cạnh được xác định bởi một lược đồ hoặc ontology để cả người lẫn máy đều hiểu và suy luận được [53], [54].*

Khác với cơ sở dữ liệu quan hệ – lưu bản ghi trong các bảng và dựng quan hệ lúc truy vấn bằng phép nối (join) – KG lưu trực tiếp mạng lưới quan hệ, nên phù hợp với các truy vấn đi qua nhiều bước quan hệ (“bạn của bạn”, “dự án thuộc quận nào, quận đó thuộc tỉnh nào”).

5.2.2. Bộ ba và các mô hình dữ liệu

Đơn vị cơ sở của KG là **bộ ba** (triple) (h, r, t) – *chủ thể* (head), *vị từ* (relation) và *tân ngữ* (tail), ví dụ (Hà Nội, *là thủ đô của*, Việt Nam). Tập hợp các bộ ba tạo thành đồ thị có hướng gán nhãn $\mathcal{G} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$, với $\mathcal{E}$ là tập thực thể và $\mathcal{R}$ là tập quan hệ (Hình 5.1). Trong thực tế có hai họ mô hình dữ liệu chính:

- **RDF** (Resource Description Framework): mỗi tài nguyên được định danh duy nhất bằng IRI (ví dụ `wd:Q1858` là Hà Nội trong Wikidata); mọi thông tin, kể cả thuộc tính, đều là bộ ba. Truy vấn bằng **SPARQL**. RDF thuận lợi cho chia sẻ và liên kết dữ liệu giữa các tổ chức (Linked Open Data).
- **Đồ thị thuộc tính** (property graph) [56]: nút và cạnh đều có nhãn và có thể mang các cặp thuộc tính – giá trị (ví dụ cạnh *đăng_tháng* có thuộc tính ngày cụ thể). Truy vấn bằng **Cypher** (Neo4j) hoặc **GQL** – ngôn ngữ truy vấn đồ thị được ISO chuẩn hóa năm 2024. Thuộc tính trên cạnh rất hữu ích để ghi thời gian hiệu lực, độ tin cậy và nguồn gốc của một sự kiện.



Hình 5.1. Một mảnh đồ thị tri thức bất động sản dạng đồ thị thuộc tính (trích từ thực nghiệm ở mục 5.4)

5.2.3. Ontology và truy vấn đồ thị

Trên tầng dữ liệu, **ontology** đặc tả lớp tri thức "lược đồ": các lớp thực thể và phân cấp giữa chúng (Căn hộ chung cư *là một* Bất động sản), miền và đối miền của quan hệ (*IN_DISTRICT* nối Phường hoặc Dự án tới Quận), tính chất của quan hệ (đối xứng, bắc cầu, nghịch đảo) và ràng buộc. Các từ vựng chuẩn như schema.org giúp nhiều nguồn dùng chung tên quan hệ, nhờ đó truy vấn được trên nhiều tập dữ liệu mà không cần ánh xạ thủ công.

Truy vấn trên KG quy về **khớp mẫu đồ thị**. Ví dụ, câu hỏi "giá/m² trung vị căn hộ ở các quận của Hà Nội, quận nào cao nhất?" viết bằng Cypher trên đồ thị của mục 5.4:

```
MATCH (m:MarketStat {scope: 'district', property_type: 'Căn hộ chung cư',
                     month: 'ALL', deal_type: 'sale'})-[:STAT_OF]→(d:District)
      -[:IN_PROVINCE]→(:Province {key: 'Hà Nội'})
WHERE m.n ⩾ 20
RETURN d.name AS quan, m.median_price_m2_mil AS gia_m2_trieu, m.n AS so_tin
ORDER BY gia_m2_trieu DESC LIMIT 5
```

Mẫu `(m)-[:STAT_OF]→(d)-[:IN_PROVINCE]→(p)` được khớp trực tiếp bằng cách duyệt cạnh từ nút này sang nút kia; cơ sở dữ liệu đồ thị lưu sẵn con trỏ tới các cạnh kề (index-free adjacency), nên chi phí mỗi bước chỉ phụ thuộc số cạnh kề chứ không phụ thuộc kích thước toàn bộ bảng như phép nối trong SQL.

5.2.4. Xây dựng đồ thị tri thức

Có hai trường phái xây dựng, thường được kết hợp [54]:

- **Tuyển chọn bởi con người** (human curation): chuyên gia hoặc cộng đồng nhập và kiểm duyệt tri thức. Cyc phải làm thủ công vì tri thức thường thức hầu như không được viết tường minh ở đâu; Wikidata dựa vào cộng đồng

hàng chục nghìn biên tập viên thường xuyên, chấp nhận nhiều giá trị mâu thuẫn cùng tồn tại (nguyên tắc *đa nguyên*) nhưng mỗi giá trị phải có nguồn tham chiếu. Ưu điểm là chất lượng cao; nhược điểm là chậm và khó mở rộng.

- **Xây dựng tự động bởi máy** (machine curation): trích xuất từ dữ liệu có cấu trúc (bảng, hộp thông tin), bán cấu trúc (HTML, JSON) và phi cấu trúc (văn bản, ảnh). Amazon Product Graph phủ hàng triệu chủng loại sản phẩm với hàng nghìn thuộc tính – không thể làm thủ công – nên dùng các mô hình trích xuất thuộc tính từ văn bản và ảnh, học chuyển giao giữa các chủng loại.

Một quy trình xây dựng KG từ dữ liệu điển hình gồm các bước:

1. **Thiết kế lược đồ/ontology** cùng chuyên gia nghiệp vụ: chọn loại thực thể, quan hệ và thuộc tính cần thiết cho các câu hỏi mục tiêu.
2. **Nhận dạng thực thể** (Named Entity Recognition – NER): tìm các đoạn văn bản chỉ thực thể (“Cầu Giấy”, “Sunshine Sky City”).
3. **Liên kết thực thể** (entity linking) và **giải nhập nhằng**: gắn mỗi đoạn với đúng một nút trong đồ thị (“Q7”, “Quận 7”, “quận bảy” → cùng một nút District).
4. **Trích xuất quan hệ** (relation extraction): xác định quan hệ giữa các thực thể, bằng luật, mô hình học có giám sát hoặc LLM với đầu ra có cấu trúc.
5. **Hợp nhất, làm sạch và kiểm tra ràng buộc:** khử trùng, xử lý xung đột, kiểm tra các bất biến (mỗi tin thuộc đúng một loại hình…).
6. **Nạp vào cơ sở dữ liệu đồ thị** và **làm giàu**: tính thêm các nút tổng hợp, chạy thuật toán đồ thị (độ trung tâm, phát hiện cộng đồng) để tạo đặc trưng mới.

5.2.5. Suy luận và nhúng đồ thị tri thức

Suy luận ký hiệu. Với ontology và luật, bộ suy luận rút ra sự kiện mới từ sự kiện đã có: từ (A, *là một*, Căn hộ) và (Căn hộ, *là một*, Bất động sản) suy ra (A, *là một*, Bất động sản) nhờ tính bắc cầu; từ quan hệ đối xứng suy ra chiều ngược lại. Suy luận ký hiệu chính xác và giải thích được, nhưng không xử lý tốt dữ liệu thiếu, nhiễu.

Nhúng đồ thị tri thức (knowledge graph embedding) [55] biểu diễn mỗi thực thể và quan hệ bằng một vector trong không gian liên tục, tương tự word2vec cho từ. Mô hình kinh điển **TransE** [57] coi quan hệ là một *phép tịnh tiến*: với bộ ba đúng (h, r, t) thì $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$. Hàm điểm và hàm mất mát biên (margin) là

$$f(h,r,t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|, \qquad \mathcal{L} = \sum_{(h,r,t)\in\mathcal{G}} \sum_{(h',r,t')\notin\mathcal{G}} \max\left(0,\ \gamma - f(h,r,t) + f(h',r,t')\right)$$

trong đó (h', r, t') là bộ ba "sai" sinh bằng cách thay ngẫu nhiên đầu hoặc đuôi, γ là biên. Sau khi huấn luyện, các vector nhúng cho phép **dự đoán liên kết** (link prediction) – bổ sung cạnh còn thiếu bằng cách xếp hạng các ứng viên t theo $f(h, r, t)$ – và đo độ tương tự giữa thực thể. Các mô hình về sau (DistMult, ComplEx, RotatE) biểu diễn tốt hơn các quan hệ một – nhiều, đối xứng, nghịch đảo. **Mạng nơ-ron đồ thị** (Graph Neural Network – GNN) đi xa hơn: biểu diễn của mỗi nút được cập nhật bằng cách tổng hợp thông điệp từ các nút kề – chủ đề của Chương 6.

Suy luận lai ký hiệu – nơ-ron (neuro-symbolic) kết hợp hai cách trên: vector nhúng đem lại khả năng tổng quát hóa và chịu nhiễu, còn đồ thị và luật đem lại tính tường minh, kiểm chứng được. Ví dụ, từ luật ký hiệu "nếu lá cây có mùi bạc hà thì cây có mùi bạc hà", "nếu người thích mùi đó thì thích cây" và luật xác suất trong một KG nhân quả "nếu thích thì có thể mua", hệ thống giải thích được vì sao một khách hàng mua một chậu cây [54].

5.2.6. Đối sánh thực thể

Vì không có chuẩn chung để xây dựng KG, cùng một thực thể thực tế thường xuất hiện trong nhiều đồ thị với tên gọi và cấu trúc khác nhau. **Đối sánh thực thể** (entity alignment) là bài toán xác định các nút ở hai đồ thị cùng chỉ một thực thể, dựa trên tên và thuộc tính giống nhau, cấu trúc lân cận tương tự, hoặc kết hợp; các phương pháp hiện đại so khớp vector nhúng của hai đồ thị, và từ 2023 LLM cho kết quả tốt nhờ hiểu ngữ nghĩa của tên và mô tả. Trong một đồ thị, bài toán tương tự là **khử trùng thực thể** – ví dụ cùng một bất động sản được đăng lại nhiều lần với nội dung sửa đôi chút (mục 5.4.1).

5.2.7. Hệ sinh thái và ứng dụng

- **Đồ thị tri thức mở:** Wikidata, DBpedia, YAGO, đám mây Linked Open Data, Data Commons (dân số, kinh tế, y tế, khí hậu).
- **Đồ thị doanh nghiệp:** Google Knowledge Graph, Microsoft Satori, Amazon Product Graph, đồ thị người dùng của LinkedIn, Facebook.
- **Cơ sở dữ liệu đồ thị:** Neo4j, Amazon Neptune, GraphDB, TigerGraph lưu trữ và truy vấn trực tiếp; **đồ thị tri thức ảo** (virtual KG) không sao chép dữ liệu mà ánh xạ truy vấn đồ thị sang các cơ sở dữ liệu quan hệ sẵn có.
- **Ứng dụng:** tìm kiếm và hỏi đáp, hệ gợi ý, phát hiện gian lận (tìm vòng giao dịch bất thường), y sinh (quan hệ gen – protein – thuốc – bệnh), chuỗi cung ứng, và làm nền tri thức cho LLM.

5.2.8. So sánh với mạng ngữ nghĩa và các thách thức

Bảng 5.1. Mạng ngữ nghĩa truyền thống và đồ thị tri thức hiện đại [54]

Tiêu chí	Mạng ngữ nghĩa (1960 – 1990)	Đồ thị tri thức hiện đại
Quy mô	Hàng nghìn khái niệm	Hàng triệu đến hàng tỉ sự kiện
Phương pháp phát triển	Từ trên xuống, thiết kế trước	Từ dưới lên, tăng dần theo dữ liệu
Biểu diễn	Luật, ontology phức tạp	Bộ ba đơn giản và vector nhúng
Suy luận	Logic phức tạp	Thuật toán đồ thị khả mở, suy luận lai ký hiệu – nơ-ron
Cách xây dựng	Kỹ thuật tri thức thủ công	Kết hợp chuyên gia, cộng đồng và học máy
Trọng tâm	Mô phỏng suy luận của con người	Tích hợp dữ liệu, tìm kiếm, phân tích, nền cho AI

Các thách thức chính của đồ thị tri thức gồm:

- **Giới hạn biểu đạt của quan hệ hai ngôi.** Câu "Los Angeles nằm giữa San Diego và San Jose trên quốc lộ 101" là một quan hệ bốn ngôi; đồ thị chỉ có cạnh nối hai nút nên phải *thực thể hóa* (reification) bằng một nút trung gian, làm đồ thị cồng kềnh. Hai câu "mọi người Thụy Điển đều có một nhà vua" và "mọi người Thụy Điển đều có một người mẹ" có cùng hình thức (Người Thụy Điển – *có* – Vua/Mẹ) nhưng ý nghĩa lượng từ khác hẳn (một nhà vua chung so với mỗi người một mẹ riêng) – bộ ba đơn giản không phân biệt được [54].
- **Chất lượng và tính đầy đủ:** đồ thị luôn thiếu cạnh, có sự kiện lỗi thời hoặc mâu thuẫn; cần ghi nguồn gốc, thời điểm và độ tin cậy cho từng sự kiện.
- **Tích hợp và quy ước định danh** giữa nhiều nguồn (đối sánh thực thể ở quy mô lớn).
- **Chi phí xây dựng và bảo trì lược đồ** khi nghiệp vụ thay đổi, và yêu cầu người dùng phải biết ngôn ngữ truy vấn – điều mà LLM sinh truy vấn (Text2Cypher, mục 5.3.6) đang giải quyết.

5.3. Sinh văn bản tăng cường truy xuất (RAG)

5.3.1. Nguyên lý

RAG kết nối LLM với một kho tri thức bên ngoài qua ba bước đúng như tên gọi: **truy xuất** (retrieve) các đoạn tri thức liên quan tới câu hỏi, **tăng cường** (augment) câu

lệnh gốc bằng các đoạn đó, rồi **sinh** (generate) câu trả lời dựa trên ngữ cảnh đã được bổ sung. Về hình thức, với câu hỏi x, bộ truy xuất $p_\eta(z \mid x)$ trả về top-k tài liệu z, và bộ sinh p_θ tạo câu trả lời y; mô hình RAG-Sequence của Lewis và cộng sự [58] lấy biên theo các tài liệu:

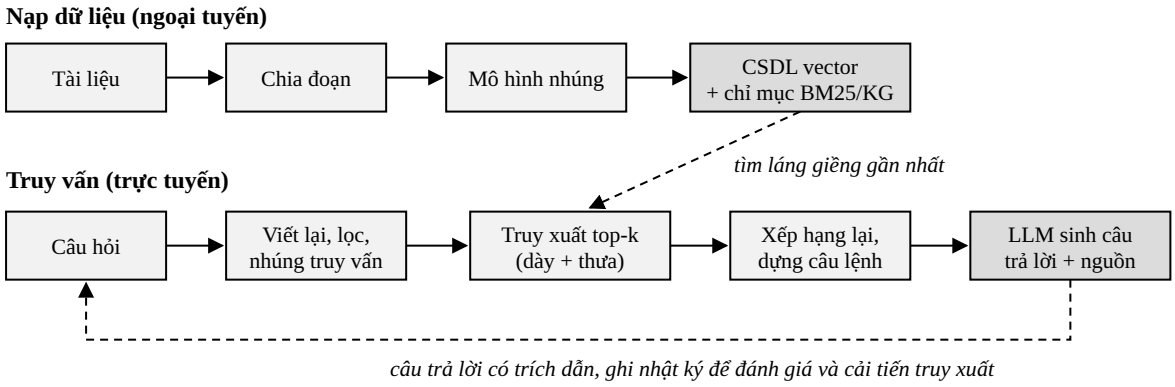
$$p(y \mid x) \approx \sum_{z \in \text{top-}k\,(p_\eta(\cdot|x))} p_\eta(z \mid x) \prod_{i=1}^{|y|} p_\theta(y_i \mid x, z, y_{1:i-1})$$

Trong các hệ thống LLM hiện nay, bộ truy xuất và LLM thường được giữ cố định (không huấn luyện đồng thời), và "lấy biên" được thay bằng việc ghép trực tiếp top-k đoạn vào câu lệnh. Như vậy, LLM không cần ghi nhớ mọi tri thức trong trọng số: tham số đảm nhận năng lực ngôn ngữ và suy luận, còn tri thức cụ thể được tra cứu từ bộ nhớ ngoài.

5.3.2. Kiến trúc và quy trình hoạt động

Một hệ thống RAG gồm hai luồng (Hình 5.2):

- **Luồng nạp dữ liệu** (ingestion, chạy trước, ngoại tuyến): thu thập tài liệu → làm sạch → chia đoạn (chunking) → mã hóa mỗi đoạn thành vector nhúng → lưu vào cơ sở dữ liệu vector kèm siêu dữ liệu (nguồn, thời gian, quyền truy cập), đồng thời có thể dựng chỉ mục từ khóa và đồ thị tri thức.
- **Luồng truy vấn** (trực tuyến): nhận câu hỏi → (tùy chọn) viết lại truy vấn, trích bộ lọc → mã hóa truy vấn → truy xuất ứng viên → xếp hạng lại và nén ngữ cảnh → dựng câu lệnh gồm chỉ dẫn hệ thống, ngữ cảnh và câu hỏi → LLM sinh câu trả lời có trích dẫn → kiểm tra đầu ra.



Hình 5.2. Hai luồng của một hệ thống RAG

5.3.3. Chia đoạn và nhúng

Chia đoạn (chunking) cắt tài liệu thành các đoạn nhỏ vì hai lý do: mô hình nhúng chỉ nhận một số token giới hạn (thường 512), và đoạn càng tập trung thì vector

càng đại diện đúng nội dung. Đoạn quá lớn chứa nhiều ý, vector bị "pha loãng" và đưa nhiều nhiễu vào ngữ cảnh; đoạn quá nhỏ mất mạch ngữ nghĩa. Một nguyên tắc thực tế: *nếu một đoạn có nghĩa với con người khi đứng một mình thì nó cũng có nghĩa với LLM*. Các chiến lược phổ biến:

- **Kích thước cố định có chồng lấn:** cắt mỗi N token với phần chồng lấn (ví dụ 400 token, chồng 100) để không mất thông tin ở biên; đơn giản nhưng không tôn trọng cấu trúc câu, đoạn.
- **Theo cấu trúc văn bản:** tách theo câu (với tiếng Việt cần công cụ tách câu, tách từ như VnCoreNLP, pyvi), tách đệ quy theo thứ tự đoạn → câu → từ cho tới khi đủ nhỏ, hoặc tách theo cấu trúc riêng của Markdown, HTML, mã nguồn.
- **Theo ngữ nghĩa:** đặt điểm cắt ở nơi độ tương tự giữa các câu liên tiếp giảm mạnh.
- **Nhỏ để tìm, lớn để đọc** (small-to-big, sentence-window): nhúng các đoạn nhỏ để truy xuất chính xác, nhưng đưa vào LLM cửa sổ lớn hơn bao quanh đoạn tìm được; biến thể phân cấp (auto-merging) gộp các đoạn con thành đoạn cha khi nhiều đoạn con cùng được truy xuất.

Nhúng (embedding) ánh xạ văn bản thành vector sao cho các văn bản cùng nghĩa nằm gần nhau. Biểu diễn *thưa* như TF-IDF, BM25 khớp chính xác từ vựng (mã sản phẩm, tên riêng, "2PN", "sổ hồng") và rẻ, nhưng không hiểu từ đồng nghĩa. Biểu diễn *dày* dùng mô hình Transformer kiểu **bi-encoder** (Sentence-BERT [64], E5 [75], BGE-M3): câu hỏi và đoạn văn được mã hóa *độc lập* thành một vector duy nhất, huấn luyện bằng học tương phản (contrastive learning) sao cho cặp liên quan có độ tương tự cosine cao, cặp không liên quan có độ tương tự thấp:

$$\cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}^\top \mathbf{d}}{\|\mathbf{q}\|\,\|\mathbf{d}\|}, \qquad \mathcal{L} = -\log \frac{\exp(\cos(\mathbf{q},\mathbf{d}^+)/\tau)}{\exp(\cos(\mathbf{q},\mathbf{d}^+)/\tau) + \sum_j \exp(\cos(\mathbf{q},\mathbf{d}_j^-)/\tau)}$$

với $\mathbf{d}^+$ là đoạn liên quan, $\mathbf{d}_j^-$ là các đoạn không liên quan và τ là tham số nhiệt độ. Khác với BERT gốc cho mỗi token một vector (phục vụ gán nhãn chuỗi, trích câu trả lời), bi-encoder cho một vector cho cả câu, nên có thể tính trước vector của toàn bộ kho và chỉ cần mã hóa câu hỏi lúc truy vấn. Mô hình nhúng nên được chọn bằng đo đạc trên chính dữ liệu của bài toán (bảng xếp hạng MTEB chỉ là điểm xuất phát) – xem Bảng 5.4.

5.3.4. Truy xuất: thưa, dày và lai

Truy xuất thưa với BM25 [61] chấm điểm tài liệu D cho truy vấn Q gồm các từ q_i:

$$\mathrm{BM25}(D,Q)=\sum_{i}\mathrm{IDF}(q_i)\cdot\frac{f(q_i,D)\,(k_1+1)}{f(q_i,D)+k_1\left(1-b+b\,\frac{|D|}{\mathrm{avgdl}}\right)}$$

trong đó $f(q_i,D)$ là tần suất từ trong tài liệu, $|D|$ là độ dài tài liệu, avgdl là độ dài trung bình, $k_1\approx 1{,}2$ điều khiển mức bão hòa tần suất và $b\approx 0{,}75$ điều khiển mức chuẩn hóa theo độ dài. BM25 thưởng cho từ hiếm (IDF cao) và không để một từ lặp nhiều lần chi phối điểm.

Truy xuất dày tìm k vector gần nhất với vector câu hỏi – chính là bài toán KNN ở mục 2.9 trên hàng triệu vector. Tìm vét cạn quá chậm, nên các cơ sở dữ liệu vector (Qdrant, Milvus, FAISS, pgvector) dùng chỉ mục **tìm láng giềng gần đúng** (ANN), phổ biến nhất là **HNSW** [63] (Hierarchical Navigable Small World): một đồ thị nhiều tầng, tầng trên thưa để "nhảy xa", tầng dưới dày để tinh chỉnh, cho thời gian truy vấn xấp xỉ $O(\log n)$ với độ phủ trên 95%.

Truy xuất lai kết hợp hai cách để tận dụng điểm mạnh của cả hai. Vì điểm BM25 và điểm cosine có thang đo khác nhau, cách gộp đơn giản và bền vững là **hợp nhất thứ hạng nghịch đảo** (Reciprocal Rank Fusion – RRF) [62] – chỉ dùng thứ hạng, không dùng điểm:

$$\mathrm{RRF}(d)=\sum_{r\in\mathcal{R}}\frac{1}{k+\mathrm{rank}_r(d)}$$

với $\mathcal{R}$ là tập các bảng xếp hạng (dày, thưa), $\mathrm{rank}_r(d)$ là thứ hạng của tài liệu d trong bảng r, và hằng số $k=60$ làm giảm ảnh hưởng của khác biệt giữa các hạng đầu.

Lọc trước theo siêu dữ liệu (pre-filtering): khi câu hỏi chứa điều kiện cấu trúc ("căn hộ 2 phòng ngủ ở Quận 7 dưới 4 tỷ"), trích các điều kiện thành bộ lọc trên siêu dữ liệu rồi chỉ tìm láng giềng trong tập thỏa mãn – hiệu quả hơn nhiều so với để vector tự "hiểu" con số. Thực nghiệm ở mục 5.4.5 cho thấy đây là thành phần đóng góp lớn nhất.

Xếp hạng lại (reranking): một **cross-encoder** [65] đưa *cả* câu hỏi và đoạn văn vào cùng một Transformer để chấm độ liên quan, chính xác hơn bi-encoder vì mô hình nhìn thấy tương tác giữa từng từ của hai bên, nhưng chậm hơn nhiều nên chỉ áp dụng cho vài chục ứng viên đầu. Xếp hạng lại kết hợp nén ngữ cảnh cũng là lời giải cho

hiện tượng “lạc ở giữa”: đưa ít đoạn hơn nhưng đúng hơn, đặt đoạn quan trọng ở đầu ngữ cảnh.

5.3.5. Các mô hình RAG

- **Naive RAG:** nạp – nhúng – tìm theo độ tương tự – sinh. Đủ tốt cho câu hỏi tra cứu sự kiện đơn giản trên văn bản phi cấu trúc, nhưng yếu với câu hỏi nhiều bước, câu hỏi tổng hợp và câu hỏi cần dữ liệu mới.
- **Advanced RAG:** cải tiến trước truy xuất (chia đoạn tốt hơn, viết lại truy vấn như Rewrite-Retrieve-Read, sinh tài liệu giả định HyDE/Query2Doc, trích bộ lọc) và sau truy xuất (xếp hạng lại, nén ngữ cảnh).
- **Modular RAG** [67]: tách hệ thống thành các mô-đun thay thế được (tìm kiếm, bộ nhớ, định tuyến, dự đoán, kiểm chứng) và cho phép các luồng phi tuyến như truy xuất lặp, truy xuất thích ứng.
- **GraphRAG** [70], [71]: thay hoặc bổ sung chỉ mục phẳng bằng đồ thị tri thức – trình bày ở mục 5.3.6.
- **Agentic RAG** [72]: một hoặc nhiều tác tử LLM (mục 1.9.2) tự lập kế hoạch truy xuất, phân rã câu hỏi, chọn công cụ, đánh giá bằng chứng và quyết định truy xuất thêm. Self-RAG [68] và CRAG [69] là các dạng “tự kiểm soát” sớm; mẫu ReAct [86] xen kẽ bước suy nghĩ với bước gọi công cụ là nền tảng của nhiều tác tử RAG.

5.3.6. GraphRAG – kết hợp đồ thị tri thức và RAG

RAG trên văn bản phẳng có ba điểm yếu cấu trúc: (1) chỉ tìm được đoạn *giống* câu hỏi, bỏ qua các *quan hệ* nối nhiều tài liệu (dự án → quận → tỉnh; công ty → nhà cung cấp → sự kiện); (2) không trả lời được câu hỏi **tổng hợp** trên toàn bộ dữ liệu (“quận nào có giá trung vị cao nhất?”) vì không có đoạn văn nào chứa sẵn đáp án, và top-k đoạn chỉ là một mẫu rất nhỏ; (3) không trả lời được câu hỏi **toàn cục** (“các chủ đề chính trong bộ tài liệu là gì?”). GraphRAG khắc phục bằng cách đưa cấu trúc đồ thị vào cả ba giai đoạn [71]: *lập chỉ mục dựa trên đồ thị*, *truy xuất có dẫn hướng bởi đồ thị* và *sinh câu trả lời được tăng cường bởi đồ thị*. Có ba hướng chính:

- **Đồ thị sinh từ văn bản (Microsoft GraphRAG [70]):** LLM trích thực thể và quan hệ từ từng đoạn văn để dựng đồ thị; thuật toán **Leiden** [74] chia đồ thị thành các cộng đồng phân cấp; LLM tóm tắt từng cộng đồng. *Tìm kiếm cục bộ* xuất phát từ các thực thể trong câu hỏi và mở rộng sang hàng xóm; *tìm kiếm toàn cục* trả lời câu hỏi bao quát bằng cách tổng hợp (map-reduce) các bản tóm tắt cộng đồng.

- **Truy vấn đồ thị có cấu trúc (Text2Cypher/Text2SPARQL):** khi đã có KG với lược đồ rõ ràng, LLM được cung cấp lược đồ và vài ví dụ để dịch câu hỏi tự nhiên thành truy vấn Cypher; cơ sở dữ liệu đồ thị thực thi chính xác phép lọc, đếm, xếp hạng, đi nhiều bước; LLM diễn giải kết quả thành câu trả lời. Cách này cho kết quả *chính xác về số liệu* nhưng phụ thuộc vào việc LLM viết đúng truy vấn và cần các lớp bảo vệ (chỉ cho phép đọc, giới hạn số dòng, thời gian chờ).
- **Đồ thị làm bộ lọc cho tìm kiếm vector:** đồ thị khoanh vùng ứng viên theo thực thể và loại quan hệ hợp lệ (ví dụ chỉ các tin thuộc một dự án), sau đó tìm kiếm vector xếp hạng trong vùng đã khoanh – vừa linh hoạt về ngữ nghĩa vừa không truy xuất nhầm sang quan hệ không liên quan.

Trong thực tế, một **bộ định tuyến** (router) phân loại câu hỏi để chọn nhánh phù hợp; đây chính là kiến trúc "Hybrid" được thực nghiệm ở mục 5.4.

5.3.7. So sánh RAG và tinh chỉnh

RAG và tinh chỉnh (fine-tuning) đều giúp LLM làm tốt hơn trong một miền chuyên biệt, nhưng đặt tri thức ở hai nơi khác nhau: tinh chỉnh thay đổi **trọng số** – phù hợp để dạy văn phong, định dạng đầu ra, hành vi ít thay đổi; RAG thay đổi **những gì mô hình nhìn thấy lúc suy luận** – phù hợp với tri thức động, cần trích dẫn. Hệ thống thực tế thường dùng cả hai.

Bảng 5.2. So sánh RAG và tinh chỉnh

Tiêu chí	RAG	Tinh chỉnh (fine-tuning)
Nơi lưu tri thức	Kho ngoài (vector, đồ thị, SQL)	Trọng số mô hình
Cập nhật tri thức	Dễ: thêm, sửa, xóa tài liệu trong kho	Khó: phải huấn luyện lại
Chi phí	Thấp ở huấn luyện, thêm chi phí truy xuất mỗi câu hỏi	Cao ở huấn luyện, suy luận không đổi
Trích dẫn, kiểm chứng	Có: dẫn tới đoạn nguồn	Khó giải thích nguồn gốc
Phân quyền dữ liệu	Lọc theo quyền tại bước truy xuất	Dữ liệu đã hòa vào trọng số, khó thu hồi
Phù hợp nhất cho	Tri thức thay đổi, dữ liệu riêng, hỏi đáp có nguồn	Văn phong, định dạng, kỹ năng chuyên biệt

5.3.8. Đánh giá hệ thống RAG

Hệ thống RAG được đánh giá ở hai tầng. **Tầng truy xuất** dùng các thước đo xếp hạng, với $\mathrm{rel}_i \in \{0,1\}$ cho biết kết quả ở vị trí i có liên quan hay không:

$$\mathrm{Recall@}k = \frac{|\{\text{tài liệu đúng}\} \cap \{\text{top-}k\}|}{|\{\text{tài liệu đúng}\}|}, \qquad \mathrm{MRR} = \frac{1}{|Q|}\sum_{q \in Q}\frac{1}{\mathrm{rank}_q}, \qquad \mathrm{nDCG@}k = \frac{}{\mathrm{I}}$$

trong đó rank_q là vị trí của kết quả đúng đầu tiên và IDCG là DCG của thứ tự lý tưởng. Recall@k đo *có tìm thấy hay không*, MRR và nDCG đo thêm *kết quả đúng nằm có cao không*.

Tầng sinh đo câu trả lời: độ chính xác so với đáp án chuẩn (khớp chính xác, khớp số với sai số cho phép, hoặc LLM làm giám khảo), **độ trung thực** (faithfulness – mọi nhận định có được ngữ cảnh hỗ trợ không), **độ liên quan** của câu trả lời, và **độ chính xác/độ phủ ngữ cảnh** – bốn thước đo của khung RAGAS [73]. Ngoài ra cần đo chi phí (số token), độ trễ (trung vị p50 và p95), tỉ lệ trích dẫn sai và khả năng **từ chối đúng** khi dữ liệu không đủ. Vì bộ câu hỏi đánh giá thường nhỏ, kết quả nên kèm khoảng tin cậy (ví dụ bootstrap) để không kết luận quá mức từ chênh lệch vài câu.

5.3.9. Ưu điểm, hạn chế và thách thức vận hành

Ưu điểm của RAG đã được nêu ở mục 5.1.4: chính xác hơn, cập nhật nhanh, chi phí thấp, có trích dẫn, kiểm soát được dữ liệu. **Hạn chế** gồm:

- **Phụ thuộc chất lượng kho tri thức và chất lượng truy xuất:** truy xuất sai thì LLM trả lời sai một cách tự tin; "rác vào, rác ra" (mục 2.4) vẫn đúng.
- **Trôi ngữ cảnh** (context drift): kho chứa tài liệu giống về ngữ nghĩa nhưng đã lỗi thời – lược đồ dữ liệu đổi tên, định nghĩa nghiệp vụ thay đổi, tài liệu không còn người chịu trách nhiệm. Cần gắn siêu dữ liệu quản trị (chủ sở hữu, ngày hiệu lực, ngày kiểm tra gần nhất) ngay từ khi nạp và loại khỏi chỉ mục tài liệu hết hiệu lực.
- **An ninh:** truy xuất không tôn trọng phân quyền có thể làm lộ dữ liệu qua độ tương tự vector; vector nhúng có thể bị đảo ngược để khôi phục một phần văn bản gốc; văn bản truy xuất có thể chứa **chèn lệnh gián tiếp** (indirect prompt injection) – chỉ thị độc hại nằm trong tài liệu. Kiểm soát truy cập phải thực thi ở tầng lọc truy xuất, và câu lệnh hệ thống phải tách rõ *dữ liệu* với *chỉ dẫn*.
- **Chi phí hạ tầng và độ trễ:** thêm các bước nhúng, tìm kiếm, xếp hạng lại; với GraphRAG còn thêm chi phí dựng và bảo trì đồ thị.

5.4. Thực nghiệm: trợ lý hỏi đáp thị trường bất động sản Việt Nam

Phần này kiểm chứng các nội dung của chương bằng một hệ thống hỏi đáp thực trên dữ liệu tin đăng bất động sản Việt Nam, so sánh bốn cách trả lời trên **cùng dữ liệu, cùng bộ câu hỏi và cùng mô hình ngôn ngữ**: chỉ dùng LLM, RAG cơ bản, truy vấn đồ thị và kết hợp có định tuyến. Mã nguồn gồm thư viện Python `vre`, pipeline dữ liệu, API FastAPI, giao diện web React và năm notebook (`00_setup` đến `05_valuation`).

5.4.1. Bài toán, dữ liệu và tiền xử lý

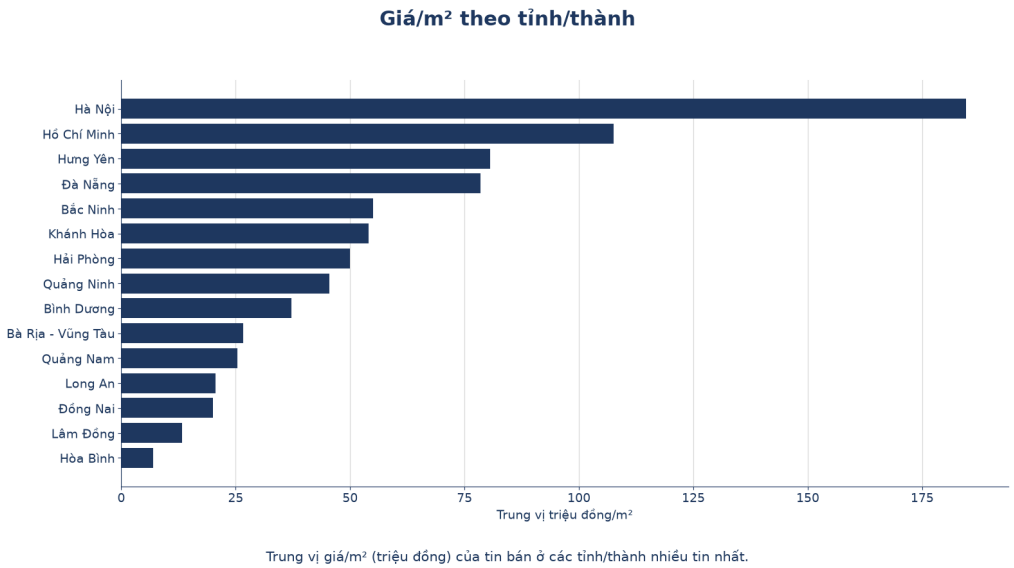
Bài toán. Người dùng hỏi bằng tiếng Việt tự nhiên, thường viết tắt (“2PN”, “Q7”, “sổ hồng”) và trộn nhiều loại yêu cầu: tìm một tin cụ thể, tìm tin theo điều kiện, hỏi số liệu thống kê (giá trung vị, số tin, xu hướng theo tháng), so sánh khu vực, câu hỏi nhiều bước, định giá một bất động sản, và cả những câu hỏi mà dữ liệu không trả lời được.

Dữ liệu. Bộ dữ liệu công khai *vietnam-real-estates* [77] (giấy phép CC BY-NC 4.0) gồm 3.500.744 tin đăng thô trong 10 phân đoạn. Pipeline DuckDB thực hiện chuẩn hóa địa danh, sửa giá nhập theo nghìn đồng, ẩn số điện thoại và email (bảo vệ dữ liệu cá nhân), rồi khử trùng (Bảng 5.3). Giá trong dữ liệu là **giá chào bán** do người đăng tự nhập, không phải giá giao dịch.

Bảng 5.3. Kết quả làm sạch và khử trùng dữ liệu

Bước	Số tin	Ghi chú
Tin thô	3.500.744	10 phân đoạn Parquet
Loại tin không có nội dung	1.868	
Loại trùng chính xác	416.939	Trùng toàn bộ nội dung
Loại trùng tiền tố	544.017	Đăng lại với nội dung bị cắt hoặc nối thêm
Tin giữ lại	**2.537.920**	2.537.601 tin bán, 319 tin cho thuê
Tin gần trùng (SimHash)	407.138	Giữ lại, gom thành 185.704 cụm (gắn `dup_cluster_id`)

Mức thiếu dữ liệu đáng kể: thiếu phường 14,8%, thiếu tên đường 39,8%, thiếu dự án 71,5%, thiếu số phòng ngủ 46,2%; 209.160 tin không đọc được giá và 46.285 tin có giá ngoại lai. Hình 5.3 minh họa mức chênh lệch giá rất lớn giữa các địa phương – một trong những lý do câu hỏi thống kê cần được trả lời bằng tính toán chính xác chứ không thể suy ra từ vài tin mẫu.



Hình 5.3. Giá/m² trung vị (triệu đồng) của tin bán ở các tỉnh/thành có nhiều tin nhất

Chỉ mục vector. Mỗi tin được chia đoạn kích thước cố định 400 token, chồng lấn 100 token (tối đa 64 đoạn mỗi tin), lưu trong Qdrant với hai vector cho mỗi điểm: vector dày 1.024 chiều và vector thưa BM25 (tách từ tiếng Việt bằng pyvi), kèm chỉ mục trên siêu dữ liệu (tỉnh, quận, loại hình, giá, diện tích, số phòng, tháng đăng) để lọc trước. Mô hình nhúng được chọn bằng đo đạc trên 20.000 đoạn và 1.000 câu truy vấn lấy từ chính dữ liệu (Bảng 5.4): `multilingual-e5-large` [75] có độ chính xác truy xuất cao nhất, dù chậm hơn `bge-m3` khoảng 18%. Toàn bộ lượt nhúng, tải lên và dựng chỉ mục HNSW chạy trên một GPU cá nhân mất 11,2 giờ (có khả năng tiếp tục sau khi bị ngắt).

Bảng 5.4. So sánh mô hình nhúng trên dữ liệu bất động sản (20.000 đoạn, 1.000 truy vấn)

Mô hình	Tham số (triệu)	Số chiều	Tốc độ (đoạn/giây)	Recall@1	Recall@10	MRR@10
multilingual-e5-small	118	384	802,3	0,591	0,769	0,649
multilingual-e5-base	278	768	429,6	0,589	0,759	0,643
bge-m3	568	1.024	120,3	0,626	0,782	0,681
multilingual-e5-large	560	1.024	98,8	**0,656**	**0,816**	**0,711**

Đồ thị tri thức. Bảng tin sạch được nạp vào Neo4j (bằng công cụ nhập hàng loạt, 75 giây) thành đồ thị gồm 2.695.799 nút và 13.430.582 cạnh. Lược đồ (Hình 5.1) gồm: `Listing` (tin đăng) nối với phân cấp hành chính `Ward` → `District` → `Province` (63 tỉnh, 648 quận/huyện, 6.258 phường/xã), `Street` (21.864), `Project` (5.056), `PropertyType` (5 loại hình), `Month` (10 tháng đăng) và `Amenity` (10 tiện ích trích từ mô tả, 3,4 triệu cạnh). Hai loại nút được **làm giàu** thêm:

- `MarketStat` (123.936 nút): thống kê tính sẵn (số tin, trung vị giá, trung vị giá/m², trung vị diện tích) cho từng tổ hợp *khu vực* × *loại hình* × *tháng*. Câu hỏi tổng hợp đọc các nút này thay vì quét 2,5 triệu tin – vừa nhanh vừa tránh hết thời gian chờ.
- `Cluster` (29 nút): cộng đồng của 3.391 phường tính bằng thuật toán Leiden [74] trên đồ thị tương tự giữa các phường (độ đo mô-đun 0,883), dùng cho câu hỏi "khu vực tương tự".

5.4.2. Bốn hệ thống được so sánh

Cả bốn hệ thống dùng chung mô hình sinh `gemini-flash-lite-latest` với đầu ra có cấu trúc (JSON schema) cho các bước trích bộ lọc, sinh Cypher và định tuyến:

1. **LLM only** (đối chứng): đưa thẳng câu hỏi cho LLM, không truy xuất – đo xem truy xuất đem lại thêm được gì.
2. **Basic RAG:** LLM trích bộ lọc cấu trúc từ câu hỏi (tỉnh, quận, loại hình, khoảng giá, số phòng, tháng…) → tìm lai dày + BM25 có lọc trước, gộp bằng RRF ($k = 60$) → mỗi tin chỉ giữ đoạn điểm cao nhất, lấy top-5 tin → LLM trả lời, mỗi nhận định kèm nguồn dạng `[Tin#ID]`. Nếu bộ lọc quá chặt (0 kết quả), hệ thống **nới dần** bộ lọc (tên dự án/đường trước, rồi tới điều kiện số) trước khi bỏ hẳn.
3. **Graph (Text2Cypher):** LLM nhận lược đồ đồ thị cùng các ví dụ mẫu, sinh câu Cypher; câu Cypher đi qua hai lớp bảo vệ – biểu thức chính quy chặn mọi lệnh ghi/quản trị (`CREATE`, `MERGE`, `DELETE`, `SET`, `CALL`…) và thực thi trong **giao dịch chỉ đọc** có thời gian chờ – và được tự động thêm `LIMIT` nếu thiếu. Nếu Neo4j báo lỗi, thông báo lỗi được gửi lại cho LLM để viết lại (tối đa 3 lần) – một dạng tự sửa lỗi của tác tử (mục 1.9.2). LLM diễn giải kết quả, ghi nguồn `[Cypher]` cho số liệu tổng hợp.
4. **Hybrid (định tuyến):** một LLM phân loại câu hỏi vào bốn tuyến: `basic_rag` (tìm tin theo điều kiện hoặc mô tả), `graph` (thống kê, xếp hạng, xu hướng, so sánh, nhiều bước), `hybrid` (tìm tin *trong* một dự án hoặc *trên* một con đường – dùng đồ thị khoanh tin rồi mới tìm) và `valuation` (gọi công cụ định giá

XGBoost [76] dự đoán giá/m²). Khi nhánh đồ thị lỗi hoặc trả về rỗng, hệ thống **lui về** Basic RAG.

5.4.3. Cài đặt

Đoạn mã 5.1 là hàm hợp nhất thứ hạng RRF của Basic RAG, cài đặt trực tiếp công thức ở mục 5.3.4: mỗi bảng xếp hạng (dày, BM25) đóng góp $1/(k + \text{hạng})$ cho mỗi tin.

Đoạn mã 5.1. Hợp nhất thứ hạng nghịch đảo (RRF)

```
def rrf(rankings: list[list], k: int | None = None) → list[tuple]:
    """rankings: danh sách các bảng xếp hạng, mỗi bảng là list ID theo thứ tự tốt dần."""
    k = get_settings().rrf_k if k is None else k
    scores: dict = {}
    for ranking in rankings:
        for pos, doc in enumerate(ranking, start=1):
            scores[doc] = scores.get(doc, 0) + 1 / (k + pos)
    return sorted(scores.items(), key=lambda x: -x[1])
```

Đoạn mã 5.2 là lớp bảo vệ cho Cypher do LLM sinh – điều kiện bắt buộc khi cho một mô hình ngôn ngữ truy cập cơ sở dữ liệu – và vòng lặp Text2Cypher có gửi lỗi trở lại cho LLM.

Đoạn mã 5.2. Chặn lệnh ghi, ép LIMIT và vòng lặp Text2Cypher tự sửa lỗi (rút gọn)

```
FORBIDDEN = re.compile(r"\b(CREATE|MERGE|DELETE|DETACH|SET|REMOVE|DROP|LOAD\s+CSV|CALL|FOREAC
   H|USE|SHOW|"
                       r"GRANT|DENY|REVOKE|ALTER|RENAME)\b", re.I)

def is_read_only(cypher: str) → bool:
    return FORBIDDEN.search(cypher) is None   # lớp thứ hai: giao dịch chỉ đọc của run_cypher

def text2cypher(question: str, retries: int = 2) → dict:
    prompt, system = f"Câu hỏi: {question}", cypher_system()
    for _ in range(retries + 1):
        result = ask_llm(prompt, system=system, output=CYPHER_OUTPUT)
        cypher = str((result.data or {}).get("cypher", "")).strip()
        if not is_read_only(cypher):
            error = "bị chặn: câu Cypher chứa lệnh ghi hoặc lệnh quản trị"
        else:
            cypher = ensure_limit(cypher)
            try:
                rows = run_cypher(cypher, timeout=s.cypher_timeout, max_rows=s.cypher_max_row
   s)
                return dict(cypher=cypher, rows=rows, error=None)
            except Neo4jError as exc:
                error = f"{exc.code}: {exc.message}"
        prompt = (f"Câu hỏi: {question}\n\nCâu Cypher trước:\n{cypher}\n\nLỗi: {error}\n"
                  "Hãy viết lại câu Cypher đúng.")
    return dict(cypher=cypher, rows=pd.DataFrame(), error=error)
```

Ngoài ra, câu lệnh hệ thống của bước sinh đặt kết quả truy xuất giữa hai thẻ `<tai_lieu>` và nói rõ đó chỉ là *dữ liệu* do người dùng tự nhập, phải bỏ qua mọi chỉ thị nằm bên trong – biện pháp chống chèn lệnh gián tiếp đã nêu ở mục 5.3.9. Hệ thống được đóng gói thành API FastAPI trả luồng sự kiện (SSE: token, nguồn, câu Cypher, tuyến đã chọn) và giao diện web có các trang Chat, So sánh bốn hệ thống, Bản đồ,

Định giá và Dashboard thị trường; API có giới hạn tần suất, độ dài câu hỏi và khóa truy cập.

5.4.4. Thiết lập đánh giá

Bộ câu hỏi gồm 100 câu tiếng Việt viết theo cách người dùng thật hỏi, chia thành 9 nhóm: tra cứu một tin (`lookup`, 12 câu), tìm theo điều kiện (`constraint`, 12), tổng hợp (`aggregate`, 12), nhiều bước (`multihop`, 12), theo thời gian (`temporal`, 10), so sánh (`comparison`, 10), dự án/đường phố (`project_street`, 10), định giá (`valuation`, 10) và không trả lời được (`unanswerable`, 12 – ví dụ hỏi giá năm 2024 trong khi dữ liệu chỉ có từ 2025). Ví dụ câu nhiều bước: *"Dự án Sunshine Sky City nằm ở quận nào, và giá/m² căn hộ của quận đó cao hay thấp hơn trung vị căn hộ toàn TP.HCM bao nhiêu %?"*. **Đáp án chuẩn** không gõ tay mà được tính bằng câu SQL trên toàn bộ bảng sạch trong DuckDB (lưu kèm câu SQL để kiểm tra lại); mỗi câu cũng được gán tuyến đúng để đánh giá bộ định tuyến.

Chấm điểm theo luật, không dùng LLM giám khảo (để tiết kiệm chi phí và tránh thiên lệch giám khảo): mỗi câu được 0, 1 hoặc 2 điểm. Câu đáp án là danh sách tin: 2 điểm nếu Recall@5 theo cụm tin trùng ≥ 0,8. Câu đáp án là số: 2 điểm nếu câu trả lời có một số nằm trong sai số cho phép (mặc định 5%) so với đáp án, 1 điểm nếu trong khoảng rộng hơn. Câu không trả lời được: 2 điểm nếu hệ thống từ chối đúng; ngược lại, câu trả lời từ chối ở câu có đáp án được 0 điểm. **Độ chính xác** là tỉ lệ câu đạt 2/2 điểm. Khoảng tin cậy 95% được tính bằng **bootstrap** 10.000 lần lấy mẫu lại theo câu hỏi; so sánh hai hệ thống dùng bootstrap ghép cặp trên cùng câu hỏi.

5.4.5. Kết quả

Bảng 5.5. Độ chính xác (%) theo nhóm câu hỏi, kèm khoảng tin cậy 95%

Nhóm (số câu)	LLM only	Basic RAG	Graph	Hybrid
Tra cứu một tin (12)	0,0	**100,0**	41,7 [17–67]	**100,0**
Tìm theo điều kiện (12)	0,0	16,7 [0–42]	**91,7** [75–100]	16,7 [0–42]
Tổng hợp (12)	16,7 [0–42]	16,7 [0–42]	**83,3** [58–100]	**83,3** [58–100]
Nhiều bước (12)	0,0	0,0	**83,3** [58–100]	**83,3** [58–100]
Theo thời gian (10)	0,0	10,0 [0–30]	**70,0** [40–100]	**70,0** [40–100]
So sánh (10)	40,0 [10–70]	10,0 [0–30]	**100,0**	90,0 [70–100]
Dự án/đường phố (10)	0,0	0,0	**100,0**	**100,0**
Định giá (10)	10,0 [0–30]	20,0 [0–50]	20,0 [0–50]	20,0 [0–50]

Không trả lời được (12)	33,3 [8–58]	66,7 [42–92]	**83,3** [58–100]	**83,3** [58–100]
Tổng (100)	**11,0** [5–17]	**28,0** [20–37]	**75,0** [66–83]	**72,0** [63–81]

Bảng 5.6. So sánh ghép cặp giữa các hệ thống (bootstrap 10.000 mẫu)

So sánh	Phạm vi	Chênh lệch (điểm %)	CI 95%	p
Basic RAG − LLM only	Toàn bộ 100 câu	+17,0	[+6,0; +28,0]	0,002
Hybrid − Basic RAG	Toàn bộ 100 câu	+44,0	[+33,0; +54,0]	< 0,001
Hybrid − Basic RAG	Nhóm tổng hợp	+66,7	[+25,0; +100,0]	0,001
Hybrid − Basic RAG	Nhóm nhiều bước	+83,3	[+58,3; +100,0]	< 0,001
Hybrid − Graph	Toàn bộ 100 câu	−3,0	[−12,0; +7,0]	0,298

Bảng 5.7. Chi phí và độ trễ

Hệ thống	Độ trễ p50 (ms)	Độ trễ p95 (ms)	Token vào/ra trung bình	Trích dẫn sai	Lui về Basic RAG
LLM only	1.770	9.568	83 / 160	0%	–
Basic RAG	3.480	22.945	1.887 / 268	0%	–
Graph	3.434	20.078	4.016 / 326	0%	–
Hybrid	2.700	19.181	4.154 / 407	0%	7%

Bộ định tuyến của Hybrid chọn đúng tuyến ở 96% số câu (97,7% nếu bỏ nhóm không trả lời được). Ở tầng truy xuất, Bảng 5.8 trích kết quả thử nghiệm loại bỏ (ablation) trên 27 câu hỏi tìm tin, đo bằng chỉ mục đầy đủ 2,5 triệu tin với `multilingual-e5-large`; bộ xếp hạng lại là cross-encoder đa ngữ `mmarco-mMiniLMv2-L12`, chấm lại 20 ứng viên đầu.

Bảng 5.8. Ablation truy xuất trên chỉ mục đầy đủ (27 câu)

Cấu hình	Recall@5	MRR	nDCG@10
Dày (dense)	0,29	0,28	0,28
BM25	0,27	0,30	0,26

Lai (dense + BM25, RRF)	0,36	0,30	0,30
Lai + xếp hạng lại	0,37	0,41	0,36
Dày + lọc trước	0,50	0,60	0,51
BM25 + lọc trước	0,58	0,68	0,60
Lai + lọc trước	0,53	0,65	0,55
Dày + lọc trước + xếp hạng lại	0,64	**0,74**	**0,66**
Lai + lọc trước + xếp hạng lại	**0,67**	0,71	**0,66**

Thử nghiệm kích thước đoạn (256, 400 và 512 token) trên một tập con chứa mọi tin đáp án cho kết quả gần như nhau khi có lọc trước (Recall@5 từ 0,90 đến 0,92), cho thấy với tin đăng bất động sản – văn bản ngắn, mỗi tin một chủ đề – kích thước đoạn không phải yếu tố quyết định.

5.4.6. Phân tích và thảo luận

- **LLM không có truy xuất gần như vô dụng với dữ liệu riêng** (11%): mô hình không biết tin đăng cụ thể hay số liệu của bộ dữ liệu; điểm có được chủ yếu đến từ việc từ chối đúng và đoán đúng chiều so sánh. Ở 81/100 câu nguyên nhân lỗi là *thiếu ngữ cảnh* – minh họa trực tiếp hạn chế 2 và 4 ở mục 5.1.1.
- **Basic RAG giỏi tra cứu nhưng không tính toán được.** RAG đạt 100% ở nhóm tra cứu một tin và cải thiện 17 điểm so với LLM only (có ý nghĩa thống kê, $p = 0{,}002$), nhưng gần như bằng 0 ở các nhóm tổng hợp, nhiều bước, theo thời gian: top-5 tin không thể đại diện cho trung vị của hàng chục nghìn tin. 55 câu lỗi của Basic RAG được phân loại là *ngữ cảnh không đủ* – đúng điểm yếu cấu trúc (2) của RAG phẳng đã phân tích ở mục 5.3.6.
- **Đồ thị tri thức thay đổi hẳn khả năng trả lời câu hỏi cấu trúc.** Graph và Hybrid đạt 70 – 100% ở các nhóm tổng hợp, nhiều bước, so sánh, dự án/đường phố; Hybrid hơn Basic RAG 44 điểm trên toàn bộ bộ câu hỏi (CI 95% từ +33 đến +54), riêng nhóm nhiều bước hơn 83 điểm. Các nút `MarketStat` tính sẵn biến câu hỏi thống kê thành một phép tra cứu trên đồ thị – ví dụ "quận nào ở TP.HCM có giá nhà/m² trung vị thấp nhất (ít nhất 20 tin)?" được trả lời đúng (Củ Chi, 33,5 triệu/m², 394 tin) trong khoảng 250 ms thực thi Cypher.
- **Định tuyến là sự đánh đổi.** Hybrid giữ được 100% ở nhóm tra cứu (nơi Graph chỉ đạt 41,7%), nhưng thua Graph ở nhóm *tìm theo điều kiện* (16,7% so với 91,7%): bộ định tuyến gửi các câu này sang Basic RAG đúng như thiết

kế, trong khi điều kiện số (giá, diện tích, số phòng) được Cypher lọc chính xác trên toàn đồ thị hơn là bộ lọc trước của tìm kiếm vector. Chênh lệch tổng giữa Hybrid và Graph (−3 điểm) **không có ý nghĩa thống kê** ($p = 0{,}298$) – với 100 câu, không thể kết luận hệ nào tốt hơn. Hướng cải tiến rõ ràng là chuyển nhóm câu hỏi tìm theo điều kiện số sang nhánh đồ thị hoặc nhánh lai.

- **Lọc trước theo siêu dữ liệu quan trọng hơn chọn dense hay BM25.** Ở Bảng 5.8, chỉ thêm lọc trước đã nâng Recall@5 của tìm kiếm dày từ 0,29 lên 0,50 và MRR từ 0,28 lên 0,60; kết hợp lọc trước, tìm lai và xếp hạng lại cho Recall@5 cao nhất (0,67). Không có lọc trước, tìm lai chỉ nhỉnh hơn từng phương pháp đơn lẻ – vì phần lớn câu hỏi bất động sản chứa điều kiện số và địa danh mà vector nhúng nắm bắt kém.
- **Lỗi còn lại của nhánh đồ thị** chủ yếu là *Cypher chạy được nhưng sai kết quả* (12 câu với Graph, 5 câu với Hybrid). Ví dụ câu so sánh giá căn hộ Cầu Giấy giữa hai quý được LLM dịch thành một câu Cypher sai logic gộp tháng, trả về rỗng nên hệ thống trả lời "không đủ dữ liệu"; câu hỏi diện tích trung vị đất ở Củ Chi trả về 175 m² so với đáp án SQL 186,95 m² – cần kiểm tra thêm sự khác biệt giữa thống kê tính sẵn trong `MarketStat` và định nghĩa trong câu SQL đáp án. Lỗi kiểu này **không bị lớp bảo vệ phát hiện** vì câu truy vấn hợp lệ; cần thêm kiểm chứng ngữ nghĩa (ví dụ đối chiếu số dòng, khoảng giá trị hợp lý).
- **Định giá vẫn khó** với cả bốn hệ thống (20%): mô hình XGBoost dùng đặc trưng cấu trúc đạt sai số tuyệt đối trung bình 32,6 triệu/m² (MAPE 24,0%, $R^2 = 0{,}73$), bổ sung vector mô tả nén bằng PCA cải thiện lên 24,1 triệu/m² (MAPE 19,1%, $R^2 = 0{,}84$), đều tốt hơn rõ rệt so với dự đoán bằng trung vị của phường (MAPE 46,0%); nhưng với ngưỡng chấm 5%, một ước lượng giá chào bán có nhiễu lớn hiếm khi đạt điểm tối đa.
- **Chi phí:** truy xuất làm tăng số token vào từ khoảng 80 lên 1.900 – 4.150 mỗi câu và tăng độ trễ trung vị lên 2,7 – 3,5 giây; độ trễ p95 khoảng 20 giây chủ yếu do giới hạn tần suất của API mô hình. Không có trích dẫn sai `[Tin#ID]` nào trong 100 câu.

Hạn chế của thực nghiệm. Bộ câu hỏi chỉ có 100 câu nên khoảng tin cậy rộng; chấm theo luật không đo được độ trung thực kiểu RAGAS; giá chào bán có nhiễu và dữ liệu tập trung mạnh ở Hà Nội và TP.HCM; địa danh theo địa giới trước khi sắp xếp đơn vị hành chính năm 2025; tin cho thuê quá ít để huấn luyện mô hình định giá thuê.

5.5. Tiểu kết chương 5

Chương 5 đã trình bày hai kỹ thuật đưa tri thức tường minh vào hệ thống thông minh dựa trên LLM. Đồ thị tri thức – hậu duệ của mạng ngữ nghĩa và AI biểu tượng – tổ chức tri thức thành thực thể và quan hệ, với mô hình dữ liệu RDF hoặc đồ thị thuộc tính, được xây dựng bằng sự kết hợp giữa con người và máy, hỗ trợ suy luận ký hiệu, vector nhúng như TransE và đối sánh thực thể. RAG bổ sung bộ nhớ phi tham số cho LLM qua chuỗi chia đoạn – nhúng – truy xuất lai (BM25, vector dày, RRF) – xếp hạng lại – sinh có trích dẫn, tiến hóa từ Naive RAG tới GraphRAG và Agentic RAG.

Thực nghiệm trên 2,5 triệu tin đăng bất động sản Việt Nam đã định lượng được vai trò của từng thành phần: truy xuất văn bản nâng độ chính xác từ 11% lên 28%, chủ yếu ở câu hỏi tra cứu; đồ thị tri thức với thống kê tính sẵn và Text2Cypher nâng lên 72 – 75%, đặc biệt ở câu hỏi tổng hợp và nhiều bước; còn ở tầng truy xuất, lọc trước theo siêu dữ liệu đóng góp nhiều hơn việc chọn giữa tìm kiếm dày và thưa. Kết quả khẳng định KG và RAG không thay thế mà **cộng sinh** với nhau: RAG phẳng mạnh ở mô tả tự do, KG mạnh ở cấu trúc và tính toán, và một bộ định tuyến – mầm mống của tác tử – quyết định dùng nguồn nào cho từng câu hỏi. Việc học trực tiếp trên cấu trúc đồ thị bằng mạng nơ-ron đồ thị và việc phối hợp nhiều tác tử sẽ là nội dung của Chương 6 và Chương 7.

TÀI LIỆU THAM KHẢO

1. Russell, S. và Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (ấn bản thứ 4). Pearson.
2. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
3. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
4. Goodfellow, I., Bengio, Y. và Courville, A. (2016). *Deep Learning*. MIT Press.
5. LeCun, Y., Bottou, L., Bengio, Y. và Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
6. Krizhevsky, A., Sutskever, I. và Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 25.
7. Hochreiter, S. và Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
8. Cho, K. và cộng sự (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of EMNLP 2014*.
9. Vaswani, A. và cộng sự (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
10. LeCun, Y., Bengio, Y. và Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.
11. Turing, A. M. (1950). Computing machinery and intelligence. *Mind*, 59(236), 433–460.
12. McCarthy, J., Minsky, M. L., Rochester, N. và Shannon, C. E. (1955). A proposal for the Dartmouth summer research project on artificial intelligence. (Tái bản trong *AI Magazine*, 27(4), 2006).
13. Rumelhart, D. E., Hinton, G. E. và Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
14. Cortes, C. và Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
15. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
16. Simonyan, K. và Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *ICLR 2015*.

17. Bengio, Y., Simard, P. và Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166.
18. Szegedy, C. và cộng sự (2015). Going deeper with convolutions. *Proceedings of CVPR 2015*.
19. He, K., Zhang, X., Ren, S. và Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of CVPR 2016*.
20. Ioffe, S. và Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of ICML 2015*.
21. Srivastava, N. và cộng sự (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
22. Tan, M. và Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *Proceedings of ICML 2019*.
23. Howard, A. G. và cộng sự (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*.
24. Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179–211.
25. Werbos, P. J. (1990). Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10), 1550–1560.
26. Pascanu, R., Mikolov, T. và Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *Proceedings of ICML 2013*.
27. Gers, F. A., Schmidhuber, J. và Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10), 2451–2471.
28. Schuster, M. và Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), 2673–2681.
29. Sutskever, I., Vinyals, O. và Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 27.
30. Bahdanau, D., Cho, K. và Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *ICLR 2015*.
31. Chung, J., Gulcehre, C., Cho, K. và Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv:1412.3555*.

32. Kingma, D. P. và Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR 2015*.
33. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (ấn bản thứ 3). O'Reilly Media.
34. Vũ Hữu Tiệp (2018). *Machine Learning cơ bản*. Nhà xuất bản Khoa học và Kỹ thuật.
35. Thủ tướng Chính phủ (2021). Quyết định số 127/QĐ-TTg ngày 26/01/2021 ban hành Chiến lược quốc gia về nghiên cứu, phát triển và ứng dụng trí tuệ nhân tạo đến năm 2030.
36. Pedregosa, F. và cộng sự (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
37. Paszke, A. và cộng sự (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*.
38. Chollet, F. và cộng sự (2015). Keras. https://keras.io.
39. Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.
40. Street, W. N., Wolberg, W. H. và Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. *Proceedings of SPIE 1905*, 861–870.
41. Xiao, H., Rasul, K. và Vollgraf, R. (2017). Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv:1708.07747*.
42. Krizhevsky, A. (2009). *Learning multiple layers of features from tiny images*. Technical report, University of Toronto.
43. Maas, A. L. và cộng sự (2011). Learning word vectors for sentiment analysis. *Proceedings of ACL 2011*, 142–150.
44. Almeida, T. A., Hidalgo, J. M. G. và Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. *Proceedings of ACM DocEng 2011*, 259–262.
45. Quillian, M. R. (1968). Semantic memory. Trong Minsky, M. (chủ biên), *Semantic Information Processing*, 227–270. MIT Press.
46. Miller, G. A. (1995). WordNet: A lexical database for English. *Communications of the ACM*, 38(11), 39–41.
47. Lenat, D. B. (1995). CYC: A large-scale investment in knowledge infrastructure. *Communications of the ACM*, 38(11), 33–38.

48. Berners-Lee, T., Hendler, J. và Lassila, O. (2001). The Semantic Web. *Scientific American*, 284(5), 34–43.
49. Auer, S. và cộng sự (2007). DBpedia: A nucleus for a web of open data. *Proceedings of ISWC 2007*, 722–735.
50. Bollacker, K. và cộng sự (2008). Freebase: A collaboratively created graph database for structuring human knowledge. *Proceedings of ACM SIGMOD 2008*, 1247–1250.
51. Vrandečić, D. và Krötzsch, M. (2014). Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10), 78–85.
52. Singhal, A. (2012). Introducing the Knowledge Graph: things, not strings. *Google Official Blog*, 16/5/2012.
53. Hogan, A. và cộng sự (2021). Knowledge graphs. *ACM Computing Surveys*, 54(4), 1–37.
54. Chaudhri, V. K. và cộng sự (2022). Knowledge graphs: Introduction, history, and perspectives. *AI Magazine*, 43(1), 17–29.
55. Ji, S., Pan, S., Cambria, E., Marttinen, P. và Yu, P. S. (2022). A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2), 494–514.
56. Robinson, I., Webber, J. và Eifrem, E. (2015). *Graph Databases* (ấn bản thứ 2). O'Reilly Media.
57. Bordes, A., Usunier, N., Garcia-Durán, A., Weston, J. và Yakhnenko, O. (2013). Translating embeddings for modeling multi-relational data. *Advances in Neural Information Processing Systems (NeurIPS)*, 26.
58. Lewis, P. và cộng sự (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
59. Karpukhin, V. và cộng sự (2020). Dense passage retrieval for open-domain question answering. *Proceedings of EMNLP 2020*, 6769–6781.
60. Guu, K., Lee, K., Tung, Z., Pasupat, P. và Chang, M.-W. (2020). REALM: Retrieval-augmented language model pre-training. *Proceedings of ICML 2020*.
61. Robertson, S. và Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.

62. Cormack, G. V., Clarke, C. L. A. và Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *Proceedings of ACM SIGIR 2009*, 758–759.
63. Malkov, Y. A. và Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824–836.
64. Reimers, N. và Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of EMNLP-IJCNLP 2019*, 3982–3992.
65. Nogueira, R. và Cho, K. (2019). Passage re-ranking with BERT. *arXiv:1901.04085*.
66. Liu, N. F. và cộng sự (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173.
67. Gao, Y. và cộng sự (2023). Retrieval-augmented generation for large language models: A survey. *arXiv:2312.10997*.
68. Asai, A., Wu, Z., Wang, Y., Sil, A. và Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR 2024*.
69. Yan, S.-Q., Gu, J.-C., Zhu, Y. và Ling, Z.-H. (2024). Corrective retrieval augmented generation. *arXiv:2401.15884*.
70. Edge, D. và cộng sự (2024). From local to global: A Graph RAG approach to query-focused summarization. *arXiv:2404.16130*.
71. Peng, B. và cộng sự (2024). Graph retrieval-augmented generation: A survey. *arXiv:2408.08921*.
72. Singh, A., Ehtesham, A., Kumar, S. và Talaei Khoei, T. (2025). Agentic retrieval-augmented generation: A survey on agentic RAG. *arXiv:2501.09136*.
73. Es, S., James, J., Espinosa-Anke, L. và Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. *Proceedings of EACL 2024 (System Demonstrations)*, 150–158.
74. Traag, V. A., Waltman, L. và van Eck, N. J. (2019). From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports*, 9, 5233.
75. Wang, L. và cộng sự (2024). Multilingual E5 text embeddings: A technical report. *arXiv:2402.05672*.

76. Chen, T. và Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of ACM SIGKDD 2016*, 785–794.
77. tinixai. *vietnam-real-estates* [bộ dữ liệu]. Hugging Face Datasets, giấy phép CC BY-NC 4.0. https://huggingface.co/datasets/tinixai/vietnam-real-estates (truy cập 10/2026).
78. Kaplan, J. và cộng sự (2020). Scaling laws for neural language models. *arXiv:2001.08361*.
79. Bommasani, R. và cộng sự (2021). On the opportunities and risks of foundation models. *arXiv:2108.07258*.
80. Mnih, V. và cộng sự (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.
81. Silver, D. và cộng sự (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529, 484–489.
82. Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.
83. Novikoff, A. B. J. (1962). On convergence proofs on perceptrons. *Proceedings of the Symposium on the Mathematical Theory of Automata*, 12, 615–622.
84. Drucker, H., Burges, C. J. C., Kaufman, L., Smola, A. và Vapnik, V. (1997). Support vector regression machines. *Advances in Neural Information Processing Systems (NeurIPS)*, 9.
85. Arthur, D. và Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. *Proceedings of ACM-SIAM SODA 2007*, 1027–1035.
86. Yao, S. và cộng sự (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023*.
87. Ehrlinger, L. và Wöß, W. (2016). Towards a definition of knowledge graphs. *SEMANTiCS 2016 (Posters and Demos)*.